# Facyke RMM System

Handover & User Guide

Complete reference for all staff — from first login to advanced automation

Version 10.0 · September 2026

Prepared for

**Facyke-AI Agency**

# Facyke RMM System — Complete Handbook

**Remote Monitoring and Management Platform**

Version 1.0 — Built exclusively for Facyke-AI Agency

---

**Prepared for:** Facyke-AI Agency **System URL:** http://localhost:8501 **API URL:** http://localhost:5000 **Document version:** 9.0 (React Frontend, Cross-Platform Agent, Screenshot Capture, IoT/MQTT, Performance Hardening, Binary Deployment, Mobile Device Management, API & Token Usage Monitoring)

---

## Foreword

This handbook is the single, authoritative reference for everyone who uses, operates, or maintains the RMM system. It begins where the system begins — with installation — and walks through every feature, every screen, and every action you will ever need to perform.

This document is written in plain language. Technical jargon is explained when first introduced. Whether you are setting up the server for the first time, handling your first support ticket, or performing a monthly billing run, the relevant chapter will tell you exactly what to do and why.

**Who should read what:**

| Role | Start Here | Then Read |
|---|---|---|
| System installer / IT admin setting up for first time | Part I (Installation) | Parts II, VII |
| New employee — first week on the job | Part II (Getting Started) | Part III, Chapter 15 |
| IT Support staff (helpdesk, tickets) | Part II, Part III, Chapter 15 | Part IV (Management) |
| Technician (devices, patches, scripts) | Parts II–VI | Part VIII (Technical) |
| Manager (reports, billing) | Parts II, VII | Part III overview |
| Administrator (users, system health) | All parts | Part VIII |
| Developer / maintainer | Parts I, VIII | Everything |

**Callout conventions used throughout:**

> **NOTE:** Background information or extra context.

> **TIP:** A smarter or faster way to do something.

> **WARNING:** Something that can cause problems or is hard to reverse.

> **IMPORTANT:** Information you must not skip.

---

# Table of Contents


**PART I — INSTALLATION AND SETUP**

- Chapter 1: System Requirements
- Chapter 2: Installing Prerequisites
- Chapter 3: Setting Up the RMM Application
- Chapter 4: First-Time Configuration
- Chapter 5: Starting All Services
- Chapter 6: Deploying the Agent on Managed Machines
- Chapter 7: First-Time Setup Walkthrough
- Chapter 7a: Docker Deployment (Alternative Installation)

**PART II — GETTING STARTED**

- Chapter 8: What is the RMM System?
- Chapter 9: Logging In and Navigation
- Account Lockout and Failed Login Protection
- Forgot Your Password — Self-Service Reset
- Chapter 9a: Multi-Factor Authentication (MFA)
- Chapter 9b: My Profile Page
- Chapter 10: Your Role and What You Can Access

**PART III — MONITORING**

- Chapter 11: The Dashboard — Your Command Center
- Chapter 12: Devices — Monitoring Your Fleet
- Chapter 13: Alerts — Staying Ahead of Problems
- Chapter 14: App Center — Software Inventory
- Chapter 15: Network Discovery

**PART IV — MANAGEMENT**

- Chapter 16: Managing Tickets
- Chapter 17: Working with Customers
- Chapter 18: Automation Profiles

**PART V — PATCHING**

- Chapter 19: OS Patch Management
- Chapter 20: Software Patches

**PART VI — TOOLS**

- Chapter 21: Scripts — Running Custom Automation
- Chapter 22: Disk Management
- Chapter 23: Maintenance Actions

**PART VII — BUSINESS**

- Chapter 24: Reports
- Chapter 25: Billing
- Chapter 26: Administration
- Account Lockout and Unlock

- Deactivating, Reactivating, and Deleting Users
- Account Security Policies (Lockout, Password Expiry, Dormant Accounts, Login Anomaly Detection)
- Regional Settings (Currency and Timezone)

**PART VIII — TECHNICAL REFERENCE**

- Chapter 27: System Architecture
- Chapter 28: Script Writing Guide
- Chapter 29: Alert Rule Design
- Chapter 30: Automation Profile Design
- Chapter 31: User Roles and Permissions Matrix
- Chapter 32: Common Troubleshooting
- Chapter 33: API Input Validation
- Chapter 34: Continuous Integration (CI/CD)
- Chapter 35: Load Testing

**PART IX — ADVANCED FEATURES**

- Chapter 36: Support Inbox — Email-to-Ticket
- Chapter 37: Remote Terminal
- Chapter 38: Real-time Event Feed
- Chapter 39: Agent Auto-Update

**PART X — EXTENDED DEPLOYMENT AND OPERATIONS**

- Chapter 40: Frozen Executable Agent Deployment (USB / No-Python Machines)
- Chapter 41: Agent Resilience — Unicode Safety, Command Timeout, and Stuck Command Recovery
- Chapter 42: Client Portal — Self-Service Ticket Access for Client Users
- Chapter 43: Departments — Grouping Users and Tickets
- Chapter 44: AI Assistant — Contextual Guidance on Every Page

**PART XI — COMMERCIAL AUDIT ADDITIONS**

- Chapter 45: API Documentation (Swagger/OpenAPI)
- Chapter 46: Alert Webhooks — Slack, Teams, Custom
- Chapter 47: Configurable SLA Policies
- Chapter 48: Billing Automation — Recurring Invoices
- Chapter 49: Database Backup and Recovery
- Chapter 50: GDPR Compliance — Data Export and Erasure

**PART XII — NEW FEATURES AND DEPLOYMENT OPTIONS**

- Chapter 51: React Frontend — Alternative Web Interface
- Chapter 52: Agent Binary Deployment (No Python Required)
- Chapter 53: Remote Screenshot Viewer
- Chapter 54: IoT Sensor Monitoring
- Chapter 55: Mobile Device Management — Managing Phones Without an Agent

**PART XIII — GOVERNANCE AND COST CONTROL**

- Chapter 56: API & Token Usage Monitoring — Finding and Controlling Runaway Usage

**PART XIV — QUALITY, TESTING, AND SECURITY**

- Chapter 57: Consistent List Pagination
- Chapter 58: Automated Test Coverage — What's Verified on Every Change

- Chapter 59: Security Hardening — What Changed and Why

**Appendix A: Glossary**

**Appendix B: Quick Reference Cards**

---

# PART I — INSTALLATION AND SETUP

# Chapter 1: System Requirements

## Who uses this chapter

The person responsible for installing and hosting the RMM system — typically a senior technician, IT administrator, or the developer who built the system.

## What you need before you begin

This chapter describes the hardware and software requirements for the machine that will **host** the RMM system (the server). This is not the managed client machine — those requirements are in Chapter 6.

## Minimum Hardware (RMM Server)

| Component | Minimum | Recommended |
|---|---|---|
| Processor | 2 cores, 2.0 GHz | 4 cores, 3.0 GHz+ |
| RAM | 4 GB | 8 GB |
| Disk Space | 20 GB free | 50 GB+ (for logs, reports, database growth) |
| Network | 100 Mbps LAN | 1 Gbps LAN + internet access |
| OS | Windows 10 (64-bit) | Windows 10/11 or Windows Server 2019/2022 |

## Required Software

The following software must be installed on the RMM server before the application will run:

| Software | Version | Purpose |
|---|---|---|
| Python | 3.11 or later | Runs the API, dashboard, and agent |
| PostgreSQL | 14 or later | Primary database |
| Redis (Memurai for Windows) | 6.x or later | Background task queue |
| pip | Included with Python | Python package installer |
| Git (optional) | Any recent | For cloning the repository |

## Network Ports Used

| Port | Service | Direction |
|---|---|---|
| 8501 | Streamlit Dashboard | Inbound from users' browsers |
| 5000 | Flask API | Inbound from dashboard and agents |
| 5432 | PostgreSQL | Localhost only (internal) |
| 6379 | Redis/Memurai | Localhost only (internal) |

**NOTE:** If agents will connect from remote networks, port 5000 must be accessible through any firewall between the RMM server and those networks.

**WARNING:** This system is designed for internal/LAN deployment. Do not expose port 5000 or 8501 to the public internet without first adding a reverse proxy (e.g., nginx) with HTTPS.

# Chapter 2: Installing Prerequisites

## Step 1: Install Python 3.11+

1. Open a browser and go to **python.org/downloads**.
2. Download the latest Python 3.11.x or 3.12.x Windows installer (64-bit).
3. Run the installer.
4. On the first screen, check the box **"Add Python to PATH"**. This is essential.
5. Click **Customize installation**.
6. Ensure **pip** is checked.
7. On the Advanced Options screen, check **"Install for all users"**.
8. Click **Install**.
9. When complete, click **Disable path length limit** if prompted.
10. Open PowerShell and verify the installation:

```
python --version
pip --version
```

Both commands should display version numbers. If they display errors, Python was not added to PATH — reinstall and ensure step 4 is completed.

## Step 2: Install PostgreSQL

1. Go to **postgresql.org/download/windows** and download the installer for PostgreSQL 14, 15, or 16.
2. Run the installer.
3. Accept all defaults for the installation directory.
4. When prompted for a **superuser password**, set a strong password for the `postgres` account. **Write this down** — you will need it in the next step.
5. Leave the default port as **5432**.
6. Leave the default locale.
7. Complete the installation. Uncheck "Launch Stack Builder" at the end.

**Create the RMM database and user:**

1. Open the **SQL Shell (psql)** application that was installed with PostgreSQL.
2. Press Enter to accept all defaults at the prompts until you reach the password prompt.
3. Enter the superuser password you set in step 4.
4. At the `postgres=#` prompt, run these commands exactly as shown (type each line and press Enter):

```
CREATE USER rmm_app WITH PASSWORD 'your_secure_password_here';
CREATE DATABASE rmmdb OWNER rmm_app;
GRANT ALL PRIVILEGES ON DATABASE rmmdb TO rmm_app;
\q
```

Replace `your_secure_password_here` with a strong password of your choice. **Write this down** — you will need it in Chapter 4.

> **NOTE:** The PostgreSQL Windows service starts automatically after installation. You can verify it is running in **Windows Services** (search "Services" in Start menu) — look for "postgresql-x64-14" or similar.

### Step 3: Install Redis via Memurai

Memurai is a Redis-compatible server designed for Windows. It is the recommended Redis implementation for this system on Windows.

1. Go to **memurai.com/get-memurai** and download the free Developer edition installer.
2. Run the installer. Accept all defaults.
3. Memurai installs as a Windows service and starts automatically.
4. Verify Redis is running — open PowerShell and run:

```
Test-NetConnection -ComputerName localhost -Port 6379
```

You should see `TcpTestSucceeded : True`.

> **TIP:** If you already have an existing Redis installation or use another Redis-compatible server, you can use that instead. The only requirement is a Redis server accessible at localhost:6379 (or update the REDIS_URL environment variable accordingly).

---

# Chapter 3: Setting Up the RMM Application

## Step 1: Obtain the Application Files

If you have the project as a ZIP file:

1. Copy the ZIP to a suitable directory, e.g., `C:\RMM\`.
2. Right-click the ZIP → **Extract All** → extract to `C:\RMM\RemoteManagementSystem\`.

If you are cloning from a Git repository:

```
git clone <repository-url> C:\RMM\RemoteManagementSystem
```

The resulting directory structure should look like:

```
RemoteManagementSystem\
■■■ api\
■■■ agent\
■■■ dashboard\
■■■ scripts_library\
■■■ build_pdf.py
■■■ CLAUDE.md
```

## Step 2: Install API Dependencies

1. Open PowerShell and navigate to the `api` folder:

```
Set-Location C:\RMM\RemoteManagementSystem\api
```

1. Create a Python virtual environment:

```
python -m venv venv
```

1. Activate the virtual environment:

```
.\venv\Scripts\Activate.ps1
```

Your prompt should now show `(venv)` at the beginning.

1. Install all required packages:

```
pip install -r requirements.txt
```

This will install Flask, SQLAlchemy, Celery, JWT, bcrypt, and all other API dependencies. It may take 2–3 minutes.

> **WARNING:** If PowerShell shows "execution of scripts is disabled", run this command first:
> `Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser`

## Step 3: Install Dashboard Dependencies

1. Open a second PowerShell window and navigate to the `dashboard` folder:

```
Set-Location C:\RMM\RemoteManagementSystem\dashboard
```

1. Create and activate a virtual environment:

```
python -m venv venv
.\venv\Scripts\Activate.ps1
```

1. Install dependencies:

```
pip install -r requirements.txt
```

This installs Streamlit, Plotly, Pandas, and requests.

### Step 4: Install Agent Dependencies

On the RMM server (you can also do this on each managed machine later):

1. Navigate to the `agent` folder:

```
Set-Location C:\RMM\RemoteManagementSystem\agent
```

1. Create and activate a virtual environment:

```
python -m venv venv
.\venv\Scripts\Activate.ps1
```

1. Install dependencies:

```
pip install -r requirements.txt
```

This installs psutil, requests, pywin32, wmi, and schedule.

---

# Chapter 4: First-Time Configuration

## Creating the API Environment File

The API requires a configuration file called `.env` inside the `api\` directory. This file holds sensitive settings such as the database password and secret keys.

1. In the `api\` folder, create a new file named `.env` (note: no extension, just `.env`).
2. Open it with a text editor (Notepad, VS Code, etc.) and paste the following template:

```
SECRET_KEY=replace-with-a-long-random-string-at-least-32-chars
DATABASE_URL=postgresql://rmm_app:your_secure_password_here@localhost:5432/rmmdb
JWT_SECRET_KEY=replace-with-another-long-random-string-at-least-32-chars
ORG_REGISTRATION_TOKEN=replace-with-a-unique-org-token-for-agent-registration
REDIS_URL=redis://localhost:6379/0
CELERY_BROKER_URL=redis://localhost:6379/0
CELERY_RESULT_BACKEND=redis://localhost:6379/1
SUPERADMIN_PASSWORD=YourSuperAdminPassword123
SUPERADMIN_EMAIL=superadmin@rmm.local
CORS_ORIGINS=http://localhost:8501
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=
SMTP_PASSWORD=
SMTP_FROM=RMM System <noreply@yourcompany.com>
```

1. Make the following substitutions:

| Placeholder | Replace with |
|---|---|
| `replace-with-a-long-random-string-at-least-32-chars` | A random string of 32+ characters (e.g., `d7f3a1c9e8b2f4d6a7c3e9f1b5d8a2c4`) |
| `your_secure_password_here` | The PostgreSQL password you set in Chapter 2, Step 2 |
| `replace-with-another-long-random-string-at-least-32-chars` | A different random string of 32+ characters |
| `replace-with-a-unique-org-token-for-agent-registration` | A unique token agents use to register (e.g., `rmm-prod-2026-companyname-abc123`) |
| `YourSuperAdminPassword123` | A strong password (min 10 chars) for the built-in superadmin account. **Required** — the API will refuse to start without this set. |

**IMPORTANT:** The `ORG_REGISTRATION_TOKEN` is what prevents unauthorized agents from registering with your RMM server. Use a long, unique, hard-to-guess value. You will need this same token when configuring the agent's `config.ini` file in Chapter 6.

**WARNING:** Never commit the `.env` file to version control (Git). It contains secrets. The `.gitignore` file already excludes it.

## Configuring Email Notifications (Optional)

If you want the system to send email alerts when critical thresholds are crossed, fill in the SMTP settings:

- For Gmail: use `smtp.gmail.com`, port `587`, and your Gmail address. For the password, use a Google **App Password** (not your regular Gmail password) — generate one at **myaccount.google.com/apppasswords**.
- For Microsoft 365: use `smtp.office365.com`, port `587`.
- For a local SMTP relay: use your relay server's hostname and port.

If left blank, the system operates normally but sends no email notifications.

## Initializing the Database

With the virtual environment activated in `api\`:

1. Run the database seeder to create all tables and the first admin user:

```
Set-Location C:\RMM\RemoteManagementSystem\api
.\venv\Scripts\Activate.ps1
python seed.py
```

1. The seed script will:

- Create all database tables (devices, users, tickets, alerts, etc.)
- Create a default admin user
- Create a sample default customer
- Seed the 7 built-in PowerShell scripts into the script library

1. The output will display the **admin email and password** created. Note these immediately — this is your first login credential.

**TIP:** If the seed script fails with a database connection error, verify that PostgreSQL is running and that the `DATABASE_URL` in your `.env` file contains the correct password and database name.

---

## Chapter 5: Starting All Services

The RMM system consists of five separate processes that must all be running for full functionality. Each runs in its own terminal window.

### Service Startup Order

**Always start services in this order.** Starting them out of order can cause startup errors.

```
1. PostgreSQL (Windows Service — starts automatically)
2. Redis/Memurai (Windows Service — starts automatically)
3. Flask API
4. Celery Worker
5. Celery Beat
6. Streamlit Dashboard
7. Agent (on each managed machine)
```

### Starting the Flask API (Terminal 1)

```
Set-Location C:\RMM\RemoteManagementSystem\api
.\venv\Scripts\Activate.ps1
python app.py
```

Expected output:

```
* Running on http://0.0.0.0:5000
* Debug mode: on
```

Leave this terminal open. Do not close it — closing it stops the API.

### Starting the Celery Worker (Terminal 2)

```
Set-Location C:\RMM\RemoteManagementSystem\api
.\venv\Scripts\Activate.ps1
celery -A tasks.celery_app worker --pool=solo -l info
```

Expected output: Lines showing `[celery@HOSTNAME] ready.`

> **NOTE:** The `--pool=solo` flag is required on Windows. Celery's default pool type (`prefork`) is not compatible with Windows.

### Starting Celery Beat (Terminal 3)

```
Set-Location C:\RMM\RemoteManagementSystem\api
.\venv\Scripts\Activate.ps1
celery -A tasks.celery_app beat -l info
```

Expected output: Lines showing `beat: Starting...` and scheduled tasks being logged.

Celery Beat is the scheduler. It triggers automatic tasks such as evaluating alert rules every 60 seconds and running scheduled automation profiles.

### Starting the Dashboard (Terminal 4)

```
Set-Location C:\RMM\RemoteManagementSystem\dashboard
.\venv\Scripts\Activate.ps1
streamlit run app.py
```

Expected output:

```
You can now view your Streamlit app in your browser.
Local URL: http://localhost:8501
Network URL: http://192.168.x.x:8501
```

Open your browser and go to **http://localhost:8501**. You should see the RMM login page.

### Verifying All Services Are Running

Open a new PowerShell window and run:

```
netstat -ano | findstr ":5000 :8501 :5432 :6379"
```

You should see lines for each port, confirming all services are listening.

### Quick Health Check via Browser

1. Open: **http://localhost:5000/api/health**
2. You should see a JSON response like: `{"status": "ok", "db": true, "redis": true, "version": "1.0.0"}`. A `"status": "degraded"` response means PostgreSQL or Redis is unreachable — the health endpoint now tests actual connectivity.
3. If `db` or `redis` is `false`, check that the respective service is running.

---

# Chapter 6: Deploying the Agent on Managed Machines

The agent is a lightweight Python program that runs on every machine you want to monitor. It sends metrics to the API and receives commands from it.

## What You Need on Each Managed Machine

- Windows 10 or Windows 11 (64-bit)
- Python 3.11+ with pip
- Network access to the RMM server on port 5000
- Administrator privileges to install and run the agent

## Step 1: Install Python on the Managed Machine

Follow the same steps as Chapter 2, Step 1. Verify with:

```
python --version
```

## Step 2: Copy the Agent Files

Copy the entire `agent\` folder from the RMM server to the managed machine. You can use a USB drive, a network share, or any other transfer method.

Suggested destination on the managed machine: `C:\RMM\agent\`

## Step 3: Install Agent Dependencies

On the managed machine, open PowerShell as **Administrator**:

```
Set-Location C:\RMM\agent
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

## Step 4: Configure the Agent

Open `C:\RMM\agent\config.ini` in a text editor. You will see:

```
[api]
url = http://localhost:5000
org_token =

[agent]
device_id =
agent_token =
heartbeat_interval = 60
software_interval = 21600
version = 1.0.0

[logging]
level = INFO
```

```
file = rmm_agent.log
```

Make these changes:

1. `url` — change `http://localhost:5000` to the actual IP address or hostname of your RMM server. Example: `http://192.168.1.100:5000`
2. `org_token` — enter the `ORG_REGISTRATION_TOKEN` value. Find it in either:

- The dashboard: **Admin** → **System Info** tab → **Agent Enrollment Token** card (click Reveal)
- Or directly in the `.env` file on the server: `ORG_REGISTRATION_TOKEN=...`

3. Leave `device_id` and `agent_token` blank — the agent fills these in automatically on first registration.

Example of a completed config:

```
[api]
url = http://192.168.1.100:5000
org_token = rmm-prod-2026-companyname-abc123

[agent]
device_id =
agent_token =
heartbeat_interval = 60
software_interval = 21600
version = 1.0.0

[logging]
level = INFO
file = rmm_agent.log
```

### Step 5: Run the Agent

On the managed machine, open PowerShell as **Administrator** and run:

```
Set-Location C:\RMM\agent
.\venv\Scripts\Activate.ps1
python rmm_agent.py
```

The first time it runs, the agent will register itself with the RMM server and display:

```
Device registered successfully. Device ID: <uuid>
Starting heartbeat loop...
```

After this, the agent sends a heartbeat every 60 seconds. You can now go to the RMM dashboard and see this machine appear in the Devices page.

### Step 6: Set Up Auto-Start (Optional but Recommended)

To ensure the agent restarts automatically when the machine reboots:

1. Create a batch file `start_agent.bat` in `C:\RMM\agent\`:

```
@echo off
cd C:\RMM\agent
call venv\Scripts\activate.bat
python rmm_agent.py
```

1. Add this batch file to Windows Task Scheduler or place a shortcut in the Startup folder (`Win+R` → type `shell:startup`).

**IMPORTANT:** The agent should run as Administrator for full capability. Task Scheduler can be configured to run the task with highest privileges.

## Verifying Agent Registration

1. Go to the RMM dashboard at http://localhost:8501.
2. Log in.
3. Click **Devices** in the sidebar.
4. The newly registered machine should appear with a green online dot.
5. Within 60 seconds, its CPU, RAM, and disk metrics will begin populating.

## Deploying the Agent on Other WiFi/LAN Machines

Use this procedure to monitor any Windows, Linux, or macOS machine on the same network — without physically accessing the RMM server machine.

**NOTE:** The API already binds to `0.0.0.0:5000`, so any machine on the same LAN can reach it. The only change needed is pointing the agent at the server's LAN IP instead of localhost.

**Step 1: Get the server's LAN IP**

Go to **Admin** → **System Info** tab → **Server IP Addresses** card. Each LAN IP is displayed in a copyable code block. Alternatively, check your router's DHCP client list for the hostname of the RMM server machine.

**Step 2: Copy the agent folder to the target machine**

Copy the entire `agent\` folder to the target machine using a USB drive, network share, email, or git clone. Suggested destination: `C:\RMM\agent\`

**Step 3: Get the org token**

Go to **Admin** → **System Info** tab → **Agent Enrollment Token** card → click **Reveal**. Copy the token.

**Step 4: Run the setup helper**

On the target machine, open PowerShell and run:

```
Set-Location C:\RMM\agent
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
python setup_agent.py 192.168.x.x <org_token>
```

Replace `192.168.x.x` with the server's actual LAN IP and `<org_token>` with the token from Step 3. The script updates `config.ini` automatically and clears any previous device ID so the agent registers fresh.

**Step 5: Start the agent**

```
python rmm_agent.py
```

Within 60 seconds the device appears in the **Devices** tab with a green online dot.

> **TIP:** Follow Step 6 in this chapter (Task Scheduler auto-start) to ensure the agent restarts after reboots on WiFi machines too.

---

## Chapter 7: First-Time Setup Walkthrough

After all services are running and at least one agent has checked in, complete these setup steps to make the system fully operational.

### Step 1: Log In as Administrator

1. Go to **http://localhost:8501**.
2. Log in with the admin credentials output by `seed.py` in Chapter 4.
3. You should land on the RMM home page showing the stat cards.

### Step 2: Create Your First Customer

Every device must belong to a customer. Even if you are managing your own organization's devices, you still need a customer record.

1. Click **Customers** in the sidebar.
2. Click **+ New Customer**.
3. Enter your organization or first client's name.
4. Fill in contact email and phone.
5. Select the appropriate tier (standard, premium, or enterprise).
6. Click **Save**.

### Step 3: Assign Devices to the Customer

1. Click **Devices** in the sidebar.
2. Find the newly registered device (it will appear with no customer assigned initially).
3. Click the device to expand it.
4. Assign it to the customer you just created.

### Step 4: Create Your First Alert Rules

Without alert rules, no alerts will ever fire — the system needs rules to know when something is wrong.

1. Click **Alerts** in the sidebar.
2. Click the **Alert Rules** tab.
3. Create at minimum these four rules (full instructions in Chapter 13):

- CPU critical: metric=cpu, operator=gt, threshold=90, severity=critical, cooldown=15
- Disk critical: metric=disk, operator=gt, threshold=90, severity=critical, cooldown=60
- RAM critical: metric=ram, operator=gt, threshold=90, severity=critical, cooldown=15
- Device offline: metric=offline, severity=warning, cooldown=60

### Step 5: Create Additional User Accounts

1. The `Admin` → `Users` tab shows all current users.
2. Use the create user form to add accounts for each team member.
3. Assign the appropriate role: **admin** for administrators, **technician** for IT staff, **viewer** for managers or clients.
4. Share credentials with each user and instruct them to change their password after first login.

### Step 6: Create Your First Automation Profile

A basic automation profile ensures devices are regularly maintained without manual intervention.

1. Click **Automation** in the sidebar.
2. Click **Create / Edit Profile**.
3. Name it "Weekly Standard Maintenance".
4. Set schedule: weekly, Sunday, 02:00.
5. Enable OS patches (critical + security only), Restore Point, and Delete Temp Files.
6. Save the profile.

The system is now fully operational. The following parts of this handbook cover day-to-day usage.

---

# Chapter 7a: Docker Deployment (Alternative Installation)

## What it is

Docker Compose is an alternative to the manual installation in Chapters 2–6. It starts all six services (PostgreSQL, Redis, Flask API, Celery worker, Celery beat, and Streamlit dashboard) with a single command. This is the recommended method for production deployments.

## What you need

- **Docker Desktop** installed on the server. Download from docker.com/products/docker-desktop.
- The RMM project folder on the server (e.g. `C:\RMM\RemoteManagementSystem\`).

**NOTE:** Docker includes PostgreSQL, Redis, and Python inside containers. No separate installation of those is required.

## Step 1: Create the API Environment File

Create `api\.env` as described in Chapter 4. When using Docker, the database host must be `db` (the service name), not `localhost`:

```
SECRET_KEY=replace-with-32-char-random-string
DATABASE_URL=postgresql://rmm_app:changeme@db:5432/rmmdb
JWT_SECRET_KEY=replace-with-another-32-char-string
ORG_REGISTRATION_TOKEN=replace-with-unique-token
SUPERADMIN_PASSWORD=YourSuperAdminPassword123
SUPERADMIN_EMAIL=superadmin@rmm.local
CORS_ORIGINS=http://localhost:8501
REDIS_URL=redis://redis:6379/0
CELERY_BROKER_URL=redis://redis:6379/0
CELERY_RESULT_BACKEND=redis://redis:6379/1
```

**IMPORTANT:** Use `@db:5432` not `@localhost:5432` in `DATABASE_URL`.

## Step 2: Start All Services

```
Set-Location C:\RMM\RemoteManagementSystem
docker-compose up -d
```

Wait 15–20 seconds, then verify all containers are running:

```
docker-compose ps
```

All six services should show status `Up`.

### Step 3: Verify

Open **http://localhost:8501** — the login page should appear.

Health check: `http://localhost:5000/api/health` should return `{"db": true, "redis": true, "status": "ok", "version": "1.0.0"}`.

## Useful Commands

```
# View logs
docker-compose logs -f api
docker-compose logs -f celery_worker

# Stop all services (data preserved)
docker-compose down

# Stop and delete all data (IRREVERSIBLE)
docker-compose down -v

# Rebuild after code changes
docker-compose build api dashboard
docker-compose up -d
```

**WARNING:** `docker-compose down -v` permanently deletes the database. All devices, tickets, users, and reports are lost.

## Docker vs Manual Installation

| Situation | Recommended method |
|---|---|
| Production server | Docker (Chapter 7a) |
| Clean lab / demo | Docker (Chapter 7a) |
| Local dev with debugger | Manual (Chapters 2–6) |
| Reusing existing PostgreSQL/Redis | Manual (Chapters 2–6) |

---

# PART II — GETTING STARTED

# Chapter 8: What is the RMM System?

## What it is

RMM stands for Remote Monitoring and Management. An RMM system is a platform used by IT teams to monitor, manage, and support computers and servers — often for multiple clients — from a single central dashboard.

Think of it like a control tower for an airport. Every aircraft (your managed devices) sends constant status updates. The tower (the RMM) watches all of them, raises alarms when something is wrong, and allows controllers (your IT staff) to take action remotely.

This system is modeled after industry tools like NinjaOne and ConnectWise Automate. It is composed of four main components:

### 1. The Dashboard (Streamlit frontend)

A web-based interface running at http://localhost:8501. This is what you see when you open the system in your browser. It has 16 pages covering everything from device monitoring to billing.

### 2. The API (Flask backend)

Running at http://localhost:5000, this is the engine that powers the dashboard. It stores and retrieves all data, handles authentication, and processes commands.

### 3. The Agent (Python)

A small program installed on each managed Windows machine. The agent runs continuously, sending a heartbeat to the API every 60 seconds containing:

- CPU usage percentage
- RAM usage percentage
- Disk usage per drive
- System uptime in seconds
- Installed software inventory (every 6 hours)

The agent also receives commands from the API — such as "run this script" or "reboot" — and executes them.

### 4. The Database (PostgreSQL)

All data — devices, tickets, alerts, customers, scripts, patches, billing — is stored in a PostgreSQL database called `rmmdb`.

## How it fits your workflow

When a client's machine has a problem — say the hard drive is nearly full — the following chain occurs automatically:

1. The agent reports disk usage of 92% in its heartbeat.
2. The API evaluates this against your alert rules. If a rule says "disk > 90% → critical", an alert is created.
3. The alert appears on the Alerts page and on the Dashboard.

4. An email notification is sent if SMTP is configured.
5. A ticket is automatically created if the alert rule has "auto-create ticket" enabled.
6. A technician investigates, runs a cleanup script, and resolves the ticket.

Steps 1 through 5 happen with zero human involvement. That is the power of an RMM.

---

# Chapter 9: Logging In and Navigation

## Who uses this chapter

Everyone. This is the entry point to every page in the system.

## Supported Devices and Screen Sizes

The dashboard is responsive and adapts to the following screen sizes:

| Device | Screen Width | Experience |
|---|---|---|
| Desktop / large monitor | >1024px | Full layout — sidebar expanded, multi-column grids |
| Tablet (landscape) | 768–1024px | Reduced padding, slightly smaller metrics |
| Mobile / tablet (portrait) | ≤768px | All columns stack vertically, full-width login card, 44px touch targets on buttons and inputs |
| Small phone | ≤480px | Further padding and font-size reduction |

On mobile, the sidebar collapses automatically — tap the ■ **menu button** (top left) to open it. All navigation works the same way as on desktop.

**NOTE:** The dashboard runs inside Streamlit's web runtime, which sets its own viewport. On some older mobile browsers, responsive breakpoints may not trigger as expected. Chrome and Safari on iOS/Android (current versions) are fully supported.

## Step-by-step: Logging In

1. Open your web browser (Chrome, Firefox, or Edge recommended).
2. Go to: **http://localhost:8501**
3. You will see the RMM login page — dark green background with a centered login card.
4. In the **Email address** field, type your full email address.
5. In the **Password** field, type your password. Characters appear as dots.
6. Click the green **Sign In** → button.
7. If your account has **Multi-Factor Authentication (MFA)** enabled, you will see a second screen asking for a 6-digit code from your authenticator app. Enter the code and click **Verify** →. See Chapter 9a for full MFA details.
8. Correct credentials (and MFA code if required) take you to the RMM Dashboard.
9. "Invalid credentials" means wrong email or password. Check Caps Lock.

**NOTE:** Your session is maintained in browser memory. If you share a page URL, the recipient must log in with their own credentials.

**NOTE:** If you have forgotten your password, use the **"Forgot your password?"** link on the login page. See the subsection below for details.

## First Login — Forced Password Change

If an administrator created your account with the **"Require password change on first login"** option enabled, the system will intercept your first login and display a **Set New Password** screen instead of the normal dashboard. You cannot navigate to any other page until you complete the password change.

1. Enter a new password (minimum 8 characters).
2. Confirm the new password.
3. Click **Set Password** →.
4. You are immediately taken to the dashboard. The temporary password no longer works.

## Understanding the Interface Layout

After logging in, the screen has two sections:

### The Sidebar (left panel)

Your navigation menu. On desktop it is always visible on the left. On mobile and tablet it collapses — tap the ■ icon in the top-left corner to open it, and tap anywhere on the main content area to close it. Pages are grouped:

- **MONITORING:** Overview (Dashboard), Devices, Alerts
- **MANAGEMENT:** Tickets, Customers, Automation
- **PATCHING:** OS Patches, Software Patches
- **TOOLS:** Scripts, Disk Management, Maintenance, Network Discovery
- **BUSINESS:** Reports, Billing, Admin

At the top of the sidebar: your name, email, and colored role badge. At the bottom: the **Sign Out** button.

### The Main Content Area (right panel)

Each page's content appears here. Every page has a title, optional filters, action buttons, and the main data view.

## Step-by-step: Navigating Between Pages

1. Look at the sidebar on the left.
2. Find the section heading for the page you want.
3. Click the page name — for example, **Devices** under MONITORING.
4. The right panel loads that page. Your session is maintained as you navigate.

## Practical Example: First Login as a New Employee

Sarah is a new junior technician. It is her first day.

1. Her manager gave her credentials: `sarah@company.com` / `Welcome123!`
2. She opens Chrome and goes to http://localhost:8501.
3. She enters her email and password.
4. She clicks **Sign In** →.
5. She sees the RMM Dashboard with "Sarah" displayed at the top of the sidebar alongside a yellow TECHNICIAN badge.
6. She clicks **Devices** to see all managed machines.

## Account Lockout — What Happens When Too Many Failed Attempts Occur

The system automatically locks an account after **3 consecutive failed login attempts**. This protects against password-guessing attacks.

**What you will see:**

- On the 3rd failed attempt, the login page shows a red error message:

`"Account locked. Try again after HH:MM:SS"` — the remaining lockout time is displayed.

- The lockout lasts **5 minutes** from the last failed attempt.
- After 5 minutes, the lockout clears automatically. Try logging in again normally.
- If your account is locked, contact your system administrator. They can unlock it immediately from the Admin panel without waiting for the 5 minutes to expire.
- All admin users receive an email notification when any account is locked, so administrators are alerted even if you do not contact them.

**NOTE:** The superadmin account is exempt from the lockout policy and can never be locked out.

**NOTE:** The login page is rate-limited: a maximum of 10 login attempts per minute per IP address is enforced, regardless of which account is being targeted.

## Forgot Your Password? — Self-Service Password Reset

If you have forgotten your password, you can reset it yourself without contacting an administrator, provided your email address is valid in the system.

### *Step-by-step: Resetting a Forgotten Password*

1. Go to the RMM login page (http://localhost:8501).
2. Click the **"Forgot your password?"** button below the Sign In button.
3. A text field appears — enter your email address and click **Send Reset Link**.
4. The system always shows the same success message regardless of whether the email is recognised. This is intentional — it prevents attackers from discovering which email addresses are registered.
5. If your email matches an active account, you will receive an email within a few minutes containing a password reset link.
6. Click the link in the email. It opens the RMM dashboard with a full-screen **Reset Password** form.
7. Enter your new password (see password strength requirements below).
8. Confirm the new password and click **Reset Password** →.
9. Your password is updated. You are redirected to the login page. Log in with your new password.

**IMPORTANT:** The reset link expires after **1 hour**. If you do not use it in time, repeat the process.

**NOTE:** Password reset requires SMTP email to be configured on the server. If you do not receive the email, check your spam folder. If still not received, ask your administrator whether SMTP is configured.

**Password strength requirements** (applies to all password changes system-wide):

- Minimum 8 characters
- At least 1 uppercase letter (A–Z)
- At least 1 number (0–9)

- At least 1 special character (`!@#$%^&*` and similar)

---

# Chapter 9a: Multi-Factor Authentication (MFA)

## What it is

Multi-Factor Authentication (MFA) adds a second verification step to the login process. After entering your email and password, you are asked for a 6-digit time-based code from an authenticator app on your phone (such as Google Authenticator, Authy, or Microsoft Authenticator). Even if someone knows your password, they cannot log in without the code.

## Who uses it

Any user can enable MFA on their own account. Administrators are strongly encouraged to enable it.

## Logging In with MFA Enabled

1. Enter your email and password on the login page and click **Sign In** →.
2. A new screen appears: **Two-Factor Authentication Required**.
3. Open your authenticator app and find the RMM entry.
4. Enter the current 6-digit code shown in the app.
5. Click **Verify** →.
6. You are taken to the RMM dashboard.

The 6-digit code refreshes every 30 seconds — use the current code shown in the app. If you click **Back**, you return to the login screen.

> **NOTE:** If you lose access to your authenticator app, contact your system administrator. They can disable MFA on your account via Admin → Users → Edit.

## Setting Up MFA on Your Account

1. Log in to the RMM dashboard.
2. Click **My Profile** in the sidebar (under ACCOUNT at the bottom).
3. On the right side of the page, find the **Multi-Factor Authentication** section.
4. The badge shows **DISABLED** in red if MFA is not yet set up.
5. Click **Enable MFA**.
6. A QR code appears along with a text backup key.
7. Open your authenticator app on your phone and scan the QR code.
8. The app will show a 6-digit code for "RMM System".
9. Enter that 6-digit code in the **Verify Code** field and click **Activate MFA**.
10. The badge changes to **ENABLED** in green. MFA is now active on your account.

> **TIP:** Copy the text key shown under the QR code and store it securely. This is your backup key if you need to add the account to a new phone.

### Disabling MFA on Your Account

1. Go to **My Profile** in the sidebar.
2. In the MFA section, the badge shows **ENABLED**.
3. Enter your current password in the confirmation field.
4. Click **Disable MFA**.
5. The badge changes to **DISABLED**.

> **WARNING:** Disabling MFA reduces your account security. Only do this if you are replacing your authenticator app or device — set MFA up again immediately after.

### Administrator: Disabling MFA for a Locked-Out User

If a user loses their authenticator app and cannot log in:

1. Go to **Admin** → **Users** tab.
2. Find the user and click **Edit**.
3. Uncheck **MFA Enabled** and save.
4. The user can now log in with just their password and re-enroll MFA from My Profile.

---

## Chapter 9b: My Profile Page

### What it is

The My Profile page is a personal settings page accessible to every logged-in user. It allows you to view your account details, change your password, and manage MFA — all without requiring admin assistance.

### Accessing My Profile

Look for **My Profile** in the sidebar under the **ACCOUNT** section at the bottom. Click it to open the page.

### Left Column — Account Details and Password

Shows your full name, email address, and role badge. Below that, the **Change Password** form lets you enter your current password and set a new one (minimum 8 characters). Click **Update Password** to save. The change takes effect immediately.

### Right Column — MFA Management

Shows the current MFA status badge (green ENABLED or red DISABLED) and the appropriate action form. See **Chapter 9a** for step-by-step MFA instructions.

---

# Chapter 10: Your Role and What You Can Access

## What it is

The system uses Role-Based Access Control (RBAC). Different users have different access depending on their role. There are five roles: **superadmin**, **admin**, **technician**, **viewer**, and **client**.

## The Five Roles Explained

### Viewer

Read-only access. Viewers can browse the dashboard, look at device metrics, read alerts, and view tickets. They cannot create tickets, run scripts, approve patches, or access the Admin page. Use this role for managers who need visibility without making changes.

### Technician

Full day-to-day operational access. Technicians can:

- Create and update tickets and comments
- Acknowledge and resolve alerts, create alert rules
- View and manage devices
- Run scripts on devices
- Approve and manage patches
- Perform maintenance actions (reboot, shutdown, etc.)
- Manage customers
- Create and edit automation profiles
- Generate reports

Technicians cannot access the Admin page, manage other users, or create invoices.

### Administrator

Complete access. Includes everything technicians do, plus:

- The Admin page (System Info, Audit Log, Users)
- User management (create, edit, deactivate users)
- Billing and invoice creation
- System-wide configuration

### Client

Customer-scoped, self-service access. Client users can:

- Submit new support tickets from the Client Portal
- View the status and full comment thread of their own tickets
- Receive technician responses on their tickets

Clients see only tickets belonging to their linked customer. They have no access to devices, alerts, metrics, billing, or any management feature. When a client logs in, they are automatically redirected to the Client Portal — they cannot navigate to any other page.

When creating a client user, an administrator must assign the **client** role and link the user to a specific **Customer** record. A client account without a linked customer will show an empty portal. See Chapter 42

for the full Client Portal setup guide.

> **TIP:** Client accounts are ideal for customer contacts who want to submit and track their own tickets without calling in. They share the same login URL as staff.

**Super Administrator**

Full system access including emergency recovery. The superadmin account exists permanently — it is re-created automatically every time the API starts if it does not exist. There is exactly one superadmin account per system. It cannot be modified or deleted through the web interface. To change its password, a server administrator must use the CLI tool (`reset_superadmin.py`). See Chapter 26 for details.

## Role Badges in the Sidebar

- **SUPERADMIN** — purple badge
- **ADMIN** — red badge
- **TECHNICIAN** — yellow/amber badge
- **VIEWER** — green badge
- **CLIENT** — blue badge

## What Happens If You Exceed Your Role

If a viewer or technician tries to access the Admin page, they see: "Admin access required. This page is restricted to admin users." The page stops loading.

For other restrictions (such as a viewer trying to create a ticket), the API enforces the rule — the action will be rejected with a permission error.

## Practical Example: Understanding What You Can Do

Mark is a technician. Monday morning he sees a critical alert.

- View alert details — **YES**
- Click Acknowledge — **YES**
- Click Resolve — **YES**
- Go to Admin page → Audit Log — **NO** (admin only)
- Create a billing invoice — **NO** (admin only)
- Generate a device report — **YES**

---

# PART III — MONITORING

# Chapter 11: The Dashboard — Your Command Center

## What it is

The Dashboard is the first page you land on after login. It is your at-a-glance health check for the entire system. There are two dashboard views:

1. **The Home Page** (`app.py`) — shows five key stat cards plus a navigation hint. This is what you see immediately after login.
2. **The Overview Page** (`01_Dashboard.py`) — a richer view with charts, a device health map, recent alerts, and an activity feed. Access it by clicking **Overview** in the sidebar.

## Who uses it

Everyone. The dashboard is the starting point for all roles.

## The Five Stat Cards

Both views display five stat cards across the top:

| Card | What it shows |
|---|---|
| Total Devices | Total registered devices in the system |
| Online | Devices currently online (heartbeat received recently) |
| Warning | Devices with at least one metric in the 75–90% range |
| Critical | Devices with a metric over 90% or with an active critical alert |
| Open Tickets | Tickets that are not yet resolved or closed |

> **TIP:** The colored accent under each card indicates health. Green = normal, amber = caution, red = action required.

## The Overview Page — Four Additional Sections

Click **Overview** in the sidebar to see the full dashboard. Below the stat cards:

### Device Status Donut Chart (left)

A circle chart dividing your fleet into: Healthy, Warning, Critical, and Offline. Hover over a slice to see the exact count.

### Device Health Map (right)

A grid of mini device cards. Each shows hostname, online/offline status (green dot = online, grey = offline), and current CPU, RAM, and disk percentages. Scan this quickly to spot problems. **Each card is clickable** — clicking navigates directly to the Devices page with that device's hostname pre-filled in the search box and a banner confirming which device was selected.

### Recent Alerts (bottom left)

The 10 most recent open alerts. Each row shows a severity color bar, the alert message, and timestamp.

**Activity Feed (bottom right)**

A log of recent system actions — logins, ticket creations, device registrations, alert acknowledgements.

## Step-by-step: Morning Health Check

This is the recommended routine to start your shift:

1. Log in at http://localhost:8501.
2. Look at the five stat cards. Note any red numbers.
3. Click **Overview** in the sidebar.
4. Scan the donut chart. If the Critical slice is non-zero, investigate.
5. Scan the Device Health Map for high CPU, RAM, or disk numbers. Click any card to jump directly to that device in the Devices page.
6. Read Recent Alerts. Are there critical alerts you have not seen before?
7. Glance at the Activity Feed for unexpected overnight activity.
8. If anything needs attention, navigate to Alerts, Devices, or Tickets from the sidebar.

## Practical Example: Client Has a Server Down

ACME Corp calls at 9am — their server is unresponsive.

1. Open the Dashboard Overview.
2. Look at the Device Health Map for any ACME device showing offline (grey dot).
3. Note the hostname of the offline device.
4. Click **Devices** in the sidebar.
5. Search for the ACME server. Check the **Last Seen** time.
6. Check **Alerts** to see if an "offline" alert fired.
7. Create a ticket to document the issue.
8. Contact ACME to say you are investigating.

**TIP:** The dashboard does not auto-refresh. Press F5 before making decisions based on the numbers you see.

---

# Chapter 12: Devices — Monitoring Your Fleet

## What it is

The Devices page shows every registered machine — every computer or server that has the agent installed and has checked in, plus any network-discovered agentless devices. This is the most information-dense page in the system.

## Who uses it

All roles. Technicians and administrators interact with it most frequently.

## OS Filter Tabs

The Devices page is divided into seven tabs, each showing a count badge:

| Tab | What it shows |
|---|---|
| **All** | Every device — agent-managed and agentless |
| **Windows** | Agent-managed Windows computers |
| **macOS** | Agent-managed Mac computers |
| **Linux** | Agent-managed Linux machines |
| **Android** | Phones and tablets discovered via network scan, identified by Android OUI |
| **iOS** | Apple phones and tablets discovered via network scan, identified by Apple OUI |
| **Agentless** | All network-discovered devices regardless of type |

## Two Device Display Modes

**Agent-managed devices** (Windows/macOS/Linux tabs) show:

- Hostname, IP address, OS, last seen, online/offline status dot
- Live CPU, RAM, and disk usage gauges
- Remote reboot and shutdown buttons
- Customer assignment control

**Agentless devices** (Android/iOS/Agentless tabs) show:

- IP address, MAC address, vendor badge (e.g., "Apple, Inc."), platform badge
- Last seen timestamp, online/offline status dot
- **Ping Now** button — triggers an immediate reachability check and updates online status
- **Edit** button — opens an inline form to manually correct the hostname, platform (windows/mac/linux/android/ios/unknown), and device type (desktop/laptop/mobile/server/unknown). Useful when automatic detection could not identify the device (see Chapter 15).
- **Delete** button — removes the agentless device record permanently
- Customer assignment control

**NOTE:** Agentless devices do not report CPU, RAM, or disk metrics. They are presence-monitored only — the system confirms they are reachable on the network, nothing more.

## Delete Confirmation

Deleting a device is a two-step process to prevent accidental removal. The first click on **Delete** changes the button to **Sure? Confirm / Cancel**. You must click **Confirm** within the same page load to proceed. Navigating away or clicking **Cancel** abandons the deletion. This applies to both agent-managed and agentless devices.

## What You See (Agent Devices)

| Column | Description |
|---|---|
| Hostname | The device's computer name (e.g., ACME-SRV01) |
| IP Address | The device's local IP address |
| OS | Operating system (e.g., Windows 10, Windows Server 2022) |
| Status | Online (green dot) or Offline (grey dot) |
| Last Seen | Timestamp of the last heartbeat |
| CPU % | Current CPU usage percentage |
| RAM % | Current RAM usage percentage |
| Disk % | Primary disk usage percentage |
| Customer | Which customer this device belongs to |

## Understanding Online vs Offline

A device is **online** if its agent has sent a heartbeat in the last few minutes. If no heartbeat has arrived — because the machine is off, the agent crashed, or there is a network problem — the device shows **offline**.

**IMPORTANT:** An offline device is not necessarily broken. It could be powered off outside business hours. Always check the Last Seen timestamp to understand how long it has been offline.

## Status Categories

| Status | Meaning |
|---|---|
| Healthy | Online, all metrics in normal range |
| Warning | Online, at least one metric 75–90% |
| Critical | Online with a metric >90%, or has an active critical alert |
| Offline | Agent not reporting — no recent heartbeat |

### Step-by-step: Viewing a Device's Detailed Metrics

1. Click **Devices** in the sidebar.
2. Find the device. Use search or filter controls if available.
3. Click the device row or name to expand it.
4. You will see:

- Full OS name and version
- Platform (Windows, Linux, macOS)
- CPU, RAM, and disk metrics
- Last seen timestamp
- Agent registration date
- Uptime

### Metrics History

Each agent-managed device has a **Metrics History** button that renders a line chart of CPU, RAM, and disk usage over time.

- By default the chart shows the **last 24 hours** of readings.
- If no data exists in the last 24 hours (e.g. the agent was recently restarted or offline for a day), the system automatically falls back to the **last 7 days** of data and displays an information banner:

> *No data in last 24 h — showing last N readings (oldest ~Xh ago). Agent may be offline.*

- The chart title changes to **"7-day usage history (agent offline)"** when the fallback is active.
- If there is no data in the last 7 days at all, the message **"No metric history available"** is shown.

> **NOTE:** The metrics history button is only available on agent-managed devices (Windows/macOS/Linux tabs). Agentless devices (phones, tablets, network-only devices) do not report metrics and have no history to display.

### Exporting Devices to CSV

A **Download CSV** button at the top of the Devices page exports all currently-filtered devices to a CSV file. The export includes: hostname, IP address, platform, online status, CPU%, RAM%, Disk%, and last seen timestamp. The active OS tab and search box filter apply before export.

### Step-by-step: Investigating a Device After an Alert

You have received an alert that ACME-SRV01 has high CPU. Here is what to do:

1. Go to **Devices** in the sidebar.
2. Find ACME-SRV01.
3. Check the CPU column — is it still high?
4. Click the device to expand details.
5. Look at the CPU history if available. Is this a spike or sustained load?
6. Check Last Seen — is the device still online?
7. If CPU is still high and device is online, consider:

- Going to **Scripts** and running a "Get Running Processes" script.
- Going to **Maintenance** and scheduling a reboot if agreed with the customer.

8. Create or update a ticket with your findings.

### Practical Example: New Technician Surveys Their Assigned Clients

James is new, assigned to three small business clients. First day:

1. Go to **Customers** in the sidebar. Note the three client names.
2. Go to **Devices**.
3. Look for devices with those customer names in the Customer column.
4. Note which are online, which are offline, any with high metrics.
5. Go to **Dashboard Overview** → Device Health Map for a visual summary.

> **TIP:** If a device has not been seen for several hours during business hours, contact the customer proactively. Do not wait for them to call you.

---

# Chapter 13: Alerts — Staying Ahead of Problems

## What it is

The Alerts page manages all active system notifications. An alert is generated when a device's metric crosses a threshold you defined. Alerts are your early warning system.

The page has two tabs: **Active Alerts** and **Alert Rules**.

## Who uses it

All roles can view alerts. Technicians and administrators can acknowledge, resolve, and create rules.

## The Active Alerts Tab

**Four stat cards at the top:**

- **Open Alerts:** Total unresolved
- **Critical:** How many are critical severity
- **Acknowledged:** Seen but not yet resolved
- **Warning:** How many are warning severity

**The severity filter** lets you show only critical, warning, or info alerts.

**Each alert row** (when expanded) shows:

- A colored severity bar (red = critical, amber = warning, blue = info)
- Severity and status badges
- Which device triggered it
- The timestamp
- Full alert message
- Two action buttons: **Acknowledge** and **Resolve**

## Alert Severity

| Severity | Color | Meaning |
|---|---|---|
| critical | Red | Immediate action required — service may be impacted |
| warning | Amber | Attention needed soon — degraded performance |
| info | Blue | Informational — no immediate action needed |

### Alert Status

| Status | Meaning |
|---|---|
| open | New alert, not yet seen |
| acknowledged | A technician has seen it and is dealing with it |
| resolved | The underlying issue has been fixed |

## Exporting Alerts to CSV

A **Download CSV** button at the top of the Active Alerts tab exports all currently-filtered alerts to a CSV file with: device, rule name, severity, status, and triggered date.

## Step-by-step: Acknowledging an Alert

Acknowledging means "I have seen this and I am dealing with it." It does not mean the problem is fixed.

1. Click **Alerts** in the sidebar.
2. Find the alert. Critical alerts appear first.
3. Click the alert to expand it.
4. Click **Acknowledge**.
5. Confirmation: "Alert acknowledged." The badge changes from OPEN to ACKNOWLEDGED.

## Step-by-step: Resolving an Alert

Resolve only after actually fixing the underlying issue.

1. Find and expand the alert.
2. Verify the problem is fixed (check device metrics on the Devices page).
3. Click **Resolve**.
4. Confirmation: "Alert resolved." The alert is removed from the Active list.

## The Alert Rules Tab

Without rules, no alerts are ever generated. Rules define when alerts are created.

Each rule shows:

- Rule name
- Metric monitored (cpu, ram, disk, battery, offline)
- Condition (`gt 90` means "greater than 90%")
- Cooldown period (minutes before the rule can fire again for the same device)
- Severity level
- Active/Inactive status and a toggle

## Step-by-step: Creating an Alert Rule

1. Click **Alerts** → **Alert Rules** tab.
2. Scroll to the **Create Alert Rule** section.
3. Fill in the form:

- **Rule Name:** e.g., "High CPU Warning"
- **Severity:** critical, warning, or info
- **Metric:** cpu, ram, disk, battery, or offline
- **Operator:** `gt` (greater than), `gte` (≥), `lt` (less than), `lte` (≤)
- **Threshold (%):** The value to compare against
- **Cooldown (min):** How long before this rule can fire again for the same device
- **Auto-create ticket:** Check to auto-create a ticket every time this rule fires

4. Click **Create Rule**.
5. The rule will appear in the list and start evaluating on the next heartbeat.

## Practical Example: Alert Fires at 2am

It is 2am. Alex (on-call) gets an email: "CRITICAL: Disk at 97% on CORP-SRV02."

1. Alex logs in from their laptop.
2. Goes to **Alerts** → finds the critical alert.
3. Clicks **Acknowledge** — marks it as seen.
4. Goes to **Devices** → finds CORP-SRV02 → checks disk details. Drive C: at 97.3%.
5. Goes to **Maintenance** → selects CORP-SRV02 → confirms checkbox → clicks **Delete Temp Files**.
6. Waits 5 minutes. Disk drops to 94% — still critical but immediate action taken.
7. Creates a ticket: "CORP-SRV02 critical disk usage — need full cleanup" with priority Critical.
8. Goes to **Scripts** → runs "Get Largest Files" on CORP-SRV02 → adds output to the ticket as a comment.
9. Resolves the alert.
10. In the morning, a senior technician completes the full cleanup and closes the ticket.

## Recommended Baseline Alert Rules

Set up these rules in every new environment:

| Rule Name | Metric | Operator | Threshold | Severity | Cooldown | Auto-ticket |
|---|---|---|---|---|---|---|
| Critical CPU | cpu | gt | 90 | critical | 15 | Yes |
| CPU Warning | cpu | gt | 75 | warning | 30 | No |
| Critical RAM | ram | gt | 90 | critical | 15 | Yes |
| RAM Warning | ram | gt | 80 | warning | 30 | No |
| Critical Disk | disk | gt | 90 | critical | 60 | Yes |
| Disk Warning | disk | gt | 75 | warning | 120 | No |
| Device Offline | offline | — | — | warning | 60 | No |
| Low Battery | battery | lt | 20 | warning | 60 | No |

# Chapter 14: App Center — Software Inventory

## What it is

The App Center shows all software installed on a selected device. Use it to audit what is running on client machines, check software versions, identify unauthorized applications, and prepare for patch management.

## Who uses it

IT support staff, technicians, and administrators.

## What You See

After selecting a device:

- A summary showing how many software packages are installed
- A searchable, filterable table of every installed application with:
- Software name
- Version number
- Publisher

## Step-by-step: Viewing Installed Software

1. Click **App Center** in the sidebar.
2. Use the device selector dropdown to choose a device.
3. A table loads with all software on that device.
4. Use the **Search** field to filter by application name, version, or publisher.

**NOTE:** Software inventory is collected every 6 hours. Data reflects the last scan. If software was installed in the past 6 hours, it may not appear yet.

## Practical Example: Compliance Audit for Adobe Acrobat

Your manager asks you to confirm all DataSafe Ltd devices have Adobe Acrobat version 2024 or later.

1. Go to **Customers** → note all devices for DataSafe Ltd.
2. Go to **App Center**.
3. For each device, select it, then search for "Adobe Acrobat".
4. Note the version.
5. Create a ticket or note for any device not meeting the requirement.

# Chapter 15: Network Discovery

## What it is

Network Discovery performs a real ICMP ping sweep of a subnet you specify. It discovers all reachable hosts — computers, phones, printers, IoT devices — identifies their MAC address and vendor, and lets you save them permanently to your device fleet. Phones and tablets discovered this way appear in the Android and iOS tabs on the Devices page and are pinged automatically every 5 minutes.

## Who uses it

Technicians and administrators.

## Scan Results Table

| Column | Description |
|---|---|
| Platform icon | Visual indicator (Font Awesome brand icons — Win/Mac/Lin/iOS/And/—) |
| IP Address | The discovered host's IP |
| MAC Address | Hardware MAC address from ARP table |
| Vendor | OUI lookup result (e.g., "Apple, Inc.", "Samsung Electronics") |
| Platform | Detected OS family |
| Status | Online (responded to ping) or Offline |

## Auto-Detected Subnet Default

When you open the Network Discovery page, the **Subnet** field is automatically pre-filled with your server's current `/24` subnet — derived from the server's LAN IP address. If the RMM server is at `192.168.178.37`, the field defaults to `192.168.178.0/24`. You can override it at any time.

> **NOTE:** If you move the RMM server to a different network, the default subnet updates automatically on the next page load.

## Step-by-step: Running a Network Scan

1. Click **Network Discovery** in the sidebar (under TOOLS).
2. The **Subnet** field is pre-filled with your server's current subnet. Verify or override it.
3. Optionally select a **Customer** to assign all discovered devices to.
4. Click **Scan Network**.
5. A spinner shows while the scan runs (typically 15–30 seconds for a /24 subnet).
6. The results table populates automatically when the scan completes.

> **WARNING:** Running a network scan on a network with intrusion detection may trigger security alerts. Check with the client before scanning sensitive networks.

**NOTE:** The scan uses concurrent ICMP ping (50 parallel threads). On large subnets (/16 or wider), expect longer scan times and consider narrowing the range.

### Clearing Stale Devices from Other Networks

If you have moved to a different subnet and your Devices page shows agentless entries from an old IP range, use the built-in stale-device cleaner.

1. Open **Network Discovery** in the sidebar.
2. Click the ■■ **Clear stale devices from old networks** expander below the scan controls.
3. The panel shows your current detected subnet and lists all agentless devices whose IP addresses fall outside it.
4. If stale devices are listed, click **Delete N stale device(s)** to remove them permanently.
5. Run a fresh scan on your current subnet to rediscover devices on the new network.

**NOTE:** Only agentless (network-discovered) devices are evaluated. Agent-managed devices are never affected by this tool.

**WARNING:** Deletion is immediate and cannot be undone. Review the listed IPs before confirming.

### How Platform Detection Works

The system uses three stages to identify each discovered device's operating system, stopping as soon as a match is found:

1. **OUI vendor lookup** — the first 6 characters of the MAC address are matched against a built-in database of 500+ hardware manufacturers. Apple MACs → iOS, Samsung/Google/OnePlus MACs → Android, etc.

1. **Port probing** (fallback) — if OUI lookup cannot determine the platform, the system tries connecting to well-known ports:

- Port 62078 → iOS (iTunes Wi-Fi sync port)
- Port 5555 → Android (ADB debug port)
- **Two or more** of ports 445, 3389, 139 → Windows (requires 2 to avoid false-positives from routers and NAS devices that only expose SMB on port 445)
- Port 548 → macOS (Apple Filing Protocol)
- Port 22 → Linux (SSH)

Router and gateway devices (Fritz!Box, router, modem, etc.) are detected via their hostname and skipped entirely — they appear in the scan results for reference but are never added to your device fleet.

1. **Hostname keyword matching** (final fallback) — if both OUI and port probe fail, the system checks the device's reverse-DNS hostname (the name your router assigns, e.g. `Galaxy-S21-Ultra.fritz.box`) against 50+ keywords:

- Android brands: samsung, galaxy, pixel, xiaomi, redmi, poco, huawei, honor, oppo, vivo, realme, motorola, nokia, zte, and more
- Samsung model numbers: S10–S24, Note, A12–A73, Fold, Flip, Ultra
- iOS: iphone, ipad, ipod

Fritz!Box, ASUS, TP-Link, and most home routers assign the device's advertised name as the hostname, making this stage effective even when MAC randomization defeats OUI lookup and ADB is disabled.

**NOTE:** A device may still appear as "Unknown" if: (a) its MAC is randomized AND all listed ports are blocked AND its router-assigned hostname contains no recognisable keywords, or (b) it was offline during the latest scan. Re-running a scan when the device is connected will trigger all three detection stages again. Devices previously stored as "Unknown" are automatically upgraded to the correct platform on the next successful scan — no manual intervention needed. If a device genuinely cannot be auto-detected, use the **Edit** button on its row in the Devices page.

### Step-by-step: Saving Discovered Devices

1. Review the scan results. Verify the detected platform icons look correct.
2. Click **Save All to Devices**.
3. A confirmation shows how many devices were created, updated, or skipped (skipped = already registered with an agent).
4. Go to **Devices** → **Agentless** tab to see all saved devices.
5. Phones appear in the **Android** or **iOS** tabs depending on their detected platform. If shown as Unknown, use the Edit button to correct it.

### Automatic Online/Offline Monitoring

Once devices are saved, the system pings them automatically every 5 minutes via a background task. If a device does not respond for more than 10 minutes, its status is set to offline. No manual action is needed — the Devices page reflects current reachability automatically.

### Past Scan History

When no scan is running, the page shows the last 5 completed scans with their subnet, host count, and timestamp. Click any scan to review its results without re-scanning.

### Practical Example: Investigating an Unauthorized Device

A client suspects an unauthorized device on their network.

1. Go to **Network Discovery**.
2. Enter the client's subnet (e.g. `192.168.10.0/24`) and click **Scan Network**.
3. Compare the results against the known device list — look for unrecognized MAC addresses or vendors.
4. Any unfamiliar IP or vendor warrants investigation.
5. Save the results so the device appears in the Agentless tab for ongoing monitoring.

---

# PART IV — MANAGEMENT

# Chapter 16: Managing Tickets

## What it is

The Tickets page is the helpdesk ticketing system. Every support request, incident, or task should have a ticket. Tickets allow you to track work, communicate with teammates, and maintain a complete history of everything done for each client.

## Who uses it

IT support staff, technicians, and administrators. Viewers can see tickets but cannot create or update them.

## Ticket Table View

The Tickets page displays all tickets in a sortable table with the following columns:

| Column | What it shows |
|---|---|
| # | Ticket number (sequential ID) |
| Title | Brief issue summary — click to open the detail page |
| Priority | Color-coded badge: critical (red), high (amber), medium (blue), low (grey) |
| Status | Color-coded badge: open (red), in_progress (amber), resolved (green), closed (grey) |
| Customer | Customer name linked to this ticket |
| Assignee | Staff member currently assigned to work on it |
| Source | How the ticket was created: blank (manual), EMAIL (blue), PORTAL (green), ALERT (red) |
| SLA | Time remaining before SLA deadline, or BREACHED if overdue |
| Created | Date the ticket was opened |

Click any ticket row to open its **Detail Page**, where you can read the full description, update status, assign a technician, and post comments.

## Ticket Fields

| Field | Description | Options |
|---|---|---|
| Title | Brief summary of the issue | Free text (required) |
| Description | Detailed explanation | Free text |
| Customer | Which client this relates to | Dropdown (required) |
| Priority | Urgency level | low, medium, high, critical |
| Status | Current state | open, in_progress, resolved, closed |

| Field | Description | Options |
|---|---|---|
| Source | How it was created | manual, email (inbound email), client (Client Portal), alert (auto-created) |
| Department | Team responsible for this ticket | Dropdown (optional — see Chapter 43) |
| Assignee | Staff member responsible | Dropdown (optional) |

### Step-by-step: Creating a New Ticket

1. Click **Tickets** in the sidebar.
2. Click the **+ New Ticket** button at the top of the page.
3. In **Title**, type a brief summary. Example: "Server offline — ACME Corp"
4. In **Priority**, select the urgency:
- **critical** — complete outage or data risk
- **high** — significant impact, service degraded
- **medium** — non-urgent issue
- **low** — request or minor task
5. In **Description**, write full details: what the issue is, when it started, what has been tried.
6. In **Customer**, select the relevant customer. The customer must already exist in the system.
7. Optionally, set **Department** and **Assignee** if the ticket should be routed to a specific team or person.
8. Click **Create Ticket**.
9. The new ticket appears at the top of the table.

**WARNING:** The customer dropdown will show "— no customers —" if no customers exist yet. Create the customer in Chapter 17 first.

### Step-by-step: Finding a Ticket

1. Click **Tickets** in the sidebar.
2. Use the filter bar above the table:
- **Search field:** Type any word from the title or description.
- **Status dropdown:** Select open, in_progress, resolved, or closed. Select "All" for everything.
- **Priority dropdown:** Filter by urgency level.
3. The table updates instantly. The row count shows how many tickets match.

### Step-by-step: Working a Ticket (Detail Page)

1. Click the ticket row in the table to open the Detail Page.
2. On the Detail Page you can:
- Read the full description and all comments
- Change **Status** using the dropdown and click **Update Status**
- Change **Assignee** to hand off the ticket
- Add a **comment** (public or internal note)
3. Click ← **Back to Tickets** to return to the table.

## Exporting Tickets to CSV

A **Download CSV** button at the top of the Tickets page exports all currently-filtered tickets to a CSV file with: ID, title, customer, priority, status, assignee, source, and created date.

## Step-by-step: Adding a Comment (from Detail Page)

1. Open the ticket detail page.
2. Scroll to the **Add Comment** section.
3. Type your comment.
4. Tick **Internal note** if the comment is for team eyes only (clients cannot see internal notes).
5. Click **Post Comment**.

> **TIP:** Use comments to maintain a running log of every action. Future teammates reading the ticket should be able to understand the entire history from comments alone.

## SLA Status Indicators

| Display | Meaning |
|---|---|
| 2d | 2 days remaining |
| 4h | 4 hours remaining — approaching deadline |
| 30m | 30 minutes remaining — urgent |
| BREACHED | SLA deadline has passed |
| — | No SLA configured for this ticket |

## Ticket Status Colors

- **open** — red (needs attention)
- **in_progress** — amber (being worked on)
- **resolved** — green (fix applied)
- **closed** — grey (done)

## Practical Example: Client Calls With a Complaint

It is 2pm. TechCorp Ltd calls — their accounting software is very slow.

1. Go to **Tickets** → click **+ New Ticket**.
2. Title: "Accounting software slow performance — TechCorp Ltd"
3. Priority: **high**
4. Description: "Client reports QuickBooks running extremely slowly since this morning. All 5 workstations affected. No recent changes reported."
5. Customer: "TechCorp Ltd"
6. Click **Create Ticket**. Note the ticket ID.
7. Go to **Devices** → find TechCorp Ltd's server → check CPU.
8. CPU is at 98% — return to the ticket, expand it, add a comment: "Checked server TECHCORP-SRV01 — CPU at 98%. Investigating."
9. Update status to **in_progress**.

10. Resolve the issue. Add a comment explaining what was done.
11. Update to **resolved** and confirm with client.
12. After confirmation, update to **closed**.

---

# Chapter 17: Working with Customers

## What it is

The Customers page manages the client organizations that own the devices you support. Every device belongs to a customer. Every ticket must be associated with a customer.

## Who uses it

IT support staff, technicians, and administrators.

## Customer Fields

| Field | Description | Options |
|---|---|---|
| Name | Company or client name | Free text (required) |
| Email | Primary contact email | Email format |
| Phone | Contact phone number | Free text |
| Tier | Support level | standard, premium, enterprise |
| Primary Technician | Assigned team member | Selected from users |

## Step-by-step: Creating a New Customer

1. Click **Customers** in the sidebar.
2. Click **+ New Customer**.
3. Enter the **Name** — this is required and will identify the customer system-wide.
4. Fill in **Email**, **Phone**.
5. Select the appropriate **Tier**:

- **standard** — basic support, standard response times
- **premium** — priority support, faster response
- **enterprise** — highest tier, dedicated support

6. Optionally assign a **Primary Technician**.
7. Click **Save**.
8. The customer now appears in all dropdowns throughout the system.

## Practical Example: Onboarding a New Client

You just signed Greenway Manufacturing Ltd.

1. Go to **Customers** → click **+ New Customer**.
2. Name: "Greenway Manufacturing Ltd"
3. Email: `it-contact@greenway.com`
4. Phone: `+1 555 234 5678`
5. Tier: **premium** (they signed a premium support contract)
6. Primary Technician: assign yourself or the dedicated technician.
7. Click **Save**.
8. Deploy the agent on Greenway's machines (Chapter 6).

9. Within minutes of the agent connecting, the devices appear under Greenway Manufacturing Ltd on the Devices page.

**TIP:** The tier affects billing calculations on the Billing page — set it correctly during onboarding.

# Chapter 18: Automation Profiles

## What it is

Automation Profiles define bundles of maintenance tasks that run automatically on a schedule — daily, weekly, monthly, or on demand. A single profile can combine OS patching, software patching, disk maintenance, and cleanup tasks, and run across all your managed devices with no manual intervention.

## Who uses it

Administrators and senior technicians.

## Profile Structure

An automation profile contains:

- **Name and status:** Identifier and whether active or inactive.
- **Schedule:** When to run — daily, weekly, monthly, or once. Plus day of week and time.
- **Notification emails:** Email addresses to notify after the profile runs.
- **Run on newly installed agents:** Whether new devices automatically get this profile.
- **Four task columns:**

| Column | Tasks |
|---|---|
| OS Patch Management | Which Windows update categories to install |
| Software Patch Management | Which software to update, which to exclude |
| Disk Management | Defragment, Check Disk |
| Maintenance | Restore Point, Temp Files, Browser History, Reboot, Shutdown |

## The Profile List Tab

Shows all profiles as cards. Each card shows:

- Status dot (green = active, grey = inactive)
- Profile name, badge, schedule type, last run time
- A **Run Now** button — immediately queues the profile on all assigned devices

## Step-by-step: Creating an Automation Profile

1. Click **Automation** in the sidebar.
2. Click the **Create / Edit Profile** tab.
3. Leave the dropdown on **— New Profile —**.
4. Enter a **Profile Name**. Example: "Weekly Maintenance — Standard Clients"
5. Check **Active** to enable.
6. Set **Schedule** to `weekly`, Day to `sunday`, Time to `02:00`.
7. Enter notification email(s), comma-separated.
8. Under **OS Patch Management**: check Critical updates and Security updates.

9. Under **Disk Management**: check Run Checkdisk. Do not check Defragment unless you are sure these are HDDs.
10. Under **Maintenance**: check Create System Restore Point and Delete Temp Files.
11. Click **Save Profile**.
12. A "Profile saved!" confirmation appears.

### Step-by-step: Running a Profile Immediately

1. Go to **Automation** → **Profile List** tab.
2. Find the profile.
3. Click **Run Now**.
4. A confirmation shows how many devices the profile was queued on.
5. Navigate to **Maintenance** → Recent Maintenance Runs to watch tasks complete.

### Practical Example: Onboarding a New Client with Weekly Patching

Greenway Manufacturing needs weekly security patches every Sunday night.

1. Go to **Automation** → **Create / Edit Profile**.
2. Name: "Greenway Manufacturing — Weekly Security Patches"
3. Active: Checked. Schedule: weekly, Sunday, 23:00.
4. Notification: `it-contact@greenway.com`, your email.
5. OS Patches: Critical + Security updates only.
6. Maintenance: Create Restore Point + Delete Temp Files.
7. Save.
8. On Monday morning, check the **Maintenance** run log to verify patches were applied.

> **WARNING:** Enabling Reboot in a profile is a significant action. Always confirm with clients that a reboot is acceptable before enabling it.

---

# PART V — PATCHING

# Chapter 19: OS Patch Management

## What it is

The OS Patches page manages Windows Update deployment across all managed devices. You can see which patches are pending approval, approve them in batches, review patch history, and configure policies.

## Who uses it

Technicians and administrators.

## The Four Stat Cards

| Card | Description |
|---|---|
| Pending | Patches waiting for approval before deployment |
| Approved | Patches approved but not yet deployed |
| Deployed | Patches successfully deployed |
| Compliance % | Percentage of devices that are fully patched |

Compliance color coding: Green (90%+) = good, Amber (70–89%) = needs attention, Red (<70%) = poor.

## The Pending Patches Tab

Shows all patches waiting for your approval. Each patch shows:

- Patch name (including KB number)
- Type badge: critical, security, definition, rollup, feature, driver, update
- Which device reported this patch as available

**To approve patches:**

1. Use the checkboxes to select patches you want to approve.
2. Select one, several, or all.
3. Click **Approve Selected (N)**.
4. Approved patches are queued for deployment on next agent check-in.

> **NOTE:** Approving a patch does not immediately install it. The agent receives the approval on its next heartbeat cycle and proceeds with installation.

## The Patch History Tab

Shows all patches across all statuses:

| Column | Description |
|---|---|
| Patch Name | Full descriptive name |
| Type | Color-coded patch type badge |

| Column | Description |
|---|---|
| Status | DEPLOYED, APPROVED, PENDING, or FAILED |
| Device | Which device this patch applies to |
| Date | Deployment or creation date |

### Step-by-step: Approving This Week's Security Patches

1. Go to **OS Patches** → **Pending Patches** tab.
2. Prioritize patches with red "critical" or orange "security" badges.
3. Check those patches.
4. Click **Approve Selected (N)**.
5. Success message confirms how many were approved.
6. Check back in a few hours — switch to **Patch History** and verify they show as DEPLOYED.

### Practical Example: Checking Patch Compliance for a Client

DataSafe Ltd asks you to confirm all their servers have the latest security patches.

1. Go to **OS Patches** → **Patch History** tab.
2. Find all entries for DataSafe devices in the Device column.
3. Verify all show status DEPLOYED.
4. Check the Compliance % stat card.
5. If below 90%, go to Pending Patches and approve remaining patches.
6. Generate a `patch_summary` report (Chapter 24) for DataSafe Ltd as documentation.

---

# Chapter 20: Software Patches

## What it is

The Software Patches page manages third-party software updates on individual devices — applications like Chrome, Firefox, 7-Zip, VLC, and others that can be updated via winget or chocolatey package managers.

## Who uses it

Technicians and administrators.

## Page Layout

**Left column:**

- Device selector (only agent-managed online devices shown)
- Device info card (hostname, OS, IP)
- **Check for Updates** button

**Right column:**

- Searchable table of all installed software packages

> **NOTE:** Mobile devices (Android, iOS), network-discovered agentless devices, and any offline device do not appear in the device selector. Software inventory requires the RMM agent to be installed and running — phones and network-discovered devices have no agent and cannot report installed software. To view or manage those devices, use the **Devices** page instead.

## What the Software List Shows

The right column displays all software installed on the selected device, collected by the agent from two sources:

- **Windows Registry** — the most comprehensive source; covers everything in Add/Remove Programs
- **winget** — Microsoft's package manager; shows packages it knows about with their IDs

Each row shows: **Name**, **Version**, and **Publisher** (where available). Use the search box to filter by name or publisher.

> **NOTE:** The **Check for Updates** button is a placeholder for a future winget/Chocolatey update engine (Phase 6 roadmap). It does not currently trigger updates — use Automation Profiles (Chapter 18) for bulk software patching.

## Step-by-step: Finding a Specific Application

1. Click **Software Patches** in the sidebar.
2. Select an online device from the left column dropdown.
3. The right column loads the software list automatically.
4. Type the application name or publisher in the **Search** box to filter.

**TIP:** The **Installed** counter (top right of the software list) shows the total number of packages found on the selected device.

**NOTE:** For bulk software patching across many devices, use Automation Profiles (Chapter 18) — software patching here is device-by-device.

# PART VI — TOOLS

# Chapter 21: Scripts — Running Custom Automation

## What it is

The Scripts page is one of the most powerful features in the system. It lets you run automated scripts on managed devices from a central interface — no remote desktop needed. You can run built-in scripts from the library, upload custom scripts, and review execution history.

## Who uses it

Technicians and administrators for day-to-day use. Developers for creating and maintaining the script library.

## Supported Script Types

| Type | Badge | Language | Typical Use |
|---|---|---|---|
| ps1 | Blue | PowerShell | Windows automation, registry, services, users |
| bat | Orange | Windows Batch | Simple commands, legacy compatibility |
| py | Green | Python | Complex logic, API calls, data processing |
| sh | Purple | Shell/Bash | Linux/macOS agents |

## The Three Tabs

### Library Tab

Browse all scripts. Each script shows as an expandable card:

- Tag: ■ Built-in or ■ Custom
- On expansion: type badge, OS target, creation date, description
- Device multi-select (only online devices)
- Timeout setting (10–900 seconds, default 300)
- **Run** button

### Upload Tab

Create a new script:

- Script Name, Type, Description, and Script Content
- Click **Upload Script** to save to the library

### Run History Tab

Shows last 50 executions:

- Status icon: ■ success, ■ failed, ■ queued, ■ running, ■ timeout
- Script name, device hostname, status, triggered timestamp

Expanding a run entry shows duration, exit code, stdout, and stderr.

## Step-by-step: Running a Script on a Single Device

1. Click **Scripts** in the sidebar.
2. In the **Library** tab, search for the script.
3. Click to expand it.
4. In the device multi-select, select the target device.
5. Adjust the **Timeout** if needed.
6. Click **Run**.
7. Confirmation: "Queued on 1 device(s)."
8. Go to **Run History**. The run appears as ■ QUEUED.
9. After ~60 seconds (next agent heartbeat), refresh — status will be ■ SUCCESS or ■ FAILED.
10. Expand the entry to see output.

## Step-by-step: Running a Script on Multiple Devices

1. Follow steps 1–4 above.
2. In the multi-select, click multiple device names.
3. Click **Run**.
4. "Queued on [N] device(s)" confirms the count.
5. In Run History, one entry appears per device.

## Step-by-step: Uploading a New Script

1. Click **Scripts** → **Upload** tab.
2. Enter a clear descriptive **Name**: "Get Top 10 Largest Files"
3. Select **Type**: `ps1`
4. In **Description**: "Lists the 10 largest files on Drive C: by size."
5. Paste or write your script code in the content area.
6. Click **Upload Script**.
7. Switch to **Library** tab and search to confirm it appears.

## Reading Script Output

1. Go to **Run History** tab.
2. Find and expand the run.
3. **stdout** — normal output. This is your result data.
4. **stderr** — error output. Investigate even if exit code is 0.

A successful script: exit code 0, output in stdout.

## Practical Example: Cleanup Script on 10 Devices

Your manager asks you to run the "Clear Temp Files" script on all 10 QuickPrint Co devices overnight.

1. Go to **Scripts** → **Library** → search for "temp" or "cleanup".
2. Find and expand the appropriate script.
3. In the device multi-select, select all 10 QuickPrint Co devices (confirm they are online on Devices page first).
4. Set timeout to 120 seconds.
5. Click **Run**.

6. Confirm: "Queued on 10 device(s)."
7. Check **Run History** after 5 minutes. All should show ■ SUCCESS.
8. If any show ■ FAILED, expand that entry and read stderr to diagnose.

---

# Chapter 22: Disk Management

## What it is

Disk Management gives a visual and actionable view of disk usage across any selected device. It shows gauge charts for each drive and provides action buttons to perform disk maintenance remotely.

## Who uses it

Technicians and administrators.

## What You See

After selecting a device:

### Gauge Charts (up to 4 drives)

Each drive gets a gauge chart showing percentage used:

- **Green:** Under 75% (healthy)
- **Yellow/Amber:** 75–90% (warning)
- **Red:** Over 90% (critical)

### Summary Table

Lists all disks with used space, total capacity, and percentage.

### Action Buttons

| Button | Effect | Risk |
|---|---|---|
| Defragment | Schedules disk defragmentation | LOW (do NOT use on SSDs) |
| Check Disk | Schedules chkdsk for next reboot | LOW |
| Clean Temp Files | Deletes temp files to free space | LOW |

## Step-by-step: Investigating High Disk Usage

1. Click **Disk Management** in the sidebar.
2. Select the target device.
3. View gauge charts. Red = needs attention.
4. Read the summary table for exact GB values.
5. If critically low:

a. Click **Clean Temp Files** first. b. If still critical, go to **Scripts** and run "Get Largest Files". c. For HDDs only — consider Defragment.

1. Wait for the agent to complete the action.
2. Refresh the page to see updated disk metrics.

### Practical Example: Emergency Disk Cleanup

Alert fires at 3pm: RETAIL-PC05 disk at 93%. Machine cannot be rebooted during business hours.

1. Go to **Disk Management** → select RETAIL-PC05.
2. Drive C: is at 93% (deep red).
3. Click **Clean Temp Files**. Queued message appears.
4. Wait 2 minutes. Go to **Devices** → check disk metric for RETAIL-PC05.
5. Disk drops to 88% (yellow zone). Immediate crisis managed.
6. Create a ticket for a full disk audit during the next maintenance window.

**WARNING:** Do not run Defragment on SSDs. It wears out SSD cells and provides no benefit.

---

# Chapter 23: Maintenance Actions

## What it is

The Maintenance page lets you perform remote actions on an online device: reboot, shutdown, create restore points, delete temp files, clear browser history, and schedule a disk check. It also shows recent maintenance run history.

## Who uses it

Technicians and administrators.

## Critical Safety Features

**Only online devices appear** — the selector lists only currently online devices.

**Confirmation checkbox required** — you must check "I confirm this action on the selected device" before any button works. Without it, you get a warning and nothing executes.

## The Device Info Card

After selecting a device, a card shows:

- Hostname and IP (with green online dot)
- Operating System
- Last Seen timestamp
- Uptime

Verify this is the correct machine before taking any action.

## Available Actions

| Button | Effect | Risk Level |
|---|---|---|
| Reboot | Immediate reboot command | HIGH — interrupts active users |
| Shutdown | Immediate shutdown | HIGH — takes device offline |
| Create Restore Point | Windows system restore point | LOW — no disruption |
| Delete Temp Files | Removes temp files | LOW — safe |
| Clear Browser History | Clears browser saved data | MEDIUM — may affect user sessions |
| Check Disk | Schedules chkdsk at next reboot | LOW — no live disk changes |

**IMPORTANT:** All six actions are fully functional. Reboot and Shutdown execute immediately when the agent receives the command (within 60 seconds). Always confirm with the client before rebooting or shutting down any device.

## Step-by-step: Rebooting a Device

1. Click **Maintenance** in the sidebar.
2. Select the target device. Only online devices appear.
3. Verify the device info card shows the correct machine.
4. Before proceeding — verify the device is not actively in use and confirm with the customer.
5. Check the **"I confirm this action on the selected device"** checkbox.
6. Click **Reboot**.
7. "Reboot command sent to [hostname]." The command has been dispatched.
8. The device will appear offline for a few minutes.
9. After reboot, the agent reconnects. Verify the device returns to online on the Devices page within 10 minutes.

## Step-by-step: Checking Recent Maintenance Runs

1. Scroll to the bottom of the Maintenance page.
2. The **Recent Maintenance Runs** table shows the last 20 runs.
3. Columns: Profile name, Device, Started, Finished, Status badge.
4. Status colors:

- **success** (green) — completed without errors
- **failed** (red) — error encountered
- **running** (amber) — currently in progress
- **pending** (grey) — queued, not yet started

## Practical Example: Post-Patch Reboot

A patch was applied to CORP-SRV01. A reboot is required. It is 8pm (after hours).

1. Go to **Maintenance** → select CORP-SRV01.
2. Check Uptime in the device info card — 3 days (expected).
3. Tick "I confirm this action on the selected device."
4. Click **Reboot**.
5. Go to **Devices** → watch CORP-SRV01 go offline then return online.
6. Once back (5–10 minutes), verify the patch applied via **OS Patches** → Patch History.
7. Add a comment to the ticket: "CORP-SRV01 rebooted at 8:02pm. Back online at 8:09pm. Patch verified."

---

# PART VII — BUSINESS

# Chapter 24: Reports

## What it is

The Reports page generates formal reports about your managed environment. Reports are useful for client meetings, monthly reviews, compliance audits, and billing discussions.

## Who uses it

Administrators and senior technicians. The management team uses reports to present RMM service value to clients.

## Report Templates

| Template | What it covers |
|---|---|
| device_summary | All devices, status, OS versions, last seen times |
| patch_summary | Patch compliance, pending patches, deployed patches |
| alert_summary | All alerts in the date range, by severity and device |
| billing_summary | Billing totals, invoices generated, amounts due |

## Step-by-step: Generating a Report

1. Click **Reports** in the sidebar (under BUSINESS).
2. Click the **Generate** tab.
3. Select a **Template**.
4. Select the **Customer** (or All, if available).
5. Set the **Date Range** using Start Date and End Date.
6. Click **Generate**.
7. The report is queued for generation. Refresh the **History** tab after a few seconds — the Download button will become active once the file is ready.

**IMPORTANT:** Report generation runs as a background Celery task. If the Celery worker is not running, the report record will be created but the file will never be written — the Download button stays greyed out. Ensure the Celery worker is running before generating reports (see Chapter 5 and the Troubleshooting chapter).

## Step-by-step: Viewing Report History

1. Click **Reports** in the sidebar.
2. Click the **History** tab.
3. Each entry shows: report type, customer, generated timestamp, and a **Download** button.
4. If the Download button is greyed out, the Celery worker was not running when the report was generated. Start the Celery worker, then re-generate the report from the Generate tab.

**NOTE:** Reports are generated as CSV files stored in `api/reports/`. The Download button reads the file directly — no separate export step is required. Format shown in the UI is "csv".

### Practical Example: Monthly Client Report Package

At the end of each month, you send each client a summary of device health and incidents.

1. Go to **Reports** → **Generate** tab.
2. Template: `device_summary`, Customer: ACME Corp, Date range: entire previous month. Generate. Download.
3. Repeat for `alert_summary` and `patch_summary`.
4. Email the three reports to the ACME Corp contact.
5. File the reports in the History tab for future reference.

### Tips

- If no data exists for the selected date range, the report will be empty or show "No data". Verify agents were active during that period.
- Reports are stored in the system and can be re-downloaded at any time from the History tab.

---

# Chapter 25: Billing

## What it is

The Billing page is a full professional invoicing system. Administrators generate branded invoices for clients based on managed devices during a billing period. Each invoice receives a sequential reference number (INV-YYYY-NNNN), can carry a tax rate and custom notes, and can be downloaded as a formatted A4 PDF or emailed directly to the client from within the dashboard.

## Who uses it

Administrators only. Technicians and viewers cannot create or modify invoices.

## Before You Generate Your First Invoice

Set up your company branding so invoices look professional. Go to **Admin** → **Org Settings** and fill in:

- Company name, address, email, and phone number
- Payment terms (e.g. "Net 30") and bank/payment details
- Upload your company logo (PNG or JPG, max 400px wide)
- Optional footer message (e.g. "Thank you for your business!")
- **Currency** — select the currency for billing (EUR for Europe, USD for USA, etc.)

These details appear on every generated PDF invoice.

## The Billing Page Layout

**Summary metrics bar (top of page):**

| Metric | What it shows |
|---|---|
| Total Invoices | Count of all invoices in the current filter |
| Revenue (Paid) | Sum of all paid invoice totals |
| Outstanding | Sum of all draft and sent invoice totals |
| Overdue | Sum of all overdue invoice totals |

**Invoice list:** Each row shows the invoice number, customer, billing period, device count, rate, total, status badge, and action buttons.

**Generate Invoice form (bottom of page):** Creates a new invoice.

## Generate Invoice — Fields

| Field | Description |
|---|---|
| Customer | The client being billed |
| Period Start | Start date of the billing period (defaults to first day of current month) |

| Field | Description |
|---|---|
| Period End | End date of the billing period (defaults to last day of current month) |
| Rate / Device | Per-device monthly rate — label shows your configured currency symbol (e.g. €25.00 for EUR, $25.00 for USD) |
| Tax Rate (%) | Optional tax percentage applied to the subtotal (0 = no tax) |
| Due Date | Payment due date (defaults to Period End + 30 days) |
| Notes | Optional per-invoice notes printed on the PDF |

**NOTE:** All monetary amounts on the Billing page — invoice totals, Revenue (Paid), Outstanding, and Overdue — are displayed in the currency configured under **Admin** → **Org Settings** → **Regional Settings**. To change the currency, update Regional Settings and refresh the Billing page.

**NOTE:** Device count is determined automatically from the devices registered to that customer in the system at the time of generation. You do not enter it manually.

## Step-by-step: Generating an Invoice

1. Click **Billing** in the sidebar.
2. Scroll to the **Generate Invoice** form at the bottom.
3. Select the **Customer**.
4. Confirm or adjust **Period Start** and **Period End**.
5. Set the **Rate / Device** (your contracted rate with that client).
6. Optionally set a **Tax Rate** (e.g. 10 for 10%).
7. Optionally adjust the **Due Date**.
8. Optionally add **Notes** (e.g. "Please reference invoice number when paying").
9. Click **Generate Invoice**.
10. The invoice appears at the top of the list with status "draft" and a sequential invoice number (e.g. `INV-2026-0001`).

## Invoice Status Flow

| Status | Meaning | How to reach it |
|---|---|---|
| draft | Created, not yet sent | Automatically set on creation |
| sent | Delivered to client | Click **Send** button, or use Send Email |
| paid | Client has paid | Click **Paid** button |
| overdue | Past due date, unpaid | Click **Ovrd** button (from sent status) |

## Invoice Actions

Each invoice row has action buttons:

- **View** — Opens the full Invoice Detail page with A4 preview
- **Send** — (draft invoices only) Marks as "sent"
- **Paid** — (sent or overdue invoices) Marks as "paid"
- **Ovrd** — (sent invoices only) Marks as "overdue"
- ■ — Delete invoice (requires two clicks to confirm)

## The Invoice Detail Page

Click **View** on any invoice to open the full invoice detail view. This page shows:

- A styled A4 invoice preview with your company logo and branding
- Bill-To section with customer contact details
- Service period and due date
- Line items table with device count, rate, subtotal
- Tax calculation (if applicable) and grand total
- Payment instructions from your Org Settings bank details
- Notes and footer message

**Actions on the Invoice Detail page:**

| Button | What it does |
|---|---|
| ← Billing | Return to the Billing list |
| ■ Download PDF | Downloads a print-ready A4 PDF to your browser |
| ✉ Send Email | Emails the PDF to the customer's registered email address (requires SMTP configuration) |
| Status buttons | Same Send/Paid/Ovrd transitions as the list view |
| ■ Delete | Delete the invoice (confirm with second click) |

> **IMPORTANT:** The **Send Email** button requires SMTP to be configured in the server's `.env` file (see `SMTP_HOST`, `SMTP_USER`, `SMTP_PASS` in Chapter 4). If SMTP is not configured, clicking Send Email will return an error. Downloading the PDF and attaching it to your own email client always works regardless of SMTP configuration.

## Invoice Numbers

Invoice numbers are automatically assigned in sequential format: `INV-YYYY-NNNN` where YYYY is the current year and NNNN is a four-digit counter that resets each year (e.g. `INV-2026-0001`, `INV-2026-0002`). You cannot manually assign or edit invoice numbers.

## Billing Summary Metrics

At the top of the Billing page:

- **Total Invoices:** Count of all invoices in the current customer filter
- **Revenue (Paid):** Sum of all paid invoice totals
- **Outstanding:** Sum of all draft and sent invoice totals — what clients still owe
- **Overdue:** Sum of all overdue invoice totals — requires immediate attention

## Step-by-step: End-of-Month Billing Run

1. Go to **Admin** → **Org Settings** — confirm company details and logo are up to date.
2. Go to **Billing** at the end of every month.
3. For each active customer, scroll to the Generate Invoice form and create an invoice.
4. Verify the generated device count is correct (cross-check on the Devices page filtered by customer).
5. Click **View** on each invoice to preview the PDF before sending.
6. Click ■ **Download PDF** to save a local copy for your records.
7. Click ✉ **Send Email** to deliver the invoice directly to the customer (if SMTP configured), OR download and attach the PDF to your own email.
8. The status automatically moves to "sent" when Send Email is used.
9. When payment is received, return to Billing and click **Paid** on the relevant invoice.
10. The Outstanding metric decreases automatically.

**TIP:** Use the **Filter by Customer** dropdown at the top of the Billing page to see only one customer's invoices. This is useful when checking whether a specific client has any overdue invoices.

**NOTE:** If an invoice shows the wrong device count, delete it and regenerate. Device count is pulled live from the database at generation time — it reflects however many devices were registered to that customer when you clicked Generate Invoice.

---

# Chapter 26: Administration

## What it is

The Admin page is restricted to admin role users only. It provides four tabs: System Info, Audit Log, Users, and Org Settings. This is where you check system health, review all user actions, manage user accounts, and configure company branding for invoices.

## Who uses it

Administrators only. Technicians and viewers are blocked from this page.

## Tab 1: System Info

Four cards:

**Current User:** Your name, email, and role badge.

**System:** Configured API URL, dashboard URL, and database connection address.

**Services:** Live probe of service status:

- **Flask API** — probes `/api/health`. Shows green "Online" or red "Unreachable".
- **Streamlit Dashboard** — always shows Running (since you are using it).
- **PostgreSQL** — shows configured connection address.
- **Redis / Celery** — shows configured connection address.

> **NOTE:** PostgreSQL and Redis show the configured address, not a live probe. To verify database connectivity, check the Flask API health endpoint.

**Agent Enrollment Token:** The `ORG_REGISTRATION_TOKEN` used to enrol managed machines.

- Token is masked by default (`a8b6ea••••••••••`).
- Click **Reveal** to show the full token, **Hide** to mask it again.
- Copy the revealed token and paste it into `config.ini → org_token` on any machine you want to enrol.
- Keep this token secret — anyone who has it can register devices to your organisation.
- After all agents are enrolled, consider rotating the token in `.env` and restarting the API.

## Tab 2: Audit Log

Records every state-changing action in the system:

| Action Type | Color | Examples |
|---|---|---|
| CREATE | Green | Ticket created, device registered, user created |
| UPDATE | Blue | Ticket status changed, device updated |
| DELETE | Red | Rule deleted, device removed |
| LOGIN | Purple | User logged in |

| Action Type | Color | Examples |
|---|---|---|
| LOGOUT | Amber | User logged out |

Each entry shows: action type, resource type and ID, timestamp, IP address, and user email.

**Filtering:** Use the Action type dropdown and date range pickers to narrow results.

**Export to CSV:** A **Download CSV** button exports the currently-filtered audit log entries (timestamp, user, action, resource type, resource ID, IP address) to a CSV file.

## Step-by-step: Investigating a Suspicious Action

1. Go to **Admin** → **Audit Log** tab.
2. Set **Action type** filter to "DELETE".
3. Set a date range around when the suspected action occurred.
4. Look for unexpected DELETE events.
5. Note the User email and IP address.

## Tab 3: Users

Shows a card list of all registered users. By default, only **active** accounts are shown.

### *Account Status Badges*

Each user card may display one or more coloured status badges:

| Badge | Colour | Meaning |
|---|---|---|
| LOCKED | Red | Account is temporarily locked due to too many failed login attempts. The user cannot log in until the lockout expires or an admin unlocks it. |
| INACTIVE | Grey | Account has been deactivated. The user cannot log in. |
| ■ N attempt(s) | Amber | The user has had 1 or 2 failed login attempts but is not yet locked. A warning indicator only. |

A **locked** account card also has a red border so it stands out visually in the list.

### *Available Actions Per User*

| User State | Available Buttons |
|---|---|
| Active, not locked | Edit, Deactivate, Delete |
| Active, locked | **Unlock Account** (prominent), Edit, Deactivate, Delete |
| Inactive | Reactivate only |
| Superadmin | "Protected — use CLI" (no buttons) |

### *Creating a New User*

Available actions in the Users tab:

- **Create** new users (name, email, password, role) — includes **"Require password change on first login"** checkbox (checked by default). When enabled, the user must set a new password before accessing the dashboard.
- **Edit** existing users (change role, update name/email) — includes **"Require password change on next login"** checkbox to force a reset on an existing account.
- **Reset passwords** by editing and setting a new password

### *Showing Inactive Accounts*

Inactive (deactivated) users are hidden by default. To see them:

1. Tick the **"Show inactive accounts"** checkbox at the top of the Users tab.
2. Inactive users appear with a grey **INACTIVE** badge and greyed-out name.
3. Only the **Reactivate** button is shown for inactive users — they cannot be edited or deleted while inactive.

### *Step-by-step: Unlocking a Locked Account*

When a user is locked out, an administrator can unlock them immediately without waiting for the 5-minute timer.

1. Go to **Admin** → **Users** tab.
2. Find the locked user — their card has a red border and a red **LOCKED** badge.
3. Click the **Unlock Account** button (displayed prominently on the locked card).
4. The account is immediately unlocked. The failed attempt counter is reset to zero.
5. Notify the user they can now log in.

> **NOTE:** The lockout also clears automatically after 5 minutes without any admin action. Admin unlock is only needed if the user cannot wait.

### *Step-by-step: Deactivating a Departed Employee*

When someone leaves, deactivate their account immediately.

1. Go to **Admin** → **Users** tab.
2. Find the employee's card.
3. Click **Deactivate**.
4. A confirmation dialog appears: confirm the action.
5. The account is set to inactive. The user is immediately blocked from logging in.
6. Verify: check the Audit Log for any LOGIN events from that email after the deactivation date.

> **WARNING:** JWT tokens have an expiry time. After deactivating an account, the user may retain access until their current token expires (typically minutes). For immediate lockout, rotate the `JWT_SECRET_KEY` in the `.env` file and restart the API.

### *Step-by-step: Reactivating an Inactive Account*

1. Go to **Admin** → **Users** tab.
2. Tick **"Show inactive accounts"** to reveal inactive users.
3. Find the user's card (grey INACTIVE badge).
4. Click **Reactivate**.
5. The account is restored to active status. The user can log in again with their existing password.

### *Step-by-step: Permanently Deleting a User*

> **WARNING:** Deletion is permanent and cannot be undone. Use **Deactivate** instead if there is any chance you may need to restore the account.

Deactivation and deletion are different:

| Action | Effect | Reversible? |
|---|---|---|
| Deactivate | Blocks login; all records remain; account hidden unless "Show inactive" is ticked | Yes — Reactivate restores full access |
| Delete | Permanently removes login access; email is scrambled so the address can be reused; user disappears from all lists permanently | No |

To permanently delete a user:

1. Go to **Admin** → **Users** tab.
2. Find the user's card.
3. Click **Delete**.
4. A confirmation dialog appears: `"Permanently delete [Name] ([email])? This cannot be undone."` Read it carefully.
5. Confirm the deletion.
6. The account is permanently removed from all user lists.

> **NOTE:** You cannot delete your own account. You cannot delete the superadmin account.

## Step-by-step: Monthly Admin Security Review

On the first Monday of each month:

1. Go to **Admin** → **Audit Log**.
2. Filter by the previous month's date range.
3. Look for:

- Unexpected DELETE actions
- LOGIN attempts from unusual IP addresses
- Unusual patterns of failed actions

4. Go to **Users** tab. Verify all listed users are current employees.
5. Go to **System Info** → Services card. Verify all services are healthy.

## Account Security Policies

This section describes the automated security policies that apply to all user accounts. These run in the background without requiring any manual configuration beyond the initial `.env` setup.

### *Login Lockout Policy*

| Setting | Value |
|---|---|
| Failed attempts before lockout | 3 consecutive failures |
| Lockout duration | 5 minutes (auto-clears on next login attempt after expiry) |
| Rate limit | 10 login attempts per minute per IP address |
| Admin unlock | Available immediately via Admin → Users → Unlock Account |
| Superadmin exemption | Superadmin account is never locked |

When an account is locked:

- The user sees the remaining wait time on the login screen.
- All admin users receive an email notification.
- The lockout clears automatically after 5 minutes, or immediately when an admin clicks Unlock Account.

### *Password Expiry Policy*

Passwords expire after **90 days**.

| Event | What happens |
|---|---|
| Password 14, 7, 3, or 1 day(s) before expiry | Warning email sent to the user |
| Password reaches 90 days old | On next login, `must_change_password` is set automatically |
| User logs in with expired password | Full-screen forced password change form appears before dashboard access |
| Admin forces password change manually | Edit user → check "Require password change on next login" |
| Superadmin exemption | Superadmin password never expires |

The warning emails are sent by a Celery beat task that runs once daily. The task checks every active account's `password_changed_at` timestamp and sends warnings at the 14, 7, 3, and 1-day thresholds.

### *Dormant Account Auto-Deactivation Policy*

Accounts that have not been used for **30 or more days** are automatically deactivated.

- A Celery beat task runs daily and checks the `last_login` timestamp for all active accounts.
- Any account inactive for 30+ days is set to `is_active = False`.
- The affected user receives an email notification explaining their account has been deactivated.
- All admin users receive a summary email listing all accounts deactivated in that run.
- The superadmin account is exempt from auto-deactivation.

- Reactivation: an administrator can reactivate the account via Admin → Users → Show inactive accounts → Reactivate.

> **TIP:** If a user will be absent (e.g. long-term leave), administrators can manually deactivate their account and reactivate it on return, which resets the 30-day counter.

### *Login Anomaly Detection (New IP Alerts)*

The system records up to **10 known login IP addresses** per user account. When a user logs in from an IP address not previously seen on their account:

- An alert email is sent to **both the user and all admin users**.
- Subject line: `[RMM Security] New login location detected: user@email.com`
- The email includes the new IP address and timestamp.
- The new IP is automatically added to the user's known list.
- No alert is sent on the very first login ever (no known IPs to compare against yet).

> **NOTE:** This feature detects logins from new locations, not necessarily unauthorised access. Legitimate causes include a user logging in from a new device, a new office, or working from home for the first time. Admins should investigate the alert but treat it as informational rather than an alarm.

### *Per-Device Session Management*

The system creates **one active session per device per user**. Sessions are tracked by a device fingerprint derived from the browser's User-Agent and Accept-Language headers.

| Behaviour | Detail |
|---|---|
| Logging in from the same device again | Invalidates the previous session on that device (no duplicate sessions) |
| Logging in from a different device | Creates a new session; existing sessions on other devices remain active |
| Logging out | Only the current device session is invalidated; all other device sessions remain active |

This means a user can be logged in on multiple devices simultaneously, but each device can only hold one active session at a time.

### *Email Notifications Summary*

The following events trigger automatic email notifications. SMTP must be configured in `.env` for any emails to be sent.

| Event | Recipients |
|---|---|
| Account locked (3 failed attempts) | All admin users |
| Password expiry warning (14/7/3/1 days) | The affected user |
| Dormant account auto-deactivated | The deactivated user + all admin users |
| Login from new/unknown IP address | The affected user + all admin users |

| Event | Recipients |
|---|---|
| Password reset link requested | The requesting user (reset link) |

### *SMTP Configuration*

Email notifications require SMTP to be configured in the `.env` file on the RMM server:

```
SMTP_HOST=smtp.yourmailprovider.com
SMTP_PORT=587
SMTP_USER=your-smtp-username
SMTP_PASSWORD=your-smtp-password
SMTP_FROM=noreply@yourcompany.com
```

If `SMTP_HOST` is not set, all email notifications silently do nothing — the system continues operating normally, but no emails are sent. Set up SMTP before going live to ensure security alerts reach administrators.

> **TIP:** For testing, services such as Mailtrap or a Gmail app password (with less-secure app access or an app-specific password) work well as the SMTP provider.

## Tab 4: Org Settings

The Org Settings tab controls the company branding and payment details that appear on every generated PDF invoice. Set this up before generating your first invoice.

**Company Details section:**

| Field | Description | Example |
|---|---|---|
| Company Name | Your business name, printed at the top of invoices | Acme IT Services Ltd |
| Company Address | Full postal address (multi-line) | 123 High Street, London, W1A 1AA |
| Company Email | Contact email shown on invoice | billing@acmeit.com |
| Company Phone | Contact phone number | +44 20 7946 0958 |
| Payment Terms | Shown in the footer payment block | Net 30 |
| Bank / Payment Details | Full bank details or payment instructions | Sort: 12-34-56, Acc: 87654321 |
| Footer Message | Final line at the bottom of every invoice | Thank you for your business! |

Click **Save Company Settings** to apply changes. Changes take effect on the next invoice generated.

**Company Logo section:**

Your logo appears in the top-left corner of every PDF invoice.

- Click **Browse files** to upload a PNG or JPG image.

- The system automatically resizes it to a maximum of 400 pixels wide.
- Click **Upload Logo** to save. A preview appears immediately.
- Click **Remove Logo** to delete the current logo (invoices will show the company name in text instead).

**TIP:** A transparent-background PNG logo looks best on the white invoice background.

**NOTE:** Logo changes apply to new PDF downloads immediately. Previously downloaded PDFs are not retroactively updated.

**White-Label Branding section:**

Below the logo uploader is the **White-Label Branding** form. These fields control how the dashboard itself appears to every user — useful when reselling the RMM platform under your own company name and colours.

| Field | Description | Example |
|---|---|---|
| App Name | Displayed in the sidebar header, browser tab title, and login page | AcmeRMM |
| Tagline | Subtitle shown on the login page below the app name | Enterprise Monitoring |
| Primary Color | Hex colour code applied to buttons, tabs, nav accents, and focus borders | `#1E40AF` (blue) |

A live **colour preview block** appears next to the hex input so you can verify the choice before saving.

Click **Save Branding** to apply. Changes take effect on the next browser refresh. A 5-minute cache applies — colour changes may take up to 5 minutes to propagate across all open browser sessions.

**NOTE:** The primary colour replaces the default green (`#407E3C`) throughout the entire dashboard — buttons, active tab indicators, sidebar nav accents, input focus rings, and the login page submit button. The logo uploaded above is also shown in the sidebar header and login page.

**TIP:** After saving, open an incognito/private browser window to see the full white-label effect from a fresh session without waiting for the cache to expire.

**Regional Settings section:**

The **Regional Settings** form controls how currency and timestamps are displayed across the entire dashboard — both on-screen and in generated PDF invoices. Configure this before going live in any country or region.

| Field | Description | Supported Options |
|---|---|---|
| Currency | 3-letter ISO code used for all monetary amounts on Billing, Invoices, and summary metrics | USD, EUR, GBP, CHF, CAD, AUD, JPY, SEK, NOK, DKK |

| Field | Description | Supported Options |
|---|---|---|
| Timezone | Timezone applied when displaying timestamps throughout the dashboard | UTC, Europe/Berlin, Europe/London, Europe/Paris, Europe/Amsterdam, Europe/Zurich, America/New_York, America/Chicago, America/Denver, America/Los_Angeles, America/Toronto, America/Sao_Paulo, Australia/Sydney, Australia/Melbourne, Asia/Tokyo, Asia/Singapore, Asia/Dubai, Asia/Kolkata, Africa/Lagos |

Each currency option in the dropdown shows the full name and symbol for clarity — for example, `EUR — Euro (€)`, `USD — US Dollar ($)`, `GBP — British Pound (£)`.

Click **Save Regional Settings** to apply. Changes take effect immediately on the next page load — no server restart required.

> **IMPORTANT:** After changing currency, all billing figures on the Billing page will display the new symbol immediately. Historical invoice PDF downloads will also reflect the new symbol on re-download, since PDFs are generated on demand. Only the stored numeric amounts are unchanged — currency conversion is not performed.

> **TIP:** For a Germany deployment, set Currency = `EUR — Euro (€)` and Timezone = `Europe/Berlin`. For a USA deployment, set Currency = `USD — US Dollar ($)` and Timezone = `America/New_York` (or the appropriate US timezone). Each deployment saves its own settings — multiple clients on different deployments each configure their own regional preferences independently.

## Step-by-step: Setting Up Org Branding for the First Time

1. Go to **Admin** → **Org Settings** tab.
2. Fill in all Company Details fields.
3. Upload your company logo.
4. Click **Save Company Settings**.
5. Scroll to **Regional Settings** — select your **Currency** and **Timezone**.
6. Click **Save Regional Settings**.
7. Navigate to **Billing** and generate a test invoice for any customer.
8. Click **View** → ■ **Download PDF** to verify the branding and currency symbol look correct.
9. If the logo, details, or currency need adjusting, return to Admin → Org Settings and update.

## The Superadmin Account

**What it is:** A permanently present, highest-privilege account built into the system. It exists so that if all regular admin accounts are locked out, deactivated, or forgotten, the system can still be accessed and recovered.

**Key facts:**

- There is exactly one superadmin account per system.
- It is automatically re-created every time the Flask API starts, if it does not already exist.

- It cannot be edited or deleted through the web interface. The Edit and Delete buttons are replaced by "Protected — use CLI" in the Users tab.
- It has a purple role badge in the sidebar.
- It has full access to every feature in the system.

**Credentials** are set via environment variables in `.env`:

- Email: `SUPERADMIN_EMAIL` (default: `superadmin@rmm.local`)
- Password: `SUPERADMIN_PASSWORD` — **this is now required**. The API will refuse to start if `SUPERADMIN_PASSWORD` is not set in `.env`. There is no built-in default password.

**Changing the superadmin email or password via environment variables:**

Edit the `.env` file on the server and set:

```
SUPERADMIN_EMAIL=your-preferred-email@company.com
SUPERADMIN_PASSWORD=YourNewStrongPassword123
```

Then restart the Flask API. The account will be updated on next startup.

> **IMPORTANT:** `SUPERADMIN_PASSWORD` must be set before starting the API. If it is missing or blank, the API will raise a `RuntimeError` and refuse to start. Minimum length is 10 characters.

**Emergency password reset (when locked out of the web interface):**

If you cannot log in as superadmin and need to reset the password without a web session:

1. Open a terminal on the RMM server.
2. Navigate to the `api` folder:

```
Set-Location C:\RMM\RemoteManagementSystem\api
.\venv\Scripts\Activate.ps1
```

1. Run the reset tool:

```
python reset_superadmin.py YourNewPassword123
```

The password must be at least 10 characters. The tool confirms success and the new password takes effect immediately — no restart required.

> **WARNING:** Keep the superadmin password in a secure password manager or sealed physical document. It is your last line of defence if all other admin access is lost.

> **NOTE:** The superadmin account does not appear in the Audit Log's normal user activity in the same way as regular users — but its LOGIN events are still recorded.

---

# PART VIII — TECHNICAL REFERENCE

# Chapter 27: System Architecture

## Component Map

```
┌──────────────────────────────────────────────────────────┐
│  Browser (http://localhost:8501)                         │
│  Streamlit Dashboard — Python/Streamlit                  │
│  dashboard/app.py + dashboard/pages/*.py                 │
└──────────────────────────────────────────────────────────┘
                          │
                          │ HTTP REST API calls
                          │ (JWT Bearer token in headers)
                          ▼
┌──────────────────────────────────────────────────────────┐
│  Flask API (http://localhost:5000)                       │
│  api/app.py, api/routes/*.py                             │
│  api/models/*.py (SQLAlchemy ORM)                        │
│  api/services/*.py (business logic)                      │
│  api/tasks/*.py (Celery async tasks)                     │
└──────────────────────────────────────────────────────────┘
          │                              │
          │                              │
          ▼                              ▼
┌──────────────────┐          ┌──────────────────┐
│  PostgreSQL      │          │  Redis/Celery    │
│  localhost:5432  │          │  localhost:6379  │
│  db: rmmdb       │          │  Task queue      │
└──────────────────┘          └──────────────────┘
                                       │
                                       ▼
                              ┌──────────────────┐
                              │  Celery Workers  │
                              │  - Alert evaluation │
                              │  - Patch deployment │
                              │  - Script dispatch  │
                              └──────────────────┘
                                       │
                                       ▼
                              ┌──────────────────┐
                              │  Python Agent    │
                              │  agent/rmm_agent.py │
                              │  Heartbeat: 60s  │
                              │  SW scan: 6h     │
                              └──────────────────┘
```

## Service Startup Order

1. **PostgreSQL** — Windows Service, starts automatically
2. **Redis/Memurai** — Windows Service, starts automatically
3. **Flask API** — `cd api ; python app.py`
4. **Celery Worker** — `cd api ; celery -A tasks.celery_app worker --pool=solo -l info`
5. **Celery Beat** — `cd api ; celery -A tasks.celery_app beat -l info`
6. **Streamlit Dashboard** — `cd dashboard ; streamlit run app.py`
7. **Agent (on managed machines)** — `cd agent ; python rmm_agent.py` (run as Administrator)

## Directory Structure

```
RemoteManagementSystem/
■■■ api/
■ ■■■ app.py # Flask application factory
■ ■■■ config.py # Configuration (pool_size=10, max_overflow=20)
■ ■■■ extensions.py # SQLAlchemy, JWT, Celery setup
■ ■■■ models/ # SQLAlchemy model files
■ ■■■ routes/ # API route blueprints
■ ■■■ tasks/ # Celery task modules
■ ■■■ utils/
■ ■ ■■■ builtin_scripts.py # 7 built-in PS1 scripts
■ ■ ■■■ notifications.py # SMTP email sender
■ ■■■ reports/ # CSV report output directory
■ ■■■ seed.py # DB seed: admin user + default data
■■■ agent/
■ ■■■ rmm_agent.py # Main loop: register/heartbeat/tasks
■ ■■■ collector.py # Metrics, software inventory, patch scan
■ ■■■ heartbeat.py # API client
■ ■■■ executor.py # Task execution engine
■ ■■■ script_runner.py # Script execution
■ ■■■ config.ini # Agent configuration
■■■ dashboard/
■ ■■■ app.py # Login page
■ ■■■ pages/ # 16 Streamlit page files
■ ■■■ utils/
■ ■■■ auth.py # JWT login/logout/session management
■ ■■■ api_client.py # RMMClient: session reuse, retry, refresh
■ ■■■ nav.py # Shared sidebar component (shows logo/app_name)
■ ■■■ styles.py # CSS injection, stat cards, badges (color-swappable)
■ ■■■ branding.py # White-label branding loader (cached, early-fetch safe)
■ ■■■ formatters.py # Date, byte, color, currency formatting (fmt_currency,
fmt_datetime_tz)
■■■ scripts_library/ # Script templates on disk
```

## Authentication Flow

1. User submits email + password to the login form.
2. `utils/auth.py` calls `POST /api/auth/login`.
3. API validates credentials using bcrypt.
4. If valid, API returns a JWT access token and refresh token.
5. Dashboard stores the token in `st.session_state["access_token"]`.
6. All 16 pages call `require_auth()` at load time.
7. All API calls include the token as `Authorization: Bearer <token>`.
8. On 401, the dashboard auto-calls `POST /api/auth/refresh` and retries once.

## Agent Registration Flow

1. Agent reads `config.ini` on startup.
2. If `device_id` is blank, calls `POST /api/agent/register` with hardware info and `org_token`.
3. API creates a device record and returns `device_id` and `agent_token`.
4. Agent saves these to `agent_state.json`.
5. Every 60 seconds, agent calls `POST /api/agents/<device_id>/heartbeat` with current metrics.
6. Celery evaluates alert rules against new metrics every 60 seconds.
7. Agent polls `GET /api/agents/<device_id>/tasks` for queued commands.
8. Every 6 hours, agent scans for Windows updates and reports via `PUT /api/agents/<device_id>/patches`.

## Database Models Overview

| Model | Table | Key Fields |
|---|---|---|
| User | users | id, email, password_hash, role, is_active |
| Customer | customers | id, name, slug, email, phone, tier |
| Device | devices | id, hostname, os_name, customer_id, is_online, last_seen |
| DeviceMetrics | device_metrics | id, device_id, cpu_pct, ram_pct, disk_pct |
| InstalledSoftware | installed_software | id, device_id, name, version, publisher |
| AlertRule | alert_rules | id, name, metric, operator, threshold, severity, cooldown_minutes |
| Alert | alerts | id, rule_id, device_id, severity, message, status |
| Ticket | tickets | id, title, description, customer_id, priority, status |
| TicketComment | ticket_comments | id, ticket_id, author_id, body, is_internal |
| Script | scripts | id, name, file_type, content, is_builtin |
| ScriptRun | script_runs | id, script_id, device_id, status, exit_code, stdout, stderr |
| PatchRecord | patch_records | id, device_id, patch_name, kb_id, status |
| AutomationProfile | automation_profiles | id, name, schedule_type, is_active |
| Report | reports | id, name, template_type, customer_id, file_path |
| Invoice | invoices | id, invoice_number, customer_id, period_start, period_end, due_date, device_count, per_device_rate, tax_rate, subtotal, tax_amount, total, notes, status |

| Model | Table | Key Fields |
|---|---|---|
| OrgSettings | org_settings | id (always 1), company_name, company_address, company_email, logo_data, payment_terms, bank_details, footer_notes, app_name, tagline, primary_color, currency (ISO 3-letter, default USD), timezone (IANA tz, default UTC) |
| AuditLog | audit_logs | id, user_id, action, resource_type, ip_address |

# Chapter 28: Script Writing Guide

## Overview

Scripts are the automation workhorses of the RMM. Write them in PowerShell (ps1), Windows Batch (bat), Python (py), or Shell (sh). The agent executes them and captures stdout and stderr, which are sent back to the RMM and displayed in Run History.

## PowerShell (ps1) — Recommended for Windows

**Basic template:**

```
# Script: Get System Information
# Description: Returns OS, CPU, RAM, and disk info

try {
$os = Get-WmiObject Win32_OperatingSystem
$cpu = Get-WmiObject Win32_Processor
$disk = Get-WmiObject Win32_LogicalDisk -Filter "DriveType=3"

Write-Output "=== System Information ==="
Write-Output "OS: $($os.Caption) Build $($os.BuildNumber)"
Write-Output "CPU: $($cpu.Name)"
Write-Output "RAM: $([math]::Round($os.TotalVisibleMemorySize / 1MB, 2)) GB total"

Write-Output "=== Disk Usage ==="
foreach ($d in $disk) {
$pct = [math]::Round((1 - ($d.FreeSpace / $d.Size)) * 100, 1)
Write-Output "Drive $($d.DeviceID): $pct% used ($([math]::Round($d.FreeSpace/1GB,1)) GB
free)"
}
exit 0
} catch {
Write-Error "Script failed: $_"
exit 1
}
```

**Key PS1 conventions:**

- Use `Write-Output` (not `Write-Host`) — `Write-Host` goes to console, not stdout.
- Use `Write-Error` for error messages — appears in stderr.
- Exit with `exit 0` on success, `exit 1` on failure.
- Wrap in try/catch blocks.

**Example: Get 10 Largest Files:**

```
try {
$files = Get-ChildItem -Path "C:\" -Recurse -File -ErrorAction SilentlyContinue |
Sort-Object Length -Descending |
Select-Object -First 10

Write-Output "Top 10 largest files on C:"
Write-Output "----------------------------------------"
foreach ($f in $files) {
```

```
$sizeMB = [math]::Round($f.Length / 1MB, 1)
Write-Output "$sizeMB MB $($f.FullName)"
}
exit 0
} catch {
Write-Error "Error: $_"
exit 1
}
```

### Windows Batch (bat)

```
@echo off
:: Script: List Running Services
net start
if %ERRORLEVEL% EQU 0 (
echo Services listed successfully.
exit /b 0
) else (
echo Error. Code: %ERRORLEVEL%
exit /b 1
)
```

### Python (py)

```
#!/usr/bin/env python3
import sys
import subprocess

def main():
try:
result = subprocess.run(
["powershell", "-Command",
"Get-Process | Sort-Object CPU -Descending | Select-Object -First 10 | Format-Table
Name,CPU -AutoSize"],
capture_output=True, text=True, timeout=30
)
if result.returncode == 0:
print(result.stdout)
return 0
else:
print(f"Error: {result.stderr}", file=sys.stderr)
return 1
except Exception as e:
print(f"Script error: {e}", file=sys.stderr)
return 1

if __name__ == "__main__":
sys.exit(main())
```

## Best Practices for All Scripts

1. **Always include a description** — future team members will need it.
2. **Use exit codes correctly** — 0 = success, non-zero = failure.
3. **Handle errors** — never let scripts fail silently.
4. **Limit output** — use `Select-Object -First 50` to avoid filling the database.
5. **Test on a dev device** before running on client machines.
6. **Never hardcode credentials** — no passwords or API keys in script content.
7. **Idempotent where possible** — scripts that can run multiple times safely are safer.

---

# Chapter 29: Alert Rule Design

## Rule Anatomy

```
Rule: "Critical CPU Alert"
■■■ Metric: cpu
■■■ Operator: gt (greater than)
■■■ Threshold: 90 (%)
■■■ Severity: critical
■■■ Cooldown: 15 (minutes)
■■■ Auto-ticket: true
```

When any device's CPU exceeds 90%, a critical alert is created. It cannot fire again for the same device for 15 minutes.

## Available Metrics

| Metric | Measures | Threshold Unit |
|---|---|---|
| cpu | CPU usage from latest heartbeat | % (0–100) |
| ram | RAM usage from latest heartbeat | % (0–100) |
| disk | Primary disk usage | % (0–100) |
| battery | Battery level (laptops) | % (0–100) |
| offline | Device stops sending heartbeats | N/A |

## Operators

| Operator | Symbol | Use When |
|---|---|---|
| gt | > | Alert when metric goes above a level |
| gte | >= | Alert when metric reaches or exceeds a level |
| lt | < | Alert when metric drops below (low battery) |
| lte | <= | Alert when metric falls to or below a level |

## Cooldown Tuning

The cooldown prevents alert flooding. A 5-minute cooldown on a device constantly at 95% CPU means 12 alerts per hour for that device alone.

| Severity | Recommended Cooldown |
|---|---|
| critical | 15–30 minutes |
| warning | 60–120 minutes |

| Severity | Recommended Cooldown |
|---|---|
| info | 240+ minutes |

### Auto-Ticket Policy

Enable auto-ticket conservatively:

- **Enable for:** critical alerts (CPU >90%, disk >90%)
- **Disable for:** warning and info alerts — review manually, create tickets as needed

---

# Chapter 30: Automation Profile Design

## Design Principles

A well-designed automation profile answers:

1. What tasks need to run?
2. How often?
3. At what time to minimize disruption?
4. On which devices?
5. What should happen if something fails?

## Profile Templates

### Template 1: Nightly Security Maintenance (Standard)

```
Schedule: daily, 02:00
OS Patches: critical + security only
Disk: checkdisk = true, defrag = false
Maintenance: restore_point = true, delete_temp = true
Reboot: false
```

### Template 2: Weekend Full Maintenance

```
Schedule: weekly, Sunday, 23:00
OS Patches: all categories including rollups
Software Patches: update_all = true
Disk: defrag = true (HDDs only), checkdisk = true
Maintenance: restore_point = true, delete_temp = true
Reboot: true
```

### Template 3: Monthly Patch Tuesday

```
Schedule: monthly, second Tuesday, 22:00
OS Patches: critical + security + definitions
Disk: checkdisk = true
Maintenance: restore_point = true
Reboot: true
```

## Common Mistakes to Avoid

1. **Do not schedule reboots on always-in-use devices.** Always confirm with clients first.
2. **Test on one device before fleet-wide rollout.**
3. **Exclude known-problematic packages** in the Software Patch exclusions list.
4. **Stagger schedules** across clients to avoid simultaneous API/database load.
5. **Review run history** after each profile run for failed tasks.

---

# Chapter 31: User Roles and Permissions Matrix

## Full Permissions Matrix

| Feature / Action | Client | Viewer | Technician | Admin | Superadmin |
|---|---|---|---|---|---|
| **Dashboard** | | | | | |
| View Dashboard Overview | No | Yes | Yes | Yes | Yes |
| **Client Portal** | | | | | |
| View own tickets | Yes | No | No | No | No |
| Submit new tickets (portal) | Yes | No | No | No | No |
| **Tickets** | | | | | |
| View all tickets | No | Yes | Yes | Yes | Yes |
| Create tickets (staff form) | No | No | Yes | Yes | Yes |
| Update ticket status | No | No | Yes | Yes | Yes |
| Assign tickets | No | No | Yes | Yes | Yes |
| Add comments (public) | No | No | Yes | Yes | Yes |
| Add comments (internal) | No | No | Yes | Yes | Yes |
| Delete tickets | No | No | No | Yes | Yes |
| **Customers** | | | | | |
| View customers | No | Yes | Yes | Yes | Yes |
| Create customers | No | No | Yes | Yes | Yes |
| Edit customers | No | No | Yes | Yes | Yes |
| Delete customers | No | No | No | Yes | Yes |
| **Devices** | | | | | |
| View device list | No | Yes | Yes | Yes | Yes |
| View device metrics | No | Yes | Yes | Yes | Yes |
| **Alerts** | | | | | |
| View alerts | No | Yes | Yes | Yes | Yes |
| Acknowledge alerts | No | No | Yes | Yes | Yes |
| Resolve alerts | No | No | Yes | Yes | Yes |

| Feature / Action | Client | Viewer | Technician | Admin | Superadmin |
|---|---|---|---|---|---|
| Create/manage alert rules | No | No | Yes | Yes | Yes |
| **App Center** | | | | | |
| View software inventory | No | Yes | Yes | Yes | Yes |
| **Network Discovery** | | | | | |
| Run network scan | No | No | Yes | Yes | Yes |
| View scan results | No | Yes | Yes | Yes | Yes |
| **Reports** | | | | | |
| Generate reports | No | No | Yes | Yes | Yes |
| View/download report history | No | Yes | Yes | Yes | Yes |
| **Billing** | | | | | |
| View invoices | No | Yes | Yes | Yes | Yes |
| Create invoices | No | No | No | Yes | Yes |
| Update invoice status | No | No | No | Yes | Yes |
| **Administration** | | | | | |
| Access Admin page | No | No | No | Yes | Yes |
| View Audit Log | No | No | No | Yes | Yes |
| Manage users | No | No | No | Yes | Yes |
| Modify/delete superadmin account | No | No | No | No | CLI only |
| **Automation** | | | | | |
| View profiles | No | Yes | Yes | Yes | Yes |
| Create/edit profiles | No | No | Yes | Yes | Yes |
| Run profile now | No | No | Yes | Yes | Yes |
| Delete profiles | No | No | No | Yes | Yes |
| **OS Patches** | | | | | |
| View pending patches | No | Yes | Yes | Yes | Yes |
| Approve patches | No | No | Yes | Yes | Yes |

| Feature / Action | Client | Viewer | Technician | Admin | Superadmin |
|---|---|---|---|---|---|
| **Software Patches** | | | | | |
| View software list | No | Yes | Yes | Yes | Yes |
| Check for updates | No | No | Yes | Yes | Yes |
| **Disk Management** | | | | | |
| View disk gauges | No | Yes | Yes | Yes | Yes |
| Run disk actions | No | No | Yes | Yes | Yes |
| **Maintenance** | | | | | |
| View maintenance page | No | Yes | Yes | Yes | Yes |
| Reboot/Shutdown devices | No | No | Yes | Yes | Yes |
| Run maintenance actions | No | No | Yes | Yes | Yes |
| **Scripts** | | | | | |
| View script library | No | Yes | Yes | Yes | Yes |
| Run scripts | No | No | Yes | Yes | Yes |
| Upload scripts | No | No | Yes | Yes | Yes |
| View run history | No | Yes | Yes | Yes | Yes |
| **Remote Terminal** | | | | | |
| Open terminal sessions | No | No | Yes | Yes | Yes |
| **Departments** | | | | | |
| View departments | No | Yes | Yes | Yes | Yes |
| Create/edit/delete departments | No | No | No | Yes | Yes |

# Chapter 32: Common Troubleshooting

## Problem: Cannot log in — "Invalid credentials"

**Cause:** Wrong email, wrong password, or account is deactivated.

**Steps:**

1. Verify the email address — no typos, correct domain.
2. Check Caps Lock is off.
3. If recently changed password, try the new one.
4. If locked out, contact an administrator to check account status. If you are the only user and locked out:

```
UPDATE users SET is_active = true WHERE email = 'your@email.com';
```

---

## Problem: Dashboard shows "API error: [message]"

**Cause:** Flask API at http://localhost:5000 is not responding.

**Steps:**

1. Check if API is running:

```
netstat -ano | findstr :5000
```

1. If nothing is on port 5000, start the API:

```
cd C:\RMM\RemoteManagementSystem\api
.\venv\Scripts\Activate.ps1
python app.py
```

1. If the API starts then immediately crashes, check the terminal output. Common causes:

- PostgreSQL not running (check port 5432 or Windows Services)
- Wrong `DATABASE_URL` in `.env`
- Missing Python dependencies (`pip install -r requirements.txt`)

---

## Problem: Devices showing offline when they should be online

**Cause:** Agent stopped, device is off, or network issue.

**Steps:**

1. Check **Last Seen** timestamp. How long ago was it?
2. Under 5 minutes: likely a temporary network glitch — wait and recheck.
3. Over 10 minutes: physically check or remote into the device.
4. On the device, verify the agent is running:

```
Get-Process python -ErrorAction SilentlyContinue
```

1. If not running, restart:

```
cd C:\RMM\agent
.\venv\Scripts\Activate.ps1
```

```
python rmm_agent.py
```

1. Check `agent\rmm_agent.log` for error messages.

---

### Problem: Agent won't register — "Registration failed"

**Cause:** Wrong API URL, wrong org_token, or API is not running.

**Steps:**

1. Open `agent\config.ini` on the managed device.
2. Verify `[api]` section:

```
[api]
url = http://YOUR_SERVER_IP:5000
org_token = YOUR_ORG_TOKEN
```

1. Test connectivity from the device:

- Open a browser on that device and go to `http://YOUR_SERVER_IP:5000/api/health`
- If you see a JSON response, the API is reachable

2. Verify `org_token` matches exactly — find it in the dashboard: **Admin** → **System Info** → **Agent Enrollment Token** → Reveal. Or check `ORG_REGISTRATION_TOKEN` in the API's `.env` file directly.

---

### Problem: Report Download button is greyed out

**Cause:** The Celery worker was not running when the report was generated. Report generation is a background task — if no worker picks it up, the CSV file is never written and `file_path` stays empty in the database.

**Steps:**

1. Verify the Celery worker is running:

```
netstat -ano | findstr :6379
```

1. If not running, start it (Terminal 2):

```
cd C:\RMM\RemoteManagementSystem\api
.\venv\Scripts\Activate.ps1
celery -A tasks.celery_app worker --pool=solo -l info
```

1. Return to Reports → Generate tab and re-generate the report.
2. Wait a few seconds, then refresh Report History — the Download button will be active.

---

### Problem: Metrics History shows "No metric history available" but device is online

**Cause:** The 24-hour window has no data — typically because the agent was restarted recently or was offline for more than a day.

**Steps:**

1. The dashboard automatically falls back to a 7-day window when the 24-hour window is empty. An info banner will appear: *"No data in last 24 h — showing last N readings."*
2. If you see "No metric history available" even after the fallback, it means there is no data in the last 7 days at all. The agent has not been running long enough to build history.
3. Verify the agent is running on the device. Allow at least one heartbeat cycle (60 seconds), then click Metrics History again.

---

### Problem: Scripts stuck in "queued" status

**Cause:** Agent offline, not polling, or Celery not running.

**Steps:**

1. Verify target device is online (green dot on Devices page).
2. Agent polls every 60 seconds — wait up to 2 minutes.
3. If still queued after 5 minutes, verify Celery worker is running.
4. Check Redis is running on port 6379:

```
Test-NetConnection -ComputerName localhost -Port 6379
```

---

### Problem: Automation profile runs showing as "failed"

**Cause:** A task in the profile encountered an error.

**Steps:**

1. Go to **Maintenance** → Recent Maintenance Runs.
2. Find the failed run.
3. Note which profile and device.
4. Go to **Scripts** → Run History if the failure involves a script.
5. Read stderr for the error message.
6. Common causes:

- Device went offline during execution
- Agent lacked permissions for the task
- A specific patch installation failed

---

# Chapter 44: AI Assistant — Contextual Guidance on Every Page

## What is the AI Assistant?

The AI Assistant is a built-in chat helper that appears on every page of the dashboard. It knows which page you are on, what your role permits, and — on data-heavy pages — the live numbers currently on your screen. You can ask it anything about how to use the system and it will give you a direct, step-by-step answer.

It is powered by Anthropic's Claude AI and operates entirely within the RMM platform. Your conversations are not stored in the database and are not visible to other users.

> **NOTE:** The AI Assistant only answers questions about the RMM system. It will not answer general knowledge questions, write code for you, or assist with tasks unrelated to the platform.

---

## Opening and Closing the Assistant

The AI Assistant button appears at the bottom of the sidebar on every page, below the navigation links and the Sign Out button.

**To open it:**

1. Scroll to the bottom of the sidebar.
2. Click **AI Assistant** (sparkle icon).
3. The chat panel expands directly in the sidebar.

**To close it:**

1. Click **Close Assistant** (the button label changes when the panel is open).

The panel stays open as you navigate between pages — your conversation history is preserved for the entire session. Refreshing the browser or logging out clears the history.

---

## First-Time Welcome Message

The first time you log in during a session, the AI Assistant opens automatically on the Overview (Dashboard) page and shows a welcome message. It introduces itself and suggests a few things you can try.

This welcome message appears once per session. It does not appear again until you log out and back in.

---

## Asking a Question

1. Click inside the text box labelled **Ask anything…**
2. Type your question.
3. Click **Send** (or press **Enter**).
4. The assistant replies in a few seconds.

**Example questions you can ask:**

| Page | Example question |
|---|---|
| Overview | "What does a critical alert mean?" |
| Devices | "How do I run a script on a device?" |
| Alerts | "How do I acknowledge an alert?" |
| Network Discovery | "What is an agentless device?" |
| OS Patches | "How do I deploy patches to a device?" |
| Admin Panel | "How do I add a new user?" |
| My Profile | "How do I enable two-factor authentication?" |
| Any page | "What can I do on this page?" |

---

## Quick Action Buttons

After each reply, the assistant shows up to three **Quick Action** buttons below the chat. These are shortcuts for the most common follow-up questions on the current page. Click any of them to ask that question immediately without typing.

---

## Clearing the Conversation

If you want to start fresh:

1. Open the AI Assistant panel.
2. Click **Clear** (appears below the chat history when there are messages).
3. The history is cleared and the assistant is ready for a new conversation.

---

## Role-Aware Responses

The assistant knows your role and only suggests actions you are permitted to take:

| Your Role | What the assistant will guide you to do |
|---|---|
| **Viewer** | View dashboards, devices, alerts, tickets, customers, and reports. The assistant will never suggest creating, editing, deleting, or running anything. |

| Your Role | What the assistant will guide you to do |
|---|---|
| **Technician** | Everything a viewer can do, plus: manage devices, run scripts, deploy patches, handle tickets, acknowledge and resolve alerts, use Remote Terminal, view reports and billing. |
| **Admin** | Everything a technician can do, plus: manage users, change organisation settings, manage billing, use the enrollment token. |
| **Superadmin** | Full access including all admin capabilities and superadmin-only controls. |
| **Client** | Create and view your own support tickets only. The assistant will not guide clients to monitoring, device, or admin features. |

If you ask about a feature that requires a higher role, the assistant will tell you clearly (e.g., *"This requires admin access — please contact your administrator"*).

---

## Live Context

On pages that display live data, the assistant is aware of the current numbers. For example:

- On the **Overview** page: the assistant knows how many devices are online, offline, and critical, and how many alerts and tickets are open.
- On the **Alerts** page: it knows how many critical, warning, and acknowledged alerts are currently visible.
- On the **Devices** page: it knows how many devices are loaded.

This means you can ask questions like *"I have 3 critical alerts — what should I do first?"* and the assistant can give a contextually relevant answer.

---

## Enabling and Disabling the AI Assistant

The AI Assistant is controlled by the `AI_ASSISTANT_ENABLED` variable in the server's `.env` file:

```
AI_ASSISTANT_ENABLED=true # On (default)
AI_ASSISTANT_ENABLED=false # Off — widget still appears but sends a disabled message
```

After changing this value, the API server must be restarted for the change to take effect.

> **NOTE:** Only an administrator with access to the server's `.env` file can enable or disable the AI Assistant. This is not configurable from inside the dashboard.

---

## Troubleshooting: "AI assistant is temporarily unavailable"

If you see this message when sending a question, it means the connection to the AI service failed. This can happen if:

1. **The API server lost its connection to the Anthropic service** — this is usually temporary. Wait 30 seconds and try again.
2. **The** `ANTHROPIC_API_KEY` **is missing or invalid** — an administrator must check the `.env` file and restart the API server.
3. **The rate limit was reached** — the assistant allows 30 messages per minute per user. If you see this message after sending many messages quickly, wait one minute and try again.

The rest of the dashboard continues to work normally when the AI Assistant is unavailable — only the chat feature is affected.

---

## For Administrators: Setup

The AI Assistant requires an Anthropic API key. If it was not configured at initial deployment, follow these steps:

1. Obtain an API key from **console.anthropic.com**.
2. Open the server's `.env` file (located in the project root).
3. Add or update these three lines:

```
ANTHROPIC_API_KEY=sk-ant-api03-...your-key-here...
AI_ASSISTANT_MODEL=claude-haiku-4-5-20251001
AI_ASSISTANT_ENABLED=true
```

1. Save the file.
2. Restart the Flask API server:

```
# Kill the current API process
netstat -ano | findstr :5000
taskkill /F /PID <PID>
# Start it again
cd C:\RMM\RemoteManagementSystem\api
.\venv\Scripts\Activate.ps1
python app.py
```

1. Reload the dashboard — the **AI Assistant** button should now appear at the bottom of the sidebar.

**IMPORTANT:** Never commit the `.env` file to version control. The `ANTHROPIC_API_KEY` is a secret. Store it in `.env` only, which is listed in `.gitignore`.

---

## For Developers: Architecture Summary

| Component | File | Description |
|---|---|---|
| Chat endpoint | `api/routes/assistant.py` | `POST /api/assistant/chat` — JWT required, 30 req/min, builds system prompt, calls Anthropic SDK |
| Sidebar widget | `dashboard/utils/ai_assistant.py` | `render_ai_assistant(page_name, context)` — called at bottom of every page |
| Blueprint registration | `api/app.py` | `app.register_blueprint(assistant_bp, url_prefix="/api/assistant")` |
| Dependency | `api/requirements.txt` | `anthropic>=0.40.0` |

The system prompt sent to Claude includes: the user's role and what it permits, the current page name and its purpose, live context data (device counts, alert counts, etc.), and guardrails preventing the AI from inventing features that do not exist.

---

### Problem: MFA code rejected — "Invalid or expired code"

**Cause:** Code entered after it expired, or phone and server clocks are out of sync.

**Steps:**

1. Wait for the code to refresh in your authenticator app (timer resets to full).
2. Enter the fresh code immediately.
3. If still failing, set your phone time to automatic/network time — TOTP requires synchronized clocks.
4. If locked out permanently (lost phone), contact an administrator to disable MFA on your account.

---

### Problem: API refuses to start — RuntimeError about SUPERADMIN_PASSWORD

**Cause:** `SUPERADMIN_PASSWORD` is missing from `api\.env` or is less than 10 characters.

**Steps:**

1. Open `api\.env` in a text editor.
2. Add: `SUPERADMIN_PASSWORD=YourStrongPassword123` (min 10 characters).
3. Restart the API.

---

### Problem: Cannot access Admin page — "Admin access required"

This is intentional. Admin page is restricted to admin role only.

To grant admin access (if you have database access):

```
UPDATE users SET role = 'admin' WHERE email = 'your@email.com';
```

---

### Problem: Dashboard blank or session error after navigating

**Cause:** JWT token expired.

**Steps:**

1. Go to http://localhost:8501.
2. If you see the login screen, log in again.

---

### Problem: Network scan stuck in "RUNNING" status forever

**Cause:** The Flask API was restarted while a scan was in progress. Network scans run in a background thread — when Flask restarts, the thread is killed but the database record remains in "running" state indefinitely.

**Steps:**

1. Restart the Flask API. On startup, it automatically marks any scan that has been "running" for more than 5 minutes as "failed" and records "Scan interrupted (server restarted)" in the results.
2. Refresh the **Network Discovery** page. The stuck scan will now show as **Failed**.
3. Run a new scan.

> **NOTE:** This cleanup runs every time the API starts (`create_app()` in `api/app.py`). You do not need to manually update the database — just restart the API and re-scan.

---

### Problem: Docker container 'api' exits immediately

**Cause:** Missing or invalid `api\.env`, or `SUPERADMIN_PASSWORD` not set.

**Steps:**

1. Run `docker-compose logs api` and look for a `RuntimeError` message.
2. Verify `api\.env` exists with all required variables (see Chapter 7a).
3. Ensure `DATABASE_URL` uses `@db:5432`, not `@localhost:5432`.
4. After fixing `.env`, run `docker-compose up -d api` to restart the API container.

---

### Problem: Terminal command stuck in "running" and never produces output

**Cause:** The agent process was killed or the device rebooted while a command was executing. The command record remained in `running` state in the database because the agent never got to mark it complete. When the agent restarts, it can only pick up `pending` commands — the stuck `running` record blocks the queue.

**Steps:**

The system automatically recovers stuck commands. The API resets any command that has been in `running` status for more than 3 minutes back to `pending`, so the agent can retry on its next poll cycle. You do not need to do anything manually — wait 3 minutes after the agent restarts and the command will be re-queued automatically.

If you want to clear the stuck command immediately without waiting:

```
UPDATE terminal_commands SET status = 'pending', picked_up_at = NULL
WHERE status = 'running';
```

---

### Problem: Agent console shows UnicodeDecodeError in _readerthread

**Cause:** Python 3.14 changed how `subprocess` handles text mode. When a subprocess produces non-ASCII output (e.g., winget's progress bars containing Unicode block characters like ■■■), the internal `_readerthread` can crash even if `errors="replace"` was specified.

**This issue is fixed in the current agent build.** All subprocess calls in `collector.py`, `script_runner.py`, and `terminal_worker.py` have been updated to use binary mode — `text=True` has been removed from all subprocess calls. Output is decoded manually using `.decode("utf-8", errors="replace")`, which safely replaces any undecodable byte with a placeholder character instead of crashing.

If you see this error on an older agent deployment, update the agent files (see Chapter 39 for auto-update) or redeploy using the latest build (see Chapter 40 for USB deployment).

---

### Problem: Client user sees empty portal after login

**Cause:** The client user account was not linked to a Customer record when it was created, or was assigned the wrong role.

**Steps:**

1. Log in as an admin.
2. Go to **Admin** → **Users** tab.
3. Find the client account.
4. Click **Edit** and verify:
- **Role** is set to `client`
- **Customer** field is set to the correct customer
5. Click **Save**.
6. Ask the client to log out and back in.

---

### Problem: Frozen agent executable crashes immediately on the target machine

**Cause:** The `config.ini` was not updated before building the executable, or the target machine cannot reach the API server on port 5000.

**Steps:**

1. Verify `config.ini` in the `rmm-agent-dist` folder contains the correct server IP:

```
[api]
url = http://YOUR_SERVER_LAN_IP:5000
```

1. Verify `device_id` and `agent_token` are blank (for a fresh registration on a new machine):

```
[agent]
device_id =
agent_token =
```

1. From the target machine, open a browser and test: `http://YOUR_SERVER_IP:5000/api/health` — if this returns a JSON response, the API is reachable.
2. If unreachable, check Windows Firewall on the server machine allows inbound TCP on port 5000.
3. Check `rmm_agent.log` in the same folder as the executable for the specific error message.

---

## Starting All Services — Quick Reference

```
# Verify PostgreSQL is running
Get-Service | Where-Object {$_.DisplayName -like "*postgresql*"}

# Verify Redis/Memurai is running
Test-NetConnection -ComputerName localhost -Port 6379

# Terminal 1 — Flask API
Set-Location C:\RMM\RemoteManagementSystem\api
.\venv\Scripts\Activate.ps1
python app.py

# Terminal 2 — Celery Worker
Set-Location C:\RMM\RemoteManagementSystem\api
.\venv\Scripts\Activate.ps1
celery -A tasks.celery_app worker --pool=solo -l info

# Terminal 3 — Celery Beat
Set-Location C:\RMM\RemoteManagementSystem\api
.\venv\Scripts\Activate.ps1
celery -A tasks.celery_app beat -l info

# Terminal 4 — Streamlit Dashboard
Set-Location C:\RMM\RemoteManagementSystem\dashboard
.\venv\Scripts\Activate.ps1
streamlit run app.py

# On each managed machine — Agent (as Administrator)
Set-Location C:\RMM\agent
.\venv\Scripts\Activate.ps1
python rmm_agent.py
```

---

# Chapter 33: API Input Validation

## What it is

All write endpoints in the RMM API now enforce strict input validation using Marshmallow schemas. Every `POST`, `PUT`, and `PATCH` request is validated before reaching the database. Invalid or malformed requests are rejected immediately with a structured error response.

This applies to all 30+ write operations: login, device management, ticket creation, billing, alert rules, scripts, automation profiles, network scans, org settings, and more.

## Who uses this information

Developers integrating with the API, and administrators troubleshooting API errors.

## How validation errors are returned

When a request fails validation, the API returns HTTP `400 Bad Request` with a JSON body identifying exactly which field failed and why:

```
{
"error": "Validation failed",
"details": {
"email": ["Not a valid email address."],
"password": ["Missing data for required field."]
}
}
```

The `details` object maps each failing field name to a list of error messages. Multiple fields can fail in a single response.

## Schema files

All schemas live in `api/schemas/`. One file per domain:

| File | Covers |
|---|---|
| `api/schemas/auth.py` | Login, password change, MFA, password reset |
| `api/schemas/agents.py` | Agent registration, heartbeat, software inventory, patches |
| `api/schemas/customers.py` | Customer create/update, device groups |
| `api/schemas/devices.py` | Device update, queue task, deploy patches |
| `api/schemas/tickets.py` | Ticket create/update, comments |
| `api/schemas/alerts.py` | Alert rule create/update |
| `api/schemas/scripts.py` | Script create/update, run script |
| `api/schemas/admin.py` | User create/update |
| `api/schemas/automation.py` | Automation profile create/update |

| File | Covers |
|---|---|
| `api/schemas/network.py` | Network scan, agentless device upsert |
| `api/schemas/billing.py` | Invoice generation, invoice status |
| `api/schemas/org_settings.py` | Org settings update |

## The validation decorator

The `@validate_body` decorator in `api/utils/validation.py` gates each endpoint. If validation fails, it returns `400` before the route function runs. Route internals are unchanged — the decorator is purely additive.

```
from utils.validation import validate_body
from schemas.auth import LoginSchema

@auth_bp.route("/login", methods=["POST"])
@validate_body(LoginSchema)
def login():
data = request.get_json()
# data is guaranteed valid here
```

## Key validation rules by domain

**Authentication:** `email` must be a valid email format. `password` is required. `new_password` is required for all password-change endpoints.

**Agents:** `org_token` and `hostname` required for registration. `platform` must be one of: `windows`, `mac`, `linux`, `android`, `ios`, `unknown`. `cpu_cores` integer 1–256. `cpu_pct`, `ram_pct`, `disk_pct` float 0–100.

**Tickets:** `title` and `customer_id` required for creation. `priority` must be one of: `low`, `medium`, `high`, `critical`. `status` must be one of: `open`, `in_progress`, `resolved`, `closed`.

**Scripts:** `file_type` must be one of: `bat`, `ps1`, `py`. Script content is capped at 512 KB. `device_ids` minimum 1 device required to run.

**Org Settings:** `primary_color` must match hex pattern `#RRGGBB`.

**Unknown fields:** All schemas use `EXCLUDE` mode — extra fields in a request body are silently ignored, not rejected. Partial updates (PUT with a subset of fields) always succeed.

## Adding a new validated endpoint

1. Create a schema in the appropriate `api/schemas/*.py` file, extending `Schema` from marshmallow.
2. Import `validate_body` from `api/utils/validation.py`.
3. Apply `@validate_body(YourSchema)` to the route, after `@jwt_required()` and before the handler.

# Chapter 34: Continuous Integration (CI/CD)

## What it is

The project includes a GitHub Actions CI pipeline that automatically runs the full test suite on every code push and pull request to the `main` branch. A failing test blocks the merge — no broken code reaches main without being caught first.

## Who uses this information

Developers and maintainers.

## Pipeline configuration

File: `.github/workflows/ci.yml`

The pipeline runs on `ubuntu-latest` with a 10-minute timeout. Steps:

1. **Checkout** — clones the repository at the pushed commit.
2. **Set up Python 3.11** — with pip cache keyed on `api/requirements.txt` for fast repeat runs.
3. **Install dependencies** — runs `pip install -r api/requirements.txt`.
4. **Run pytest** — runs `python -m pytest tests/ -v --tb=short` from the `api/` working directory.

No PostgreSQL or Redis are needed in CI. The pipeline uses `sqlite:///:memory:` — created fresh and torn down in-memory on every run. All required environment variables are supplied inline via the workflow `env:` block. No secrets are stored in the repository.

## Environment variables in CI

| Variable | CI value |
|---|---|
| `SECRET_KEY` | 32-char test string |
| `JWT_SECRET_KEY` | 32-char test string |
| `DATABASE_URL` | `sqlite:///:memory:` |
| `SUPERADMIN_PASSWORD` | `CITestSuperAdmin@1` |
| `ORG_REGISTRATION_TOKEN` | `ci-org-token-unique-secret-value-here` |

## What the tests cover

The test suite at `api/tests/` covers authentication (login, token refresh, MFA, password reset), agent registration and heartbeat, device management, ticket and customer CRUD, alert rule evaluation, script creation and dispatch, billing invoice generation, admin and org settings endpoints, and schema validation (invalid payloads rejected with 400 errors).

Current count: **51 tests, all passing.**

### Running tests locally

```
Set-Location C:\RMM\RemoteManagementSystem\api
.\venv\Scripts\Activate.ps1
pytest tests/ -v --tb=short
```

Expected: `51 passed` (plus deprecation warnings about `Model.query.get()` — non-breaking, informational only).

### Extending CI

To add a new test: create a file in `api/tests/` following the existing pattern. The pipeline picks it up automatically on the next push — no workflow changes needed.

To add a workflow step (linting, security scan, etc.): edit `.github/workflows/ci.yml` and add a step under `jobs.test.steps`.

---

# Chapter 35: Load Testing

## What it is

The project includes a Locust load test scaffold that simulates concurrent RMM agents and dashboard users hitting the API. Use it to establish baseline performance numbers, find bottlenecks before production deployment, and validate the system under stress.

## Who uses this information

Developers, infrastructure engineers, and administrators preparing for production deployment.

## Setup

```
pip install locust
```

Before running, edit `tests/load/locustfile.py` line 20 and set `ORG_TOKEN` to match `ORG_REGISTRATION_TOKEN` in `api/.env`. Update `ADMIN_EMAIL` and `ADMIN_PASSWORD` if the superadmin credentials differ from defaults.

## Running the load test

Start all services first (API + PostgreSQL + Redis). Celery is optional unless testing patch deployment or automation tasks.

```
locust -f tests/load/locustfile.py --host http://localhost:5000
```

Open **http://localhost:8089** in a browser. Set the user count and spawn rate, then click **Start**.

## Simulated user types

**AgentUser** — simulates a registered RMM agent:

- On start: registers as a new agent with a unique hostname and random MAC address
- Every ~30 seconds: sends a heartbeat with randomised CPU/RAM/disk readings (task weight 10)
- Occasionally: polls for queued tasks (weight 2) or submits a software inventory update (weight 1)
- Handles token rotation: if a heartbeat response includes `new_agent_token`, the agent updates its token automatically

**DashboardUser** — simulates a logged-in technician:

- On start: logs in as superadmin and stores the access token
- Requests: list devices (weight 5), list alerts (weight 3), list tickets (weight 2), dashboard summary (weight 2), health check (weight 1)

## Recommended scenarios

| Scenario | AgentUser | DashboardUser | Spawn rate | Goal |
|---|---|---|---|---|
| Baseline | 50 | 5 | 5/s | Confirm heartbeat p95 < 300ms |
| Normal load | 100 | 10 | 10/s | Sustain 10 minutes, error rate 0% |
| Stress | 500 | 50 | 20/s | Find where errors begin appearing |
| Spike | 0→200 in 10s | 0→20 in 10s | 20/s | Verify recovery after sudden load |

## What to watch

- **Heartbeat p95 latency** — target under 300ms at normal load
- **Error rate** — target 0% under normal load, under 1% under stress
- **PostgreSQL connections** — monitor `pg_stat_activity` when errors spike
- **Flask process memory (RSS)** — watch for growth under sustained 500-agent load

## Recording results

After each run, add the results to `tests/load/README.md` using the results template in that file.

---

# PART IX — ADVANCED FEATURES

# Chapter 36: Support Inbox — Email-to-Ticket

## Who uses this chapter

Administrators who want to connect a support email address so that inbound customer emails automatically create help-desk tickets. Also useful for technicians to understand where "Email" source tickets come from.

## What it does

The Support Inbox feature polls a dedicated email mailbox (Gmail, Outlook, or any IMAP-compatible mail server) every 60 seconds. When a new email arrives:

1. A ticket is created automatically in the Tickets system.
2. An auto-reply is sent to the sender confirming the ticket number.
3. If the sender's email matches a known customer contact, the ticket is linked to that customer. Unknown senders are assigned to the default customer configured in `.env`.
4. If a reply email references an existing ticket's Message-ID thread header, the reply is added as a comment on that ticket rather than creating a duplicate.

The feature is entirely config-gated — it does nothing if the IMAP environment variables are absent.

## Activation: Setting up the email connection

All configuration is in the `.env` file on the RMM server. Open it with a text editor and fill in the IMAP section:

```
SUPPORT_IMAP_HOST=imap.gmail.com
SUPPORT_IMAP_PORT=993
SUPPORT_IMAP_USER=support@yourcompany.com
SUPPORT_IMAP_PASSWORD=your-app-password
DEFAULT_INBOUND_CUSTOMER_ID=
```

| Variable | What to put here |
|---|---|
| `SUPPORT_IMAP_HOST` | Your mail server's IMAP hostname. Gmail: `imap.gmail.com`. Outlook: `outlook.office365.com` |
| `SUPPORT_IMAP_PORT` | IMAP SSL port. Almost always `993` |
| `SUPPORT_IMAP_USER` | The full email address of the support inbox (e.g. `support@yourcompany.com`) |
| `SUPPORT_IMAP_PASSWORD` | An **app password** — not your account password (see note below) |
| `DEFAULT_INBOUND_CUSTOMER_ID` | UUID of the customer to assign when sender is unknown. Find it in Admin → Customers. Leave blank to discard emails from unrecognised senders |

**IMPORTANT:** Gmail and Microsoft 365 require an **app password** rather than your main account password. For Gmail: Google Account → Security → 2-Step Verification → App passwords. For Outlook: Microsoft Account → Security → Advanced security options → App passwords. Using your main password will result in authentication failures.

After editing `.env`, restart the Flask API and Celery worker. The inbox poller starts automatically with Celery beat — no further configuration is needed.

### How tickets appear

Tickets created from inbound email have:

- **Source:** `email` (shown as a tag on the Tickets page)
- **Title:** the email Subject line
- **Description:** the email body (plain text)
- **Customer:** matched by sender email address, or the default customer
- **Priority:** `medium` by default
- **Status:** `open`

The sender receives an automatic reply: *"Your message has been received and logged as ticket #NNNN. We will respond shortly."*

### Threading: keeping replies together

When a customer replies to the auto-reply email (or any ticket email), their reply includes a `References` or `In-Reply-To` header containing the original Message-ID. The system uses this to attach the reply as a comment on the existing ticket rather than creating a new one. This works automatically without any action from the technician.

> **NOTE:** If a customer composes a brand-new email on the same topic instead of replying, the system has no way to link it to the existing ticket — it will create a new ticket. Technicians can manually merge or close duplicates from the Tickets page.

### Turning it off

Remove or blank out `SUPPORT_IMAP_HOST` in `.env` and restart Celery beat. The poller silently does nothing when the host variable is absent.

---

# Chapter 37: Remote Terminal

## Who uses this chapter

Administrators and technicians who need to run commands directly on a managed Windows machine without physically accessing it or using a separate remote-desktop tool.

## What it is

Remote Terminal gives you a browser-based command shell into any online Windows device with an RMM agent installed. You type a command in the dashboard, press Enter, and see the output appear within seconds — as if you were sitting at that machine's command prompt.

Access is restricted to **admin**, **technician**, and **superadmin** roles. Viewer and client roles cannot use Remote Terminal.

## Opening a terminal session

1. Click **Remote Terminal** in the sidebar (under the Tools section).
2. Select the target device from the **Connect to device** dropdown. Only online, agent-managed devices appear.
3. Click **Connect**. A session is established and the terminal panel appears.
4. Type a command in the input box at the bottom and press **Enter** (or click **Send**).
5. Output appears in the dark terminal panel within 1–3 seconds.
6. When finished, click **Disconnect** to close the session cleanly.

**NOTE:** Only one terminal session can be active per device at a time. Opening a new session from the same or a different dashboard user will close any previous session on that device.

## Reading the output

The terminal panel uses colour coding:

| Colour | Meaning |
|---|---|
| White (`#E2E8F0`) | Standard output (stdout) — normal command output |
| Red (`#FF6B6B`) | Error output (stderr) — error messages and warnings |
| Grey (`#94A3B8`) | System messages — session opened/closed, timeout notices |

## Limitations and safety boundaries

| Limit | Value | Reason |
|---|---|---|
| Command timeout | 120 seconds | Prevents runaway processes from blocking the agent |
| Output cap | 512 KB per session | Prevents memory exhaustion from large command output |

| Limit | Value | Reason |
|---|---|---|
| Idle timeout | 30 minutes | Sessions with no activity are automatically closed |
| Scope | Windows only | Agent uses Windows `cmd.exe` shell via `subprocess` |

> **WARNING:** Commands run with the same privileges as the RMM agent process. If the agent is running as a local administrator (required for patch management), terminal commands also run as administrator. Use with caution — destructive commands (format, del /f /s, registry edits) take effect immediately.

### Audit trail

Every command sent through Remote Terminal is logged in the system audit log with:

- The user who sent it
- The device it was sent to
- The command text
- The timestamp

Logs are viewable in **Admin** → **Audit Log**. This provides full accountability for all remote actions.

### How it works (technical)

Remote Terminal uses a polling architecture rather than WebSockets:

- The agent's `TerminalWorker` thread polls the API every 3 seconds for pending commands on any active session.
- The dashboard polls the API every 2 seconds for new output, then reruns to display it.
- This approach works seamlessly within Streamlit's page model without requiring persistent connections.
- Commands run via `subprocess.Popen(shell=True)` on the agent; stdout is streamed line by line, batched in groups of 20 lines, and posted back to the API.

---

# Chapter 38: Real-time Event Feed

## Who uses this chapter

All users who want to stay aware of what is happening across the managed device fleet without manually refreshing individual pages.

## What it is

The Real-time Event Feed is a live log of significant system events displayed at the bottom of the main Dashboard page. It shows the most recent 20 events and refreshes automatically every 30 seconds.

Events shown include:

| Event | What triggered it | Icon |
|---|---|---|
| Device Online | An agent that was offline sends its first successful heartbeat | ■ |
| Device Offline | Celery marks a device offline after missing 3 heartbeats | ■ |
| Device Status Changed | A device's status changes (e.g. healthy → warning) | ■ |
| New Alert | An alert rule fires and creates a new alert | ■ |
| New Ticket | A ticket is created (from the dashboard or via inbound email) | ■ |

## Where to find it

Scroll to the bottom of the **Dashboard** page (the first page after login). The Live Events panel appears below the Activity Feed section.

## Requirements

The Event Feed requires Redis to be running. If Redis is unavailable:

- The Live Events panel shows an empty list (no error is displayed to the user).
- All other dashboard functionality continues to work normally.
- Events are not stored or queued while Redis is down; they are lost for that period.

When Redis recovers, new events will begin appearing again automatically.

### Event retention

The system retains the last 100 events in Redis with a 1-hour expiry. The dashboard displays the most recent 20. Older events are not stored in the database and cannot be retrieved after the 1-hour window.

### SSE endpoint for external integrations

For third-party integrations (custom dashboards, monitoring tools, notification bots), the system exposes a Server-Sent Events (SSE) stream:

```
GET /api/events/stream?token=<jwt_token>
```

The stream sends a JSON event on every system event and a keepalive every 15 seconds. The connection automatically closes after 60 seconds; clients should reconnect (the browser EventSource API does this automatically).

A polling alternative is also available for clients that cannot use SSE:

```
GET /api/events/recent?limit=20
Authorization: Bearer <jwt_token>
```

**NOTE:** The SSE token must be passed as a query parameter (`?token=`), not as a header, because the browser EventSource API does not support custom headers.

# Chapter 39: Agent Auto-Update

## Who uses this chapter

Administrators responsible for keeping the RMM agent software up to date across all managed devices.

## What it is

Agent Auto-Update allows the RMM server to push new agent code to all managed Windows devices automatically, without physically visiting each machine. Once configured, agents check for updates every 6 hours and self-update if a newer version is available.

## How to push an update

**Step 1 — Build the update package.**

Collect the updated agent `.py` files into a folder and create a zip file:

```
# From the project root
Compress-Archive -Path agent\* -DestinationPath agent_update_v1.1.0.zip
```

Only `.py` and `.txt` files are applied by the agent during update. Configuration files (`config.ini`), logs (`rmm_agent.log`), and identity files (`device_id`, `agent_token`) are automatically preserved — agents keep their registered identity after updating.

**Step 2 — Place the zip on the server.**

Copy the zip to a stable path on the RMM server, for example:

```
C:\RMM\updates\agent_update_v1.1.0.zip
```

**Step 3 — Edit `.env` on the RMM server.**

```
LATEST_AGENT_VERSION=1.1.0
AGENT_UPDATE_PATH=C:\RMM\updates\agent_update_v1.1.0.zip
AGENT_CHANGELOG=Bug fixes and performance improvements
```

| Variable | What to set |
|---|---|
| `LATEST_AGENT_VERSION` | The new version string (e.g. `1.1.0`). Agents running an older version will download the update. |
| `AGENT_UPDATE_PATH` | Absolute path to the zip file on the server. Leave blank to block downloads (agents will see `update_available=true` but cannot download). |
| `AGENT_CHANGELOG` | One-line description shown in the dashboard. Optional. |

**Step 4 — Restart the Flask API.**

```
# Kill the API process then restart
cd api
python app.py
```

Within 6 hours (at each agent's next update check cycle), all online agents will download, verify, and apply the update and restart themselves.

### Checking update status in the dashboard

Open **Devices** and look at the agent version in each device's detail panel. Devices running an older agent version than `LATEST_AGENT_VERSION` show an orange **UPDATE** badge next to the version number. Once updated, the badge disappears.

### How the update applies (technical)

1. Agent calls `GET /api/agents/update/check?version=X.Y.Z` using its existing agent token.
2. API compares the agent's version to `LATEST_AGENT_VERSION`. If the agent is behind, it returns the download URL and SHA256 checksum.
3. Agent downloads the zip from `GET /api/agents/update/download`.
4. Agent verifies the SHA256 checksum of the downloaded file. If verification fails, the update is aborted and the agent continues running the current version.
5. Agent extracts the zip to a temp directory, then copies `.py` and `.txt` files to the agent folder (skipping config, log, and identity files).
6. Agent calls `os.execv(sys.executable, [sys.executable] + sys.argv)` — this atomically replaces the running process with the new code. The new version starts with the same device identity and token.

> **NOTE:** `os.execv` is an atomic process replacement — no zombie processes, no restart gap. The agent is never fully offline during an update; the gap is typically under one second.

### Safety and rollback

- The update is skipped entirely if the SHA256 checksum does not match.
- Config, identity files, and logs are never overwritten.
- There is no automatic rollback. To roll back, set `LATEST_AGENT_VERSION` back to the previous version and place the old zip at `AGENT_UPDATE_PATH`. Agents that updated will apply the rollback package at their next check.
- Keep the previous zip file until the new version is confirmed working on all devices.

### Update check schedule

By default, agents check every **6 hours** (`update_check_interval = 21600` in `config.ini`). To change the interval on a specific agent, edit its `config.ini`:

```
[agent]
update_check_interval = 3600 ; check every hour
```

---

# PART X — EXTENDED DEPLOYMENT AND OPERATIONS

# Chapter 40: Frozen Executable Agent Deployment (USB / No-Python Machines)

## Who uses this chapter

Administrators who need to install the RMM agent on a Windows machine that does not have Python installed, and where installing Python is not practical or permitted. This chapter covers building a self-contained `.exe` from the agent source and deploying it via USB drive or direct file copy.

## What it is

The agent can be compiled into a single Windows executable using PyInstaller. The resulting file contains the Python runtime, all dependencies, and the agent code — it runs on any Windows 10 or 11 machine with no prior setup. No Python installation, no virtual environment, no `pip install` is required on the target machine.

## Prerequisites (on the build machine — the RMM server)

- Python 3.11 or later with the agent virtual environment active
- PyInstaller installed: `pip install pyinstaller`
- The agent virtual environment must have all agent dependencies installed

## Step 1 — Build the executable

Open a terminal on the RMM server, activate the agent virtual environment, and run:

```
Set-Location C:\RMM\RemoteManagementSystem\agent
.\venv\Scripts\Activate.ps1
pyinstaller --onefile --name rmm_agent rmm_agent.py
```

PyInstaller produces a single file at `agent\dist\rmm_agent.exe`. This file is approximately 15–25 MB depending on the Python version and installed packages.

> **NOTE:** The build takes 1–3 minutes. Warnings about missing optional imports during the build are normal and do not affect the output.

## Step 2 — Prepare the deployment folder

Create a deployment folder (the example uses `C:\Users\rigwe\Desktop\rmm-agent-dist`). Copy in:

1. `agent\dist\rmm_agent.exe` — the compiled executable
2. `agent\config.ini` — the agent configuration file (see Step 3 before copying)

The deployment folder must contain both files side by side:

```
rmm-agent-dist\
■■■ rmm_agent.exe
■■■ config.ini
```

### Step 3 — Configure config.ini for the target network

**This is the most important step.** The `config.ini` inside the deployment folder must point to the RMM server's LAN IP address — not `localhost`. Open `rmm-agent-dist\config.ini` and set it as follows:

```
[api]
url = http://192.168.1.100:5000
org_token = YOUR_ORG_REGISTRATION_TOKEN

[agent]
device_id =
agent_token =
heartbeat_interval = 60
software_interval = 21600
version = 1.0.0

[logging]
level = INFO
file = rmm_agent.log
```

| Field | What to set |
|---|---|
| `url` | Replace `192.168.1.100` with the actual LAN IP of the machine running the Flask API. Find this by running `ipconfig` on the server and noting the IPv4 address. |
| `org_token` | Copy from Admin → System Info → Agent Enrollment Token → Reveal in the dashboard. |
| `device_id` | Leave blank. The agent generates and saves its own ID on first registration. |
| `agent_token` | Leave blank. The API issues this on first registration. |

**WARNING:** Never copy `config.ini` from the `agent\` source folder directly — that file contains `url = http://localhost:5000`, which only works on the server itself. Always edit the copy in the deployment folder to use the server's LAN IP.

### Step 4 — Transfer to the target machine

Copy the entire `rmm-agent-dist` folder to the target machine. Options:

- **USB drive:** Copy the folder to a USB drive, plug into target, copy to local disk (e.g., `C:\RMM-Agent\`)
- **Network share:** If the target can reach a shared folder on the server, copy directly
- **Email / file link:** For remote deployments, compress the folder as a zip and send to the user

**TIP:** Always copy the folder to a local disk path before running. Running directly from a USB drive works but is slower and can fail if the drive is removed.

### Step 5 — Run the agent on the target machine

On the target Windows machine:

1. Open File Explorer and navigate to the agent folder (e.g., `C:\RMM-Agent\`).
2. Right-click `rmm_agent.exe` and select **Run as administrator**. Administrator rights are required for patch scanning and WMI hardware queries.
3. A console window opens showing the agent startup log. Within 30 seconds you should see:

- `Registered device: <device-id>`
- `Heartbeat sent`
- `Terminal worker started`

4. The agent writes its registration credentials to `agent_state.json` in the same folder. On subsequent runs, it reads this file and skips re-registration.

> **NOTE:** If a Windows Defender SmartScreen warning appears ("Windows protected your PC"), click **More info** → **Run anyway**. This is normal for unsigned executables not yet seen by Microsoft's telemetry.

### Step 6 — Verify in the dashboard

1. Open the RMM dashboard at http://YOUR-SERVER-IP:8501.
2. Go to **Devices** — the new device should appear within 60 seconds of the agent starting.
3. Verify the hostname, IP address, and OS match the target machine.

### Running the agent as a Windows Service (optional — for always-on deployments)

To have the agent start automatically when the machine boots, register it as a Windows Service using NSSM (Non-Sucking Service Manager):

```
# Download nssm from nssm.cc and place nssm.exe in C:\RMM-Agent\
nssm install RMMAgent "C:\RMM-Agent\rmm_agent.exe"
nssm set RMMAgent AppDirectory "C:\RMM-Agent"
nssm set RMMAgent Start SERVICE_AUTO_START
nssm start RMMAgent
```

The service will now start automatically at boot and restart if it crashes.

### Updating a frozen deployment

The Agent Auto-Update feature (Chapter 39) does not apply to frozen `.exe` deployments because the auto-update mechanism replaces `.py` source files, not a compiled binary. To update a frozen deployment:

1. Build a new `rmm_agent.exe` on the server with the updated code.
2. Stop the agent on the target machine (`taskkill /F /IM rmm_agent.exe` or stop the service).
3. Copy the new `rmm_agent.exe` to the target machine (USB or network).
4. Restart the agent. The existing `agent_state.json` preserves the device registration — no re-registration needed.

> **TIP:** Keep the old `device_id` and `agent_token` values from `agent_state.json` if you need to verify continuity. The device record in the dashboard will continue accumulating history under the same device entry.

# Chapter 41: Agent Resilience — Unicode Safety, Command Timeout, and Stuck Command Recovery

## Who uses this chapter

Developers maintaining the agent codebase, and administrators troubleshooting agent console errors or terminal commands that appear stuck.

## Background

Three resilience improvements were made to the agent after deployment to Windows 11 environments running Python 3.14. This chapter documents what changed and why, so future maintainers understand the design decisions.

## 1 — Unicode Safety in Subprocess Output

**Problem:** Python 3.14 changed the internal `_readerthread` used by `subprocess` when `text=True` is set. Even when `encoding="utf-8"` and `errors="replace"` are specified, the new `_readerthread` implementation raises `UnicodeDecodeError` when a subprocess produces non-ASCII bytes — for example, winget's progress bars contain Unicode block characters (■■■) that fall outside the ASCII range.

**Fix:** All subprocess calls in the agent have been changed from text mode to binary mode. The `text=True` parameter has been removed from every `subprocess.run()` and `subprocess.Popen()` call. Output bytes are decoded manually:

```
# Before (crashes on Python 3.14 with non-ASCII output):
result = subprocess.run(cmd, capture_output=True, text=True,
encoding="utf-8", errors="replace")
stdout = result.stdout

# After (safe on all Python versions):
result = subprocess.run(cmd, capture_output=True)
stdout = result.stdout.decode("utf-8", errors="replace")
```

**Files affected:**

- `agent/collector.py` — patch scanner PowerShell call and winget list call
- `agent/script_runner.py` — script execution
- `agent/terminal_worker.py` — remote terminal command execution

**Operational impact:** None visible to users. The fix is internal to the agent. Winget progress-bar lines that contain block characters are already filtered out separately by `_get_winget_software()` in `collector.py`.

## 2 — Thread-Level Command Timeout

**Problem:** In `terminal_worker.py`, the original implementation checked a deadline inside the `for raw_line in proc.stdout` loop. If a subprocess's stdout pipe never closed (for example, an interactive program waiting for input, or a network command that hangs indefinitely), the loop would block the terminal worker thread permanently. No further commands in any session would be executed until the agent restarted.

**Fix:** A `threading.Timer` is started immediately after the subprocess is launched. If the command has not completed within `_CMD_TIMEOUT` seconds (120 seconds by default), the timer fires on a separate thread and calls `proc.kill()`. This guarantees the subprocess is killed even if the stdout loop is hanging, because the kill happens on a different thread and does not depend on the loop completing.

```
Timeline:
t=0s Command starts, Timer(120s) armed
t=0-Ns stdout lines arrive, timer is cancelled on normal completion
t=120s Timer fires (separate thread) → proc.kill()
→ "[Killed: command exceeded timeout]" posted to output
→ stdout loop unblocks, terminal worker continues
```

**Operational impact:** Commands that run longer than 120 seconds are forcibly killed and the terminal shows `[Killed: command exceeded timeout]`. This is intentional. Use scripts (Chapter 21) for long-running operations — scripts run via the agent's dedicated task executor and are not subject to the terminal timeout.

## 3 — Stuck Command Auto-Recovery

**Problem:** When the agent process is killed mid-execution (device rebooted, agent crashed, process killed manually), any command that was in `running` state in the database stays `running` permanently. When a new agent starts, it only picks up `pending` commands — the stuck `running` record is never retried and the terminal appears to ignore new commands on the same session.

**Fix:** The `_expire_idle_sessions()` function in `api/routes/terminal.py` now includes a second query that resets stuck commands. It runs every time the agent polls for sessions (every 3 seconds):

```
Any terminal command with status = 'running'
AND picked_up_at older than 3 minutes
→ automatically reset to status = 'pending', picked_up_at = NULL
```

The agent picks up the now-pending command on its next poll cycle and re-executes it.

**Operational impact:** After an agent restart, stuck commands self-heal within 3 minutes. No manual database intervention is needed. If a command was partially executed before the agent died, it will run again from the beginning — commands sent via Remote Terminal should be treated as potentially re-runnable.

---

# Chapter 42: Client Portal — Self-Service Ticket Access

## Who uses this chapter

Administrators who want to give customer contacts the ability to submit and track their own support tickets without calling in. Also useful for technicians to understand how client-submitted tickets arrive.

## What it is

The Client Portal is a restricted view of the ticketing system accessible only to users with the **client** role. It allows end-customers to:

- Submit new support tickets
- View the status of all tickets linked to their customer account
- Read technician responses (public comments) on their tickets
- See when their ticket was last updated

Clients log in at the same URL as staff. After login, they are automatically redirected to the Client Portal — they cannot navigate to any other part of the system.

## Setting up a client account

1. Log in as an admin.
2. Go to **Admin** → **Users** tab → click **+ New User**.
3. Fill in the user's details:

- **Full Name:** The contact's name
- **Email:** Their email address (this is their login username)
- **Password:** Set a temporary password and tick **Force Password Change on Next Login** so they set their own password on first access
- **Role:** Select `client`
- **Customer:** Select the customer organisation this contact belongs to

4. Click **Create User**.
5. Share the dashboard URL and their credentials with the client contact.

> **IMPORTANT:** The **Customer** field is mandatory for client accounts. If it is left blank, the client will see an empty portal with no tickets and no way to submit one. Always assign a customer before saving.

## What the client sees

When a client logs in, they land on the **Client Portal** page (`21_Client_Tickets.py`) which shows:

- A list of all tickets linked to their customer
- Ticket status badges (open, in progress, resolved, closed)
- A form to submit a new ticket

Tickets submitted through the portal appear in the main Tickets list with **Source: PORTAL** (shown as a green badge). Technicians can reply by adding a public comment — the client sees these replies on their next login.

**NOTE:** Internal notes (comments marked "Internal") are not visible to client users. Use internal notes for team-only discussion that should not reach the client.

## Client Portal vs. email-to-ticket

| Method | How it works | Best for |
|---|---|---|
| Client Portal | Client logs in, submits form | Clients who prefer self-service, need to track multiple tickets |
| Email-to-Ticket (Chapter 36) | Client emails a support address | Clients who prefer email, no login required |

Both methods can be active simultaneously. Tickets from email show **Source: EMAIL**; tickets from the portal show **Source: PORTAL**.

## Deactivating a client account

If a client contact leaves the organisation or should no longer have access:

1. Go to **Admin** → **Users**.
2. Find the account.
3. Click **Edit** → set **Active** to off (or click **Deactivate**).
4. The account cannot log in but the ticket history is preserved.

---

# Chapter 43: Departments — Grouping Users and Tickets

## Who uses this chapter

Administrators who want to organise their team into functional groups (e.g., Help Desk, Infrastructure, Security) and route tickets to the correct team automatically or manually.

## What departments are

A department is a named group with an optional color code. Users can be assigned to a department. Tickets can be assigned to a department to indicate which team is responsible.

Departments are optional. You can run the full RMM system without creating any departments — they become useful once your team is large enough that routing matters.

## Creating a department

1. Go to **Admin** → **Departments** tab.
2. Click **+ New Department**.
3. Enter a **Name** (e.g., `Help Desk`, `Infrastructure`, `Security`).
4. Optionally pick a **Color** — this color appears as an accent on department badges throughout the dashboard. The default is the system green (#407E3C).
5. Click **Create**.

## Assigning users to a department

1. Go to **Admin** → **Users** tab.
2. Click **Edit** on a user.
3. Select their **Department** from the dropdown.
4. Click **Save**.

A user can belong to only one department. Unassigned users show no department badge.

## Assigning tickets to a department

When creating or editing a ticket, select the **Department** field to indicate which team should handle it. Tickets assigned to a department appear in the table with a department badge in the assignee column area.

Technicians can filter the ticket table by department to see only their team's workload.

## Department color coding

Department badges use the color set when the department was created. This makes it visually easy to distinguish tickets owned by different teams at a glance in the ticket table.

---

| Term | Definition |
|---|---|
| Agent | The Python program (`rmm_agent.py`) installed on managed Windows machines. Sends heartbeats every 60 seconds and executes remote commands. Can be deployed as Python source (requires Python on the target) or as a frozen executable (no Python required — see Chapter 40). |
| Assignee | The staff member (technician or admin) currently responsible for resolving a ticket. Set on the ticket detail page. |
| Alert | An automatic notification generated when a device metric crosses a configured threshold. |
| Alert Rule | A configuration defining when alerts trigger: which metric, which threshold, with what severity. |
| API | Application Programming Interface. The Flask server at port 5000 that handles all data operations. |
| Automation Profile | A bundle of maintenance tasks (patching, disk, cleanup) scheduled to run automatically. |
| BAT | Windows Batch script file format. |
| Celery | A distributed task queue. Runs background tasks: alert evaluation, patch deployment, script dispatch. |
| Client | A user role for customer contacts. Grants access only to the Client Portal — own tickets only. No access to devices, alerts, or management features. See Chapter 42. |
| Client Portal | The self-service ticketing interface for client-role users. Accessible at the same URL as the main dashboard. Clients submit and track their own tickets here. |
| Celery Beat | The Celery scheduler component. Triggers tasks on a schedule (every 60 seconds, daily profiles, etc.). |
| Compliance % | Percentage of managed devices that are fully patched and up to date. |
| Currency | ISO 3-letter code (USD, EUR, GBP, etc.) configured in Admin → Org Settings → Regional Settings. Controls the symbol shown on all billing amounts. |
| Cooldown | Minimum time before an alert rule can fire again for the same device. Prevents alert flooding. |
| Critical | Highest alert severity. Immediate action required. |
| Dashboard | The Streamlit web interface at port 8501. Also specifically refers to the Overview page. |
| Department | A named group used to organise staff users and route tickets. Optional. Created in Admin → Departments. See Chapter 43. |
| Docker | Container platform. Used for one-command deployment via `docker-compose up -d`. See Chapter 7a. |

| Term | Definition |
|---|---|
| Defragmentation | Disk maintenance for HDDs that reorganizes fragmented files. Never run on SSDs. |
| Device | A managed machine with the RMM agent installed. |
| Exit Code | Number returned by a script when it finishes. 0 = success; non-zero = error. |
| Flask | Python web framework used to build the RMM API server. |
| Frozen Executable | A single `.exe` file produced by PyInstaller that bundles the Python runtime, all dependencies, and the agent code. Runs on Windows machines without Python installed. See Chapter 40. |
| Heartbeat | Regular check-in signal from the agent to the API every 60 seconds, reporting current metrics. |
| HDD | Hard Disk Drive — traditional spinning magnetic disk. Can benefit from defragmentation. |
| Info | Lowest alert severity. Informational only; no immediate action needed. |
| Invoice | A billing document for a customer showing managed devices and the amount owed. |
| JWT | JSON Web Token. The authentication token used by this system. |
| KB | Knowledge Base number. Unique identifier for Windows patches (e.g., KB5034441). |
| Last Seen | Timestamp of the most recent heartbeat from a device's agent. |
| MFA | Multi-Factor Authentication. A second login step requiring a 6-digit code from an authenticator app. Enabled per user from the My Profile page. |
| Memurai | Windows-native Redis-compatible server. Used as the Redis implementation on Windows. |
| Offline | A device whose agent has not sent a heartbeat recently. |
| Online | A device whose agent is actively sending heartbeats. |
| Org Token | Organization-level authentication token in `config.ini`, used during device registration. Must match the API's `ORG_REGISTRATION_TOKEN`. |
| OS | Operating System. |
| Patch | A software update addressing a security vulnerability, bug, or performance issue. |
| PostgreSQL | The relational database system storing all RMM data. Port 5432. |
| Priority | Ticket urgency level: low, medium, high, or critical. |
| PS1 | PowerShell script file extension. |

| Term | Definition |
|---|---|
| RBAC | Role-Based Access Control — permissions determined by user role. |
| Redis | In-memory data store used as the Celery message broker. Port 6379. |
| Regional Settings | Admin → Org Settings section configuring Currency and Timezone for all billing displays and timestamp formatting. |
| RMM | Remote Monitoring and Management. |
| PyInstaller | A tool that packages Python applications into standalone executables. Used to build the frozen agent `.exe` for machines without Python. |
| Script | Code (PS1, BAT, PY, or SH) that can be executed remotely on a managed device. |
| Session | An active user login. Represented by a JWT token stored in browser memory. |
| SLA | Service Level Agreement. A deadline by which a ticket should be responded to or resolved. The ticket table shows time remaining or "BREACHED" when the deadline has passed. |
| Severity | Alert importance level: info, warning, or critical. |
| SSD | Solid State Drive. Fast storage, no moving parts. Does not benefit from defragmentation. |
| Streamlit | Python web app framework used to build the RMM dashboard. |
| Technician | User role with full operational permissions but no user management or admin access. |
| Tier | Customer support level: standard, premium, or enterprise. |
| Timeout | Maximum time for a script to run before forced termination. |
| Timezone | IANA timezone name (e.g. `Europe/Berlin`, `America/New_York`) configured in Admin → Org Settings → Regional Settings. Timestamps on the dashboard are converted to this timezone for display. |
| Token | See JWT. A cryptographic string proving a user is authenticated. |
| Viewer | User role with read-only access. Cannot make changes. |
| Warning | Medium severity alert. Indicates degraded performance needing attention soon. |
| Winget | Windows Package Manager. Used to install and update software on Windows. |

# APPENDIX B: QUICK REFERENCE CARDS

## Quick Reference Card — New Employee (First Week)

**Your daily starting routine:**

1. Open http://localhost:8501 → log in
2. Check Dashboard stat cards — note any red numbers
3. Click **Overview** — scan the Device Health Map
4. Go to **Alerts** — acknowledge any new critical alerts
5. Go to **Tickets** — check for open or in-progress tickets
6. Respond to client calls — create tickets, investigate devices
7. Update ticket statuses throughout the day
8. Sign out when done (sidebar → Sign Out)

**Creating a ticket:**

Tickets → + New Ticket → Title, Priority, Customer → Create Ticket

**Finding a device:**

Devices → search for hostname → click to expand → read CPU/RAM/disk

**Acknowledging an alert:**

Alerts → Active Alerts tab → expand alert → click Acknowledge

**Updating ticket status:**

Tickets → expand ticket → Status dropdown → new status → Update Status

---

## Quick Reference Card — IT Support Staff

**Priority escalation guide:**

- Client completely down / data at risk → **critical** ticket
- Significant business impact, service degraded → **high** ticket
- Non-urgent issue, one user affected → **medium** ticket
- Request, question, minor task → **low** ticket

**Ticket lifecycle:**

`open` → `in_progress` → `resolved` (client confirms) → `closed`

**When a client calls with a problem:**

1. Create ticket first (documents the call)
2. Note everything they tell you in the description
3. Check Devices for their device health
4. Check Alerts for any related alerts
5. Add comments as you investigate
6. Update status at each stage

**Checking software on a device:**

App Center → select device → search for application name

**Checking if a device is online:**

Devices → find hostname → look at Status column (green dot = online)

---

## Quick Reference Card — Technicians

**Run a script on multiple devices:**

Scripts → Library → find script → expand → select devices → set timeout → Run

**Approve OS patches:**

OS Patches → Pending Patches → check boxes → Approve Selected

**Reboot a device:**

Maintenance → select device → check "I confirm..." → click Reboot

**Create an alert rule:**

Alerts → Alert Rules → Create Alert Rule → fill form → Create Rule

**Create an automation profile:**

Automation → Create / Edit Profile → fill fields → configure task columns → Save Profile

**View disk health:**

Disk Management → select device → view gauges → run cleanup if needed

**Check script output:**

Scripts → Run History → expand run entry → read stdout/stderr

**Alert severity colors:**

- Red = critical (immediate action)
- Amber = warning (attention needed)
- Blue = info (no action needed)

**Ticket status colors:**

- Red = open | Amber = in_progress | Green = resolved | Grey = closed

**Script exit codes:**

- 0 = SUCCESS | Non-zero = FAILED

---

## Quick Reference Card — Managers

**Generating a monthly client report:**

Reports → Generate → select template → select customer → set date range → Generate → Download

**Report templates and what they cover:**

- `device_summary` — device health and status overview
- `alert_summary` — all alerts by severity and device for the period
- `patch_summary` — patch compliance and deployment history
- `billing_summary` — invoices and billing totals

**Checking outstanding invoices:**

Billing → view Outstanding metric at top → find unpaid/overdue invoices in the list

**Creating an invoice:**

Billing → Generate Invoice form → select customer → set period dates, rate, optional tax rate and notes → Generate Invoice → invoice assigned INV-YYYY-NNNN automatically

**Downloading a PDF invoice:**

Billing → View (on any invoice row) → ■ Download PDF

**Emailing an invoice to a client:**

Billing → View → ✉ Send Email (requires SMTP configured in .env)

**Setting up company branding for invoices:**

Admin → Org Settings tab → fill company details + upload logo → Save

**Understanding the Dashboard for management:**

- Stat cards: Total Devices, Online, Warning, Critical, Open Tickets
- Green numbers = healthy | Amber = attention needed | Red = action required

---

## Quick Reference Card — Administrators

**Access user management:**

Admin → Users tab

**View audit log:**

Admin → Audit Log tab → filter by action type and date range

**Check service health:**

Admin → System Info tab → Services card

**Deactivate a departed employee:**

Admin → Users tab → find user → Edit → Deactivate → Save

**Grant admin role (via database):**

```
UPDATE users SET role = 'admin' WHERE email = 'user@company.com';
```

**Service ports:**

- Dashboard: http://localhost:8501
- Flask API: http://localhost:5000
- PostgreSQL: localhost:5432
- Redis: localhost:6379

**Emergency service restart:**

```
# Kill API: netstat -ano | findstr :5000 → taskkill /F /PID <PID>
# Kill dashboard: netstat -ano | findstr :8501 → taskkill /F /PID <PID>
```

**Monthly security checklist:**

- [ ] Review Audit Log for unexpected DELETE events (use CSV export for records)
- [ ] Review Audit Log for unusual LOGIN IP addresses
- [ ] Verify all Users are current employees
- [ ] Confirm all admin accounts have MFA enabled (My Profile → MFA section)
- [ ] Check System Info → Services card for health
- [ ] Review Outstanding invoices in Billing
- [ ] Check Compliance % in OS Patches

---

## Quick Reference Card — Developers

**Dashboard entry point:** `dashboard/app.py` **All pages:** `dashboard/pages/01_Dashboard.py` through `16_Scripts.py` **API client:** `dashboard/utils/api_client.py` **Auth utilities:** `dashboard/utils/auth.py` **CSS/UI helpers:** `dashboard/utils/styles.py` **Flask routes:** `api/routes/` **Models:** `api/models/` **Celery tasks:** `api/tasks/` **Agent main loop:** `agent/rmm_agent.py`

**Brand colors (defaults — overridable via Admin → Org Settings → White-Label Branding):**

- Primary: `#407E3C` (green default; white-label replaces this system-wide)
- Accent: `#5a9e56`
- White: `#FFFFFF`
- Danger: `#EF4444`
- Warning: `#F59E0B`
- Success: `#22C55E`

**White-label branding utilities:**

- `dashboard/utils/branding.py` — `load_branding()` caches into `session_state["_branding"]` (5-min TTL); `fetch_branding_early()` is safe to call before `st.set_page_config`
- `inject_css()` in `styles.py` reads `session_state["_branding"]["primary_color"]` and string-replaces `#407E3C` across all CSS
- Public endpoint (no auth): `GET /api/admin/public/branding`

**Auth pattern in dashboard pages:**

```
from utils.auth import require_auth
client = require_auth() # Returns APIClient or redirects to login
```

**API call pattern:**

```
data, err = client.list_devices(per_page=100)
if err:
st.error(f"API error: {err}")
else:
devices = data.get("items", [])
```

**Currency and timezone formatting:**

```
from utils.formatters import fmt_currency, fmt_datetime_tz, CURRENCY_SYMBOLS

# Read org settings loaded once per session by require_auth()
_org = st.session_state.get("_org_settings", {})
_currency = _org.get("currency", "USD") # e.g. "EUR"
_tz = _org.get("timezone", "UTC") # e.g. "Europe/Berlin"

fmt_currency(25.0, _currency) # → "€25.00"
fmt_datetime_tz("2026-06-16T10:00:00Z", _tz) # → "2026-06-16 12:00" (Berlin=UTC+2)
```

`SUPPORTED_CURRENCIES` and `SUPPORTED_TIMEZONES` lists in `formatters.py` drive the Admin dropdowns — extend them there to add new options.

**Regional Settings API endpoint:** `PUT /api/org-settings` — accepts `currency` (str, 3-char ISO) and `timezone` (str, IANA name) alongside all other org settings fields.

**Alembic migration for currency/timezone:**
`api/migrations/versions/f1a2b3c4d5e6_currency_timezone_org_settings.py` (down_revision: `e6f7a8b9c0d1`)

**Agent heartbeat interval:** 60s (`config.ini → [agent] → heartbeat_interval`) **Software scan interval:** 21600s / 6h (`config.ini → [agent] → software_interval`) **Exit code convention:** 0 = SUCCESS, non-zero = FAILED

**Run tests:**

```
cd C:\RMM\RemoteManagementSystem\api
.\venv\Scripts\Activate.ps1
pytest tests/ -v --tb=short
```

Expected: 51 passed. CI runs this same command automatically on every push to `main` via `.github/workflows/ci.yml` (Python 3.11, sqlite:///:memory:, no external services needed).

**API input validation:**

All write endpoints are guarded by `@validate_body(Schema)` from `api/utils/validation.py`. Invalid requests return `400 {"error": "Validation failed", "details": {...}}` with field-level errors. Schemas live in `api/schemas/` — one file per domain (auth, agents, customers, devices, tickets, alerts, scripts, admin, automation, network, billing, org_settings). All schemas use `EXCLUDE` mode so partial PUT requests succeed.

**Load testing:**

```
# Edit ORG_TOKEN in tests/load/locustfile.py first
pip install locust
locust -f tests/load/locustfile.py --host http://localhost:5000
# Open http://localhost:8089 to start
```

Two user types: `AgentUser` (register + heartbeat loop, ~30s wait) and `DashboardUser` (login + browse). See `tests/load/README.md` for scenarios and results template.

---

## Quick Reference Card — WiFi Device Deployment

**Deploy agent on a WiFi/LAN machine (Windows/Linux/macOS):**

1. Get server LAN IP: Admin → System Info → Server IP Addresses
2. Copy `agent\` folder to target machine (USB, network share, or git clone)
3. On target machine:

```
cd C:\RMM\agent
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
python setup_agent.py 192.168.x.x <org_token>
```

Org token: Admin → System Info → Agent Enrollment Token → Reveal

1. Start the agent: `python rmm_agent.py`
2. Device appears in Devices tab within 60 seconds

**Discover phones/IoT devices (no agent possible):**

1. Network Discovery → enter subnet (e.g. `192.168.1.0/24`) → Scan Network
2. Review results — detected via OUI vendor, port probe, then hostname keywords (Samsung model names, brand names)
3. Click **Save All to Devices**
4. Devices appear in Devices → Android / iOS / Agentless tabs
5. System pings them every 5 minutes automatically

---

## Quick Reference Card — Superadmin Emergency Recovery

**Use this when:** All regular admin accounts are locked out, forgotten, or deactivated and you cannot log in to the web interface.

**Step 1 — Access the server machine directly** (physical or remote desktop to the RMM server).

**Step 2 — Open a terminal and reset the password:**

```
Set-Location C:\RMM\RemoteManagementSystem\api
.\venv\Scripts\Activate.ps1
python reset_superadmin.py YourNewPassword123
```

Password must be at least 10 characters. No API restart needed.

**Step 3 — Log in to the dashboard:**

- URL: http://localhost:8501
- Email: `superadmin@rmm.local` (or whatever is set in `SUPERADMIN_EMAIL` in `.env`)
- Password: the one you just set

**Step 4 — Recover access for regular admins:**

- Go to Admin → Users tab
- Re-activate or reset passwords for admin accounts as needed

**To permanently change superadmin credentials** (recommended after install):

1. Edit `.env` — set `SUPERADMIN_EMAIL` and `SUPERADMIN_PASSWORD`
2. Restart the Flask API: `cd api ; python app.py`

---

# PART XI — COMMERCIAL AUDIT ADDITIONS

# Chapter 45: API Documentation (Swagger/OpenAPI)

## What this is

The RMM system includes built-in, interactive API documentation accessible from any browser. This is primarily for developers and technically-minded administrators who want to explore or test API endpoints directly.

## How to access it

With the Flask API running, open:

```
http://localhost:5000/api/docs
```

You will see the SwaggerUI interface — a visual listing of every API endpoint, grouped by category. You can expand any endpoint to see its parameters, request body format, and response structure.

To test an endpoint from within the documentation:

1. Click **Authorize** at the top right.
2. Enter your JWT token (obtain one by logging in via `POST /api/auth/login`).
3. Click any endpoint, then click **Try it out** → fill in parameters → **Execute**.

## The raw spec

The machine-readable OpenAPI 3.0 specification is available at:

```
http://localhost:5000/api/openapi.json
```

This can be imported into Postman, Insomnia, or any other API testing tool.

## Coverage

The spec covers all 47 API paths across 16 categories: Auth, Agents, Devices, Customers, Alerts, Tickets, Scripts, Patches, Automation, Dashboard, Network, Terminal, Admin, Billing, Reports, and System.

> **NOTE:** No additional software needs to be installed. The documentation UI loads from a public CDN automatically.

---

# Chapter 46: Alert Webhooks — Slack, Teams, Custom

## What this is

In addition to email notifications, alert rules can now send real-time notifications to Slack channels, Microsoft Teams channels, or any custom HTTP endpoint when an alert fires.

## Setting up a webhook on an alert rule

When creating or editing an alert rule (Alerts → Alert Rules → Add Rule / Edit), the **Notification Channels** field accepts a JSON object. Add any combination of the following keys:

```
{
"email": ["ops@yourcompany.com", "oncall@yourcompany.com"],
"slack": "https://hooks.slack.com/services/T.../B.../...",
"teams": "https://yourcompany.webhook.office.com/webhookb2/...",
"webhook": "https://your-endpoint.com/rmm-alerts"
}
```

All keys are optional. Include only the channels you want.

## Getting a Slack webhook URL

1. Go to **api.slack.com/apps** → create an app → **Incoming Webhooks** → activate → add to channel → copy the URL.

## Getting a Teams webhook URL

1. In Teams, right-click a channel → **Connectors** → **Incoming Webhook** → configure → copy the URL.

## Custom webhook format

Your endpoint will receive a POST with JSON body:

```
{
"event": "rmm_alert",
"rule": "High CPU Usage",
"device": "WORKSTATION-01",
"severity": "warning",
"message": "CPU usage at 94% for 5 minutes",
"timestamp": "2026-07-13T14:32:00Z"
}
```

### Severity colours (Slack / Teams)

| Severity | Colour |
|---|---|
| critical | Red `#FF3B30` |
| warning | Orange `#FF9500` |
| info | Blue `#007AFF` |

> **NOTE:** Webhook delivery failures are logged server-side and never block alert processing. If your webhook endpoint is down, the alert still fires and email is still sent.

---

# Chapter 47: Configurable SLA Policies

## What SLA policies are

A Service Level Agreement (SLA) policy defines how quickly a ticket must be resolved based on its priority. For example: a critical ticket must be resolved within 4 hours; a low-priority ticket within 72 hours.

The system uses these policies to automatically calculate the **due date** on every new ticket.

## Default global policies

The following defaults are seeded automatically at installation:

| Priority | Response Target | Resolution Target (Due Date) |
|---|---|---|
| Critical | 1 hour | 4 hours |
| High | 4 hours | 8 hours |
| Medium | 8 hours | 24 hours |
| Low | 24 hours | 72 hours |

These global defaults apply to all customers unless a customer-specific override is set.

## Setting a customer-specific SLA policy

You can configure different SLA targets for individual customers — for example, a premium customer may have a 1-hour resolution SLA on high-priority tickets.

Via the API (admin role required):

```
POST /api/sla-policies/
{
"customer_id": "<customer UUID>",
"priority": "high",
"response_hours": 2,
"resolution_hours": 4
}
```

The system then uses the customer-specific policy for tickets belonging to that customer, falling back to the global default for priorities that have no customer override.

## Viewing and updating policies

```
GET /api/sla-policies/?customer_id=<id> # shows both global + customer-specific
PUT /api/sla-policies/<policy_id> # update hours
DELETE /api/sla-policies/<policy_id> # delete customer-specific (global protected)
```

**NOTE:** Deleting a customer-specific policy reverts that customer to the global default — it does not delete the global policy.

### How due dates are calculated

When a ticket is created, the system calculates the due date as:

```
due_date = ticket_created_at + resolution_hours
```

Where `resolution_hours` comes from (in priority order):

1. Customer-specific SLA policy for that priority
2. Global SLA policy for that priority
3. Built-in hardcoded defaults

If a technician manually sets a `due_date` in the ticket creation form, it overrides the SLA calculation entirely.

---

# Chapter 48: Billing Automation — Recurring Invoices

## What billing automation does

If you bill clients monthly based on the number of devices they have enrolled, the system can automatically generate draft invoices on a schedule — eliminating the need to manually create invoices each month.

## Setting up a billing profile for a customer

Update the customer record with billing profile information (Admin → Customers → edit customer, or via API):

| Field | Description | Example |
|---|---|---|
| `billing_day` | Day of month (1–28) to generate invoice | `1` (1st of each month) |
| `per_device_rate` | Charge per managed device (USD) | `15.00` |
| `tax_rate` | Tax rate as decimal | `0.08` (8%) |

Via the API:

```
PUT /api/customers/<id>
{
"billing_day": 1,
"per_device_rate": 15.00,
"tax_rate": 0.08
}
```

## What happens automatically

Every day at the time the Celery beat scheduler runs, the system checks which customers have `billing_day` equal to today's date. For each qualifying customer:

1. Counts all currently managed devices
2. Calculates: `subtotal = device_count × per_device_rate`
3. Calculates: `tax = subtotal × tax_rate`
4. Creates a **draft invoice** for the previous calendar month
5. Invoice number assigned automatically (e.g. `INV-2026-0042`)

The invoice is created in **Draft** status — it is not sent automatically. A technician or admin reviews it in **Billing** → **Invoices**, then clicks **Send** or **Send Email** when ready.

## Safeguards

- If an invoice for the same customer and billing period already exists, the task skips it — no duplicate invoices are ever created.
- If the Celery worker is offline on the billing day, the invoice will not be generated retroactively — the task only fires when the scheduler is running.

## Reviewing auto-generated invoices

1. Go to **Billing** → **Invoices**
2. Filter by customer or status = **Draft**
3. Review line items, adjust if needed
4. Click **Mark as Sent** or **Send Email** (if SMTP is configured)

---

# Chapter 49: Database Backup and Recovery

## Why this matters

The PostgreSQL database contains all your device history, tickets, alerts, invoices, and customer records. A disk failure without a backup means permanent data loss.

The system runs an automatic nightly backup of the entire database.

## How the backup works

Every 24 hours, a background Celery task:

1. Finds the `pg_dump` tool (auto-detected from standard PostgreSQL install paths)
2. Dumps the full database in SQL format
3. Compresses it with gzip
4. Saves to the backup directory as `rmmdb_YYYYMMDD_HHMMSS.sql.gz`
5. Removes backup files older than the retention period

## Configuration (`.env`)

```
BACKUP_DIR=C:\RMM\backups # where to save backup files (default: ../backups)
BACKUP_RETAIN_DAYS=7 # how many days to keep (default: 7)
```

**TIP:** Point `BACKUP_DIR` at a network drive or cloud-synced folder for off-site protection.

## Manually triggering a backup

From a terminal with the Celery worker running:

```
# From inside the api/ directory with venv active:
python -c "from tasks.backup_tasks import backup_database; backup_database.delay()"
```

Or simply wait — the task runs automatically every 24 hours.

## Restoring from a backup

**WARNING:** Restoring a backup overwrites all current data. Only do this if the database is corrupted or data is lost.

1. Stop the Flask API and Celery workers.
2. Open a terminal on the RMM server.
3. Run:

```
# List available backups
Get-ChildItem C:\RMM\backups\*.sql.gz | Sort-Object LastWriteTime -Descending

# Restore (replace filename with your chosen backup)
& "C:\Program Files\PostgreSQL\15\bin\pg_restore.exe" `
--username rmm_app --dbname rmmdb --clean `
```

```
C:\RMM\backups\rmmdb_20260713_020000.sql.gz
```

Or using Linux/WSL:

```
gunzip -c backups/rmmdb_20260713_020000.sql.gz | psql -U rmm_app -d rmmdb
```

1. Restart the Flask API and Celery workers.

> **NOTE:** The backup includes all data — devices, tickets, users, invoices, and audit logs. It does not include uploaded files, reports, or the `.env` file (those must be backed up separately).

### What happens if `pg_dump` is not found

The backup task logs an error and skips the run. It will try again the next day. To fix: ensure the PostgreSQL `bin` directory is in your system PATH, or the backup task will detect it automatically if PostgreSQL is installed in the standard location (`C:\Program Files\PostgreSQL\<version>\bin\`).

---

# Chapter 50: GDPR Compliance — Data Export and Erasure

## Overview

The system provides two admin-only tools to support compliance with data privacy regulations (GDPR Article 17 "Right to Erasure" and Article 20 "Right to Data Portability").

> **IMPORTANT:** These actions are irreversible. Only an administrator should perform them, and only in response to a verified request from the data subject.

## Exporting a user's personal data (Article 20)

A user has the right to request a copy of all personal data the system holds about them.

**Via API (admin JWT required):**

```
GET /api/admin/users/<user_id>/gdpr-export
```

The response is a JSON document containing:

- Complete user profile (name, email, role, created date, settings)
- Up to 1,000 audit log entries linked to this user (actions they performed)
- Up to 1,000 ticket comments they authored

This JSON can be downloaded and provided to the user.

The export action is itself recorded in the audit log (action: `GDPR_EXPORT`).

## Erasing a user's personal data (Article 17)

> **WARNING:** This action is **permanent and irreversible**. The user's personal data is anonymised in-place. It cannot be undone.

**Via API (admin JWT required):**

```
DELETE /api/admin/users/<user_id>/gdpr-delete
```

What is anonymised:

| Data | What happens |
|---|---|
| Email address | Replaced with `__anon__<id>@deleted.local` |
| Full name | Replaced with `Deleted User <id>` |
| Password | Cleared |
| Profile photo | Removed |
| Account status | Set to inactive |
| IP addresses in audit log | Replaced with `0.0.0.0` |
| Author email on ticket comments | Removed |

What is **not** removed:

- Audit log entries themselves (the actions were real system events) — only the IP address is scrubbed
- Ticket comments they authored — the comment text remains, but authorship is anonymised
- Invoices (financial records must be retained for legal reasons)

**Protected accounts:**

- The superadmin account cannot be anonymised (returns 403)
- An admin cannot anonymise their own account (returns 400)

## Workflow recommendation

When you receive a data erasure request:

1. Log in as admin
2. Find the user under **Admin** → **Users** — note their user ID
3. First perform the export (`GET /gdpr-export`) and save the file as a record
4. Confirm with the requester and get written consent if required
5. Perform the erasure (`DELETE /gdpr-delete`)
6. Confirm to the requester that data has been erased
7. Retain your own records of the request and erasure for compliance purposes

---

# PART XII — NEW FEATURES AND DEPLOYMENT OPTIONS

# Chapter 51: React Frontend — Alternative Web Interface

## What it is

In addition to the Streamlit dashboard, the system ships a full React/TypeScript frontend in the `frontend/` directory. It covers all the same pages as the Streamlit dashboard and communicates with the same Flask API using the same JWT authentication.

You can run both simultaneously — the Streamlit dashboard on port 8501 and the React frontend on port 3000 — or choose one as your primary interface.

## When to use the React frontend

| Situation | Recommendation |
|---|---|
| Staff using modern browsers on fast networks | Either works |
| Embedding the UI in another web product | React (easier to customise) |
| Rapid internal deployment with minimal setup | Streamlit |
| Custom branding or white-label delivery | React |
| IoT/mobile-optimised views needed | React |

## Installing and starting the React frontend

**Prerequisites:** Node.js 20 or later. Verify with:

```
node --version
npm --version
```

### Step 1: Install dependencies

```
Set-Location C:\RMM\RemoteManagementSystem\frontend
npm install
```

### Step 2: Configure API URL

The React frontend reads the API base URL from the Vite environment. In `frontend/`, create a `.env.local` file:

```
VITE_API_BASE_URL=http://localhost:5000
```

### Step 3: Start the dev server

```
npm run dev
```

Open **http://localhost:3000** — you will see the RMM login page.

### Step 4: Production build

```
npm run build
```

Output is in `frontend/dist/`. Serve with any static file server (nginx, Caddy, etc.).

### CORS setting

When using the React frontend, set `CORS_ORIGINS` in `api/.env` to include the frontend URL:

```
CORS_ORIGINS=http://localhost:3000,http://localhost:8501
```

### Login and navigation

Login, MFA, and all role-based access rules work identically to the Streamlit dashboard. The React frontend stores tokens in `localStorage` rather than URL parameters.

---

# Chapter 52: Agent Binary Deployment (No Python Required)

## What it is

The standard agent deployment requires Python 3.11 to be installed on every managed machine. For environments where installing Python is not possible (locked-down machines, USB deployment, air-gapped networks), the agent can be compiled into a single self-contained executable using PyInstaller.

## Building the binary

Run this on the **same platform** you want to deploy on. A Windows build must be done on Windows; a Linux build on Linux; a macOS build on macOS.

```
# On the RMM server or any machine with Python + the agent code
Set-Location C:\RMM\RemoteManagementSystem\agent
.\venv\Scripts\Activate.ps1
python build.py
```

The build script:

1. Installs PyInstaller automatically if not present
2. Packages `rmm_agent.py` and all dependencies into a single file
3. Copies `config.ini` template next to the binary

Output:

```
agent\dist\rmm_agent.exe (Windows)
agent/dist/rmm_agent (Linux / macOS)
agent/dist/config.ini (template — edit before deploying)
```

## Deploying the binary

1. Copy the entire `dist\` folder to the target machine (USB drive, network share, etc.)
2. Edit `dist\config.ini` — set `url` and `org_token` (same as standard deployment)
3. Run the binary as Administrator:

```
rmm_agent.exe
```

1. The device registers and appears in the Devices page within 60 seconds.

**NOTE:** The binary does not need Python, pip, or any other runtime. It is fully self-contained.

**TIP:** The binary can be added to Windows Task Scheduler for auto-start on reboot, using the same method described in Chapter 6, Step 6.

## Platform limitations

| Feature | Binary (Windows) | Binary (Linux/macOS) |
|---|---|---|
| CPU/RAM/disk metrics | Full | Full |
| Software inventory | Registry + winget | dpkg/rpm/brew |
| Patch reporting | WUA | softwareupdate/apt/dnf |
| Screenshot capture | Full (Pillow bundled) | Full (scrot required on Linux) |
| DPAPI token encryption | Full | N/A (plaintext) |
| Terminal worker | Full | Full |

# Chapter 53: Remote Screenshot Viewer

## What it is

Each agent-managed device automatically captures a screenshot of its current screen every 5 minutes and uploads it to the RMM server. This allows you to see what is happening on a device without starting a full remote desktop session.

## Who can use it

Technicians and administrators.

## Viewing a screenshot

**From the Streamlit dashboard:**

1. Click **Devices** in the sidebar.
2. Find the device under Windows, macOS, or Linux tab.
3. Expand the device row.
4. Click **View Screenshot** — the latest capture is displayed inline.
5. The screenshot timestamp is shown below the image.

**From the React frontend:**

The device detail view shows the latest screenshot automatically if one is available.

**Via API:**

```
GET /api/devices/<device_id>/screenshot
Authorization: Bearer <access_token>
```

Returns the image as `image/jpeg`. Returns HTTP 404 if no screenshot has been captured yet.

## Screenshot frequency

The default capture interval is **5 minutes** (300 seconds). This can be changed per agent by editing `agent/config.ini`:

```
[agent]
screenshot_interval = 300
```

Set to `0` to disable screenshot capture on a specific agent.

## Privacy considerations

Screenshots capture the entire screen of the managed device. On devices used by end users (not just servers), inform users that screen capture is active as part of your acceptable-use policy.

Screenshots are stored server-side as `api/screenshots/<device_id>.jpg`. Only the most recent screenshot per device is retained — previous captures are overwritten automatically.

**NOTE:** Screenshot capture returns `None` silently on headless servers with no display attached. No error is logged at INFO level — the heartbeat loop continues normally.

---

# Chapter 54: IoT Sensor Monitoring

## What it is

The IoT Sensor Agent is a lightweight variant of the standard agent designed for Raspberry Pi and other Linux single-board computers. It reads from hardware sensor backends and sends readings to the RMM API for storage and display.

## Who uses it

Technicians and administrators managing physical environments alongside IT equipment (temperature monitoring, door sensors, motion detection, etc.).

## Supported sensor types

| Sensor | Hardware | Data reported |
|---|---|---|
| CPU/board temperature | `/sys/class/hwmon` (built-in, no deps) | Temperature in °C |
| DHT11 / DHT22 | GPIO — `adafruit-dht` library | Temperature + humidity |
| BME680 | I2C — `adafruit-circuitpython-bme680` | Temperature, humidity, pressure, gas |
| PIR motion sensor | GPIO — `gpiozero` library | Motion detected (boolean) |
| Door reed switch | GPIO — `gpiozero` library | Open/closed state |

All sensor libraries are optional. Missing libraries are silently skipped — the agent reports only from available backends.

## Deploying the IoT agent

1. Copy the `agent/` folder to the Raspberry Pi.
2. Install dependencies:

```
pip install -r requirements.txt
# For DHT sensors: pip install adafruit-dht
# For BME680: pip install adafruit-circuitpython-bme680
# For GPIO sensors: pip install gpiozero
```

1. Register with the server:

```
python setup_agent.py <server_ip> <org_token>
```

1. Edit `config.ini` — add the `[iot]` section:

```
[iot]
interval = 60
dht_type = DHT22
dht_pin = 4
pir_pin = 17
door_pin = 27
```

1. Start the IoT agent:

```
python iot_agent.py
```

### Viewing sensor data

1. Click **IoT Sensors** in the sidebar (Admin/Tech access).
2. Select a device from the dropdown.
3. Time-series charts display each sensor type separately.
4. The latest reading, unit, and channel are shown for each sensor.

### MQTT ingestion (optional)

If you already have an MQTT broker (Mosquitto, HiveMQ, etc.) receiving sensor data, the RMM can subscribe to it directly without deploying the IoT agent on each device. Set in `api/.env`:

```
MQTT_HOST=192.168.1.x
MQTT_PORT=1883
MQTT_TOPIC_PREFIX=rmm
```

Topic convention: `rmm/{device_id}/sensors/{sensor_type}` Payload: `{"value": 23.5, "unit": "°C", "channel": null}`

The device must already be registered in the RMM (agent or agentless) for readings to be associated correctly.

---

# Chapter 55: Mobile Device Management — Managing Phones Without an Agent

## What it is

Real management of phones on the WiFi network — lock, wipe, "lost mode," and compliance status — without writing or installing a custom agent app. Android phones run Google's own signed "Android Device Policy" app instead, which is provisioned automatically the moment the phone's owner scans a QR code or opens an enrollment link. Nothing happens on the phone silently — that step is always required, both because Google's platform demands it and because managing someone's personal phone without their agreement is not something this system will do.

## Who uses it

Admins/technicians enrolling company-owned phones for full management, and customers enrolling their own personal phone (BYOD) through the self-service Client Portal, with an explicit on-screen consent step first.

## The two ownership modes

| Mode | Who sets it up | What it covers | What it can never touch |
|---|---|---|---|
| **Company-owned (corporate)** | Admin/Technician, via Mobile Enrollment page | The entire device | — |
| **Personal (BYOD)** | The customer themselves, via the Client Portal | Only a separate "work profile" container on the phone | Personal apps, photos, messages, and browsing outside the work profile |

BYOD enrollment can always be undone from either side — the customer can remove the work profile from their phone's own Settings at any time, or a technician can revoke it, and either way the personal side of the phone is untouched.

## One-time setup (before any phone can be enrolled)

This part happens outside the RMM, in Google's own systems, and only needs to be done once per organization:

1. Create (or pick) a project at console.cloud.google.com and enable the **Android Management API**.
2. Under **IAM & Admin** → **Service Accounts**, create a service account and download its JSON key file. Keep this file safe — treat it like a password.
3. Have a real, working **HTTPS** address for this RMM server reachable from a browser (a tunnel tool like ngrok is fine for testing — Google will not accept `localhost`).

## Setting up an integration (Admin/Technician)

1. Click **Mobile Enrollment** in the sidebar → **Android** tab.
2. Under **Add integration**, enter a name (e.g. "Acme Corp Android Fleet"), your Google Cloud project ID, and whether this integration is staff-wide or scoped to one customer. Click **Create integration**.
3. Open the new integration's entry, upload the service-account JSON key file downloaded above.
4. Enter your public HTTPS base URL and click **Start binding**. A link appears — open it in a browser and complete Google's own hosted business sign-up page. When it redirects back, the integration shows **Bound**.
5. (Optional) Paste a policy JSON in the box provided and click **Push policy** to set passcode requirements, allowed apps, etc. for every phone enrolled under this integration.

## Enrolling a company-owned phone

1. On the **Mobile Enrollment** page, under **Enroll a phone**, pick the integration, choose **Company-owned**, and click **Generate enrollment QR**.
2. A QR code and a plain link both appear.
3. On a factory-reset (or freshly wiped) Android phone, at the very first setup screen, scan the QR code — or open the link if you're setting the phone up in front of you.
4. The phone silently installs Google's Device Policy app and finishes provisioning itself. No further action is needed.
5. Within 5 minutes (the automatic sync interval), the phone appears in **Devices** → **Android** with working Lock / Lost Mode / Reboot / Reset Passcode / Wipe buttons.

## Enrolling a personal phone (BYOD, customer self-service)

1. The customer logs into their **Client Portal** (Support Portal) and opens **+ Enroll My Phone**.
2. They read the consent notice — it explains in plain language exactly what will and won't be visible or controllable, and they must tick the consent box before anything is generated.
3. They click **Generate enrollment QR**. If they're viewing the portal on a computer, they scan the QR with their phone's camera; if they're already on the phone, they tap the **Open enrollment link on this phone** button instead.
4. Android creates a separate **work profile** on their phone (shown with a small briefcase badge on its apps) — their personal side of the phone is never touched.
5. The device then shows up for that customer with self-service Lock and Lost Mode buttons — Wipe and other administrative actions stay with staff.

## Using the remote actions

From **Devices** → **Android** (or, for a customer, their own device in the Client Portal), each managed phone has:

| Button | What it does | Who can use it |
|---|---|---|
| **Lock** | Locks the phone immediately | Admin, Technician, or the customer on their own enrolled phone |
| **Lost Mode** | Puts a message and contact number on the lock screen for whoever finds it | Admin, Technician, or the customer on their own enrolled phone |
| **Reboot** | Restarts the phone | Admin, Technician |

| Button | What it does | Who can use it |
|---|---|---|
| **Reset Passcode** | Forces a new device passcode | Admin, Technician |
| **Wipe** | Erases the device (or, for BYOD, just the work profile) — asks for a second confirmation click first | Admin only |

**NOTE:** Commands are not instant. They're delivered the next time the phone checks in with Google, which is usually within a few minutes. The button confirms the command was *sent*, not that it has *finished*.

### Removing a phone from management

From the **Enrollments** list on the Mobile Enrollment page (or from the customer's own portal for their own BYOD device), click **Revoke**. For a personal (BYOD) phone this removes only the work profile — the owner's personal data, apps, and photos are never affected. For a company-owned phone this removes it from management entirely.

### iOS (iPhone) status

Not available yet, and this is a business decision rather than something that can simply be built: Apple only issues the certificate this kind of management requires (an APNs push certificate) to organizations enrolled in **Apple Business Manager**, or to already-approved MDM vendors. iPhones continue to show up under **Devices** → **iOS** the same way they always have — discovered on the network, but without remote lock/wipe/lost-mode actions — until a decision is made to either enroll in Apple Business Manager or connect an existing vendor-signed MDM server (such as Fleet or MicroMDM) to this system.

---

# PART XIII — GOVERNANCE AND COST CONTROL

# Chapter 56: API & Token Usage Monitoring — Finding and Controlling Runaway Usage

## What it is

A read-only monitoring page that shows exactly how many API calls and AI tokens the system is using, broken down by service and feature, with an estimated dollar cost for the AI Assistant's usage. It was added because usage had grown unexpectedly high and there was no way to see what was driving it. It is strictly limited to the **Super Administrator** account — even a regular Admin cannot see it or know it exists.

## Who uses it

Only the Super Administrator. This is deliberate: token/API usage and its cost are treated as sensitive operational information, one level above what a regular Admin sees.

## Where to find it

Log in as the Super Administrator account. A new **SUPERADMIN** section appears at the bottom of the sidebar with one link: **Usage Monitoring**. No other role sees this section at all — it does not appear greyed-out or hidden-but-clickable, it simply is not in their sidebar.

## What the page shows

- **Totals** for Today / 7 days / 30 days: total API calls, total AI tokens used, estimated dollar cost, and error rate.
- **By Service** — a table/chart breaking usage down by exactly what generated it: the AI Assistant, Stripe billing, ConnectWise/Autotask PSA sync, Android device management, outbound Slack/Teams/webhook notifications, email, network scans, and all internal API traffic (from the dashboard, the agents, and the React app).
- **Trend chart** — the same numbers over time, so a sudden jump is easy to spot visually.
- **Top Features & Users** — which specific page, integration action, or staff member is responsible for the largest share of calls or tokens.
- **Spike alert banner** — if any service's usage in the last hour is unusually high compared to its normal pattern over the past week, a red banner appears at the top of the page automatically.

## Setting up real spike alerts

By default the system only shows spikes visually on this page. If you also want a real email or Slack/Teams notification the moment a spike happens (for example, overnight when nobody is watching the dashboard):

1. Open **Usage Monitoring** → **Spike Alert Configuration**.
2. Turn on **Send real notifications on usage spikes**.
3. Set the **spike multiplier** — how many times above the normal average counts as a spike (3× is a sensible starting point).
4. Enter the email address(es) and/or Slack/Teams webhook URL(s) that should be notified.
5. Click **Save**.

This check runs automatically once an hour. It reuses the exact same notification system that device alerts already use — it just does not create a "device alert," since a usage spike is not tied to any one device.

## Getting a downloadable report

Click **Generate API & Token Usage Report** on the Usage Monitoring page. Like every other report in this system, it is generated in the background and appears in **Reports** → **Report History** shortly afterward as a downloadable CSV — broken down by day and by service. This report type, like the page itself, only appears for the Super Administrator; it does not show up in the report list for anyone else, even if they know the report exists.

## What this does and does not track

**Does track:** every AI Assistant conversation's token usage and an estimated cost, every call this system makes out to Stripe/PSA/Android device management/webhooks/email/network scanning, and the overall volume of every request hitting the API (from the dashboard, agents, and the React frontend).

**Does not track:** the contents of any conversation or request, any API key, password, or other secret, or anything beyond a short error message when something fails. Nothing sensitive is ever stored by this feature — only counts, timings, and which service/page/user was involved.

**Cost figures are estimates, not bills.** The dollar amount shown is calculated from a configurable price-per-token setting in the system's configuration file, not from an actual invoice — it is meant to give a sense of scale, not to be used for accounting.

---

# PART XIV — QUALITY, TESTING, AND SECURITY

# Chapter 57: Consistent List Pagination

## Who uses this chapter

Developers integrating with the API — any page, script, or third-party tool that reads a paginated list endpoint (users, alert rules, automation profiles, customer groups, patches, script runs).

## What changed

Nine list endpoints previously each hand-implemented their own paging logic, with small inconsistencies between them — most notably, the admin user-list endpoint returned its collection under the key `"users"` while every other list endpoint used `"items"`. That mismatch had silently broken pagination on the React Admin page, which was reading `data.items` and getting nothing back. All nine now go through one shared helper (`api/utils/pagination.py`) and return the same shape:

```
{"items": [...], "total": 137, "page": 1, "pages": 3}
```

## Practical impact

If you have any script or integration that reads `/api/admin/users` (or any other affected list endpoint) and was working around the old `"users"` key, update it to read `"items"` instead. Every other affected endpoint's shape is unchanged — this only fixes the one that was inconsistent.

---

# Chapter 58: Automated Test Coverage — What's Verified on Every Change

## Who uses this chapter

Anyone deciding whether a change is safe to deploy, and any developer picking up this codebase who wants to know what's actually checked before code reaches `main`.

## What's now covered

As of the latest audit-remediation pass, every part of this system has automated tests that run before any change can be merged:

| Part of the system | Tests | What it means in practice |
|---|---|---|
| Flask API (every route + every background job) | 490 tests, 72% of all code lines executed | Every user-facing endpoint and every scheduled/background task (alerts, billing, backups, patch deployment, PSA/MDM sync, etc.) has dedicated tests |
| Streamlit dashboard (every page) | 84 tests | Every one of the 26 dashboard pages loads and behaves correctly under test |
| React frontend (every page) | 39 tests | Every one of the 19 React pages has its own test |
| End-to-end browser tests | 2 tests | A real login-and-use-the-app test run against the live, fully running system |
| Agent | 19 tests | Core agent logic (not the Windows-only system calls, which can't run in an automated test environment) |

## Why this matters for you

Before this pass, roughly half the system's code — including the entire background-job engine that runs alerts, billing, backups, and patch deployment — had no automated tests at all, meaning a change there could silently break something and nobody would find out until a customer did. That gap is now closed. A CI check also fails the build if test coverage on the API ever drops below 70%, so it can't silently regress again.

## A real problem this work found

While writing these tests, a genuine data-privacy bug was discovered and fixed in the dashboard: under specific conditions, one user's dashboard data (devices, customers, alerts) could be briefly served to a different user logged into the same server. This is explained in full in Chapter 59 below — it's flagged here too because it was found specifically *because* new tests were being written, which is the strongest practical argument for why this chapter's work matters.

---

# Chapter 59: Security Hardening — What Changed and Why

## Who uses this chapter

Everyone — this chapter explains, in plain terms, what security issues were found and fixed in the most recent audit, and what (if anything) staff need to do differently.

## The most serious issue: a data mix-up between users (now fixed)

**What happened:** For a period of time, the dashboard had a bug where, in rare circumstances, one logged-in user's data — their dashboard summary, device list, customer list, or alerts — could be shown to a *different* user who loaded the same page shortly afterward, for up to about two minutes. This was caused by a caching mistake, not an intentional data-sharing feature, and it has been fixed and verified.

**What you need to do:** Nothing — the fix is already deployed. If you noticed dashboard numbers that seemed to belong to a different customer or account at some point, this explains why, and it will not happen going forward.

## A page that could be tricked into running injected code (now fixed)

The Devices page had one spot where a device's name or other details, if crafted maliciously, could run unwanted code in the browser of whoever viewed that device next. This required someone to be able to register a device or edit an agentless device's name in the first place — it was not exploitable by an outside attacker with no access at all. It is fixed: all device fields are now safely escaped before display, same as every other page in the system.

## Login credentials are now better protected

- **Two-factor authentication codes** are now encrypted in the database, not stored in plain text.
- **Password-reset links** can now only be used once — previously, a reset link stayed valid for its full one-hour window even after being used, which mattered only if the link leaked a second time (e.g. from browser or email history).
- **Stored integration credentials** (used for billing, PSA, and mobile device management integrations) use a more clearly separated encryption key than before.

## The API now sends standard browser security headers

Every response from the system now tells the browser to be stricter about what it will execute and display — standard practice that most modern web applications already do. This is invisible to normal use; it only matters if someone were trying to attack the system through the browser.

## Known software vulnerabilities — patched

Every third-party software library this system depends on was checked against the public vulnerability database. 51 out of 55 known issues were fixed by upgrading to newer, compatible versions of those libraries — no functionality changed as a result. One issue was found to require a larger rewrite of the SNMP (industrial device monitoring) component to fix properly rather than a simple version bump; it has been documented and left as a tracked, deliberate exception rather than silently ignored, and its practical

risk is limited to devices already reachable on a customer's own monitored network. The automated build process now fails immediately if any *new* vulnerability appears in the future, so this can't silently go stale again.

### Bottom line

None of this requires any action from regular staff. If you are a developer or administrator responsible for deploying this system, see `audits/security_audit.md` and `TECHNICAL_GUIDE.md` Chapter 22 for the full technical detail behind each item above.

---

*End of RMM System Complete Handbook — Version 10.0*

*For support with this guide, contact your system administrator or development team.*